



Programming with Python

AAPP015-4-1-PWP

NAME	TP NUMBER
YEO PEI WEN	TP077057
YAP XIN LING	TP077224
SIM TIAN	TP077056

Intake Code: UCDF2308ICT(SE)

Course Name: Programming with Python

Lecturer: Amardeep Singh A/L Uttam Singh

Table of Contents

Introduction.....	5
Assumptions.....	6
Source Code with Explanation.....	78
Import Files	78
Log Functions.....	79
check_password Function	80
check_username Function	80
homepage Function	81
generate_service_id Function.....	83
generate_service_id Function.....	83
CustomerSignIn Function	84
StaffLogIn Function	84
reset_customer_password Function	12
reset_staff_password Function-Part1	12
reset_staff_password Function-Part2	13
reset_staff_password Function-Part3	14
reset_staff_password Function-Part4	15
reset_staff_password Function-Part5	15
reset_staff_password Function-Part6	16
is_user_pending Function	86
verify_customer_credentials Function	86
verify_Staffcredentials Function	86
CustomerSignUp Function.....	88
StaffSignUp Function-Part1	88
StaffSignUp Function-Part2	89
StaffSignUp Function-Part3	90
StaffSignUp Function-Part4	91
save_pendingCust Function	91
save_pendingStaff Function	91
check_ic Function	93
CustomerMenu Function.....	94
Staffhomepage Function.....	94

load_data and save_data Function.....	95
get_item and get_item_price Function.....	97
get_service and get_service_price Function	98
initialize_files Function-Part1	98
update_stock Function.....	101
save_orders Function.....	101
save_purchase_order Function.....	101
save_service_order Function.....	102
save_purchase_record Function	102
get_stock Function	102
modify_order Function-Part1	103
inventory_modify_order Function-Part1.....	103
modify_order Function-Part2.....	104
inventory_modify_order Function-Part2.....	105
modify_order Function-Part3	106
modify_order Function-Part4.....	107
inventory_modify_order Function-Part5.....	107
cancel_order Function	109
inventory_cancel_order Function.....	109
inventory_stock_check Function-Part1	111
inventory_stock_check Function-Part2.....	112
inventory_stock_check Function-Part3.....	114
inventory_stock_check Function-Part4.....	115
inventory_stock_check Function-Part5	116
place_purchase_order Function-Part1	117
create_service_order Function-Part1.....	118
inventory_purchase_order Function-Part1	118
place_purchase_order Function-Part2.....	119
create_service_order Function-Part2.....	120
inventory_purchase_order Function-Part2	120
inventory_purchase_order Function-Part3	122
generate_invoice Function	123
check_order_details Function	123

load_orders_details Function	124
inquiry_of_order_status Function	125
inventory_check_purchase_order_status Function	125
service_order Function-Part1	128
service_order Function-Part2	129
service_order Function-Part3	130
service_order Function-Part4	131
make_payment Function-Part1	131
inventory_payment Function-Part1	132
inventory_payment Function-Part2	133
inventory_payment Function-Part3	134
customer_reports Function.....	135
staff_view_reports Function.....	135
staff_view_service_orders Function-Part1	136
staff_view_service_orders Function-Part2	137
customer_interface Function	138
staff_menu Function	139
staff_menu Function-Part2	140
Additional features source code	146
6.2 Collect Feedback	151
Conclusion.....	172
References.....	173

Introduction

KL Central Computer Company (KLCCC) utilise an automated system to manage rapidly expanding computer sales and repair orders. This report focuses on designing and implementing a system using Python programming language that works automatically to handle the rise in business transactions. Python is a high-level programming language that is commonly used to integrate systems effectively Click or tap here to enter text.. By writing Python software solutions, all the transactions of different roles such as customers, admin staff, super user, and inventory staff can be efficiently maintained. Python is a reliable programming language that is easy to read and comprehend for programmers. It easily handles data, variables, and functions of the system and able to eliminate ambiguity (McKinney, 2022). The system facilitates an effortless shopping experience for the customers while meeting the evolving demands of the company. Detailed pseudocode and flowchart were created in the design phase of the system. In the implementation stage of the system, accurate code written in Python aims to fulfil the needs of system users. This report will explain how the Python code works as well as discusses the additional features that improve user experience.

Assumptions

The system created by Python fully satisfied all the needs of the assignment question and it enables KL Central Computer Company (KLCCC) to manage all of the transactions of the company as well as customers. In this system, each user of the system can only own an account and has a role that defines their access and permissions within the system, for instance, each role like customer, admin, super user, or inventory staff has a distinct set of functionalities. As for purchase orders, both customers and inventory staff can place multiple items in one order number. Users are considered forgotten their password if they fail to log in to the system after several attempts and are redirected to the reset password page.

Pseudocode

Homepage

```
START

FUNCTION homepage()
    WHILE True
        OUTPUT "Welcome to KL Central Computer Company (KLCCC) "
        OUTPUT "Please select a role to access to the system"
        OUTPUT "1. Customer"
        OUTPUT "2. Admin"
        OUTPUT "3. Super User"
        OUTPUT "4. Inventory Staff"
        OUTPUT "Please enter your role: "
        INPUT choice
        IF choice == 1 THEN
            CALL FUNCTION CustomerMenu()
            RETURN role = "customer"
        ELSEIF choice == 2 THEN
            CALL FUNCTION AdminMenu()
            RETURN role = "admin"
        ELSEIF choice == 3 THEN
            CALL FUNCTION SuperUserMenu()
            RETURN role = "super user"
        ELSEIF choice == 4 THEN
            CALL FUNCTION InventoryStaffMenu()
            RETURN role = "inventory staff"
        ELSE
            OUTPUT "Invalid input. Please try again."
        ENDIF
    ENDWHILE
ENDFUNCTION

CALL homepage()
```

Customer Menu

```
FUNCTION CustomerMenu()
    WHILE True
        OUTPUT "1. Sign In."
        OUTPUT "2. Register"
        OUTPUT "3. Exit"
        OUTPUT "Please enter your choice: "
        INPUT choice
        IF choice == 1 THEN
            CALL FUNCTION CustomerSignIn()
        ELSEIF choice == 2 THEN
            CALL FUNCTION CustomerSignUp()
        ELSEIF choice == 3 THEN
            CALL FUNCTION homepage()
        ELSE
            OUTPUT "Invalid input. Please try again."
            CALL FUNCTION CustomerMenu()
        ENDIF
    ENDWHILE
ENDFUNCTION
```

Customer Sign In Function

```
FUNCTION CustomerSignIn()
    SET max_attempts = 3
    SET attempts = 0

    WHILE attempts < max_attempts
        OUTPUT "Enter Username: "
        INPUT username
        IF is_user_pending(username, "users.txt") THEN
            OUTPUT "Your account is still pending approval. Please wait for confirmation."
            CALL CustomerSignIn()
        ENDIF
        OUTPUT "Enter Password: "
        INPUT password
        IF verify_customer_credentials(username, password, "customers.txt") THEN
            OUTPUT "Login successful!"
            CALL log_action(username, "Customer Sign In")
            CALL customer_interface(username)
        ELSE
            INCREMENT attempts BY 1
            OUTPUT "Incorrect username or password. You have " + (max_attempts - attempts) + " attempt(s) left."
        ENDIF
    ENDWHILE
    OUTPUT "Too many failed attempts. Please reset your password."
    CALL reset_customer_password(username, "customers.txt") AS valid, message
    OUTPUT message
ENDFUNCTION
```

Check Is User Pending Approval Function

```
FUNCTION is_user_pending(username, filename)
    OPEN filename AS filehandler IN READ MODE
    IF filehandler FOUND THEN
        FOR each line IN filehandler
            SPLIT line BY comma INTO pending_username
            IF pending_username == username THEN
                RETURN True
            ENDIF
        ENDFOR
    ELSE
        OUTPUT " File not found"
        RETURN False
    ENDIF
    RETURN False
ENDFUNCTION
```

Verify Customer Credentials Function

```
FUNCTION verify_customer_credentials(username, password, filename)
    OPEN filename AS filehandler IN READ MODE
    IF filehandler FOUND THEN
        FOR each line IN filehandler
            SPLIT line BY comma INTO stored_username AND stored_password
            IF username == stored_username AND password == stored_password THEN
                RETURN True
            ENDIF
        ENDFOR
    ELSE
        OUTPUT " File not found"
        RETURN False
    ENDIF
    RETURN False
ENDFUNCTION
```


Customer Sign Up Function

```
FUNCTION CustomerSignUp
    WHILE True
        OUTPUT "Create username (At least 4 characters and must contain only alphabetic character)"
        INPUT username
        IF check_username(username, "customers.txt") THEN
            OUTPUT "Username already exists. Please try another one"
        ELSEIF LENGTH of username IS LESS THAN 4 THEN
            OUTPUT "Invalid username. Please try another one"
        ELSEIF username CONTAINS non-alphabetic characters THEN
            OUTPUT "Invalid username. Username must contain only alphabetic characters"
        ELSE
            BREAK
        ENDIF
    ENDWHILE

    WHILE True
        OUTPUT "Create password (At least 8 characters, including uppercase letter, number, and special character)"
        INPUT password
        CALL check_password(password) AS valid, message
        IF valid IS False THEN
            OUTPUT message
            CONTINUE
        ENDIF
        OUTPUT "Re-enter password"
        INPUT verify_pw
        IF verify_pw == password THEN
            OUTPUT "Password match"
            OUTPUT "Your registration is pending to be approved"
            SET role TO "customer"
            SET status TO "pending"
            CALL save_pendingCust(username, password, role, status, "users.txt")
            BREAK
        ELSE
            OUTPUT "Passwords do not match. Please try again"
        ENDIF
    ENDWHILE
ENDFUNCTION
```

Check Username Existence Function

```
FUNCTION check_username(username, filename)
    OPEN filename AS filehandler IN READ MODE
    IF filehandler FOUND THEN
        SET existing_username = []
        FOR each line IN filehandler
            EXTRACT the first part of the line UNTIL the first comma AS stored_username
            ADD stored_username TO existing_username
        END FOR
        IF username IS IN existing_username THEN
            RETURN True
        ENDIF
    ELSE
        OUTPUT " File not found"
        RETURN False
    ENDIF
    RETURN False
ENDFUNCTION
```

Check Validity of Password Function

```
FUNCTION check_password(password)
    IF LENGTH of password IS LESS THAN 8 THEN
        RETURN False, "Password must be at least 8 characters."
    SET has_upper TO False
    SET has_digit TO False
    SET has_special TO False
    SET special_characters TO "!@#$%^&*_-+=()./?:;\"<>[]{}|"

    FOR each character IN password
        IF character IS uppercase THEN
            SET has_upper TO True
        ENDIF
        IF character IS a digit THEN
            SET has_digit TO True
        ENDIF
        IF character IS IN special_characters THEN
            SET has_special TO True
        ENDIF
    ENDFOR
    IF has_upper IS False THEN
        RETURN False, "Password must contain at least 1 uppercase letter"
    ENDIF
    IF has_digit IS False THEN
        RETURN False, "Password must contain at least 1 number"
    ENDIF
    IF has_special IS False THEN
        RETURN False, "Password must contain at least 1 special character"
    ENDIF
    RETURN True, "Password is valid"
ENDFUNCTION
```

Save Pending Customer Details Function

```
FUNCTION save_pendingCust(username, password, role, status, filename)
    OPEN filename AS filehandler IN APPEND MODE
    IF filehandler FOUND THEN
        WRITE username, password, role, and status TO filehandler SEPARATED BY commas
    ENDIF
ENDFUNCTION
```

Reset Customer Password Function

```

FUNCTION reset_customer_password(username, filename)
    WHILE True
        OUTPUT "Password Requirements: At least 8 characters, including uppercase letter, number, and special character"
        OUTPUT "Please enter new password"
        INPUT new_password
        IF LENGTH of new_password IS LESS THAN 8 THEN
            OUTPUT "Password must be at least 8 characters"
            CONTINUE
        ENDIF
        SET has_upper TO False
        SET has_digit TO False
        SET has_special TO False
        SET special_characters TO "!@#$%^&*_-+=()./?;:\"<>[]{}"
        FOR each character IN new_password
            IF character IS uppercase THEN
                SET has_upper TO True
            ELSEIF character IS a digit THEN
                SET has_digit TO True
            ELSEIF character IS IN special_characters THEN
                SET has_special TO True
            ENDIF
        ENDFOR

        IF has_upper IS False THEN
            OUTPUT "Password must contain at least 1 uppercase letter"
            CONTINUE
        ELSEIF has_digit IS False THEN
            OUTPUT "Password must contain at least 1 number"
            CONTINUE
        ELSEIF has_special IS False THEN
            OUTPUT "Password must contain at least 1 special character"
            CONTINUE
        ENDIF

        CALL modify_customer_password(username, new_password, filename) AS success
        IF success IS True THEN
            RETURN True, "Password has been reset successfully"
        ELSE
            RETURN False, "Failed to reset password. Please try again"
        ENDIF
    ENDWHILE
ENDFUNCTION

```

Modify Customer Password Function

```

FUNCTION modify_customer_password(username, new_password, filename)
    SET modified TO False
    SET records = []
    OPEN filename AS filehandler IN READ MODE
    IF filehandler FOUND THEN
        READ all lines FROM filehandler INTO records

        OPEN filename AS filehandler IN WRITE MODE
        FOR each line IN records
            EXTRACT stored_username and stored_password FROM line USING comma as the separator
            IF stored_username == username THEN
                WRITE username and new_password TO filehandler SEPARATED BY a comma
                SET modified TO True
            ELSE
                WRITE the original line TO filehandler
            ENDIF
        ENDFOR
    ELSE
        OUTPUT "No record found"
    ENDIF

    RETURN modified
ENDFUNCTION

```

Customer Interface Function

```

FUNCTION customer_interface(username)
    WHILE True
        OUTPUT "1. Purchase Order"
        OUTPUT "2. Service/Repair Order"
        OUTPUT "3. Modify Purchase/Service/Repair Order"
        OUTPUT "4. Make payment"
        OUTPUT "5. Inquiry of Order Status"
        OUTPUT "6. Cancel Orders"
        OUTPUT "7. Reports"
        OUTPUT "8. Give feedback"
        OUTPUT "9. Log Out"

        OUTPUT "Please select an option to continue with"
        INPUT option
        IF option IS an integer THEN
            IF option == 1 THEN
                CALL place_purchase_order(username) AS order, order_id
            ELSEIF option == 2 THEN
                CALL create_service_order(username)
            ELSEIF option == 3 THEN
                CALL modify_order(username)
            ELSEIF option == 4 THEN
                CALL make_payment(username)
            ELSEIF option == 5 THEN
                CALL inquiry_of_order_status(username)
            ELSEIF option == 6 THEN
                CALL cancel_order(username)
            ELSEIF option == 7 THEN
                CALL customer_reports(username)
            ELSEIF option == 8 THEN
                CALL collect_feedback(username)
            ELSEIF option == 9 THEN
                OUTPUT "Logging out."
                CALL log_action(username, "Customer Log Out")
                CALL homepage()
            ELSE
                OUTPUT "Invalid option. Please try again."
            ENDIF
        ELSE
            OUTPUT "Invalid input. Please try again."
        ENDIF
    ENDWHILE
ENDFUNCTION

```

Generate Order ID Function

```

FUNCTION generate_order_id
    RETURN "OR" + current date and time in format YearMonthDayHourMinuteSecond
ENDFUNCTION

```

Get Item Name Function

```
FUNCTION get_item(item)
    IF item == 1 THEN
        RETURN "Desktop"
    ELSEIF item == 2 THEN
        RETURN "Laptop"
    ELSEIF item == 3 THEN
        RETURN "Mouse"
    ELSEIF item == 4 THEN
        RETURN "Keyboard"
    ELSEIF item == 5 THEN
        RETURN "Monitor"
    ELSEIF item == 6 THEN
        RETURN "Charger"
    ELSE
        RETURN None
    END IF
END FUNCTION
```

Get Item Price Function

```
FUNCTION get_item_price(item_name)
    IF item_name == "Desktop" THEN
        RETURN 2500
    ELSEIF item_name == "Laptop" THEN
        RETURN 4000
    ELSEIF item_name == "Mouse" THEN
        RETURN 80
    ELSEIF item_name == "Keyboard" THEN
        RETURN 150
    ELSEIF item_name == "Monitor" THEN
        RETURN 500
    ELSEIF item_name == "Charger" THEN
        RETURN 100
    ELSE
        RETURN 0
    ENDIF
ENDFUNCTION
```

Save Purchase Order Function

```
FUNCTION save_purchase_order(order)
  OPEN "orders.txt" AS filehandler IN APPEND MODE
  IF filehandler FOUND THEN
    SET order_details TO EMPTY STRING
    FOR each item IN order['items']
      CONCATENATE item['name'], item['quantity'], item['price'] AND ADD TO order_details
      CONCATENATE order['order_id'], order['username'], order['status'], order['total_price'], order_details
      WRITE THE CONCATENATED ORDER DETAILS TO filehandler
    ENDFOR
    CLOSE filehandler
  ELSE
    OUTPUT " File not found"
  ENDIF
ENDFUNCTION
```

Place Purchase Order Function

```
FUNCTION place_purchase_order(username)
  order_id = CALL generate_order_id
  SET order_items = []
  SET total_price = 0
  stock = CALL get_stock
  FOR each item IN stock
    IF item is not for staff only THEN
      OUTPUT item index, item name, AND item price
    ENDIF
  ENDFOR

  WHILE True
    OUTPUT "Please enter the number of item you want to purchase (Enter '0' to finish)"
    INPUT item
    IF item == 0 THEN
      BREAK WHILE LOOP
    ENDIF
    item_name = CALL get_item(item)
    IF item_name is None THEN
      OUTPUT "Invalid choice, please try again"
      CONTINUE WHILE LOOP
    ENDIF
    OUTPUT "Please enter the quantity: "
    INPUT quantity
    IF quantity <= 0 THEN
      OUTPUT "Quantity must be greater than zero."
      CONTINUE WHILE LOOP
    ENDIF
    item_stock = stock[item_name]
    IF item_stock['quantity'] >= quantity THEN
      ADD item_name, quantity, AND item_stock ['price'] TO order_items
      total_price = item_stock ['price'] * quantity
      DECREASE item_stock['quantity'] by quantity
    ELSE
      OUTPUT "Insufficient stock."
    ENDIF
  ENDWHILE

  IF order_items is not empty THEN
    SET status = "Unpaid"
    SET order TO CREATE an order dictionary WITH order_id, username, order_items, total_price, AND status
    CALL save_purchase_order(order)
    OUTPUT "Order placed successfully."
    CALL log_action(username, "Place Purchase Order")
    CALL customer_interface(username)
  ELSE
    OUTPUT "No items added to the order."
  ENDIF
  RETURN order_id, order_items, total_price
END FUNCTION
```

Generate Service Order ID Function

```
FUNCTION generate_service_id
    RETURN "SR" + current date and time in format YearMonthDayHourMinuteSecond
ENDFUNCTION
```

```
FUNCTION get_service(service_option)
    IF service_option == 1 THEN
        RETURN "Device Repair"
    ELSEIF service_option == 2 THEN
        RETURN "Software Installation"
    ELSEIF service_option == 3 THEN
        RETURN "Hardware Upgrade"
    ELSEIF service_option == 4 THEN
        RETURN "Computer Repair"
    ELSE
        RETURN None
    ENDIF
ENDFUNCTION
```

```
FUNCTION get_service_price(service_name)
    IF item_name == "Device Repair" THEN
        RETURN 100
    ELSEIF item_name == "Software Installation" THEN
        RETURN 200
    ELSEIF item_name == "Hardware Upgrade" THEN
        RETURN 250
    ELSEIF item_name == "Computer Repair" THEN
        RETURN 250
    ELSE
        RETURN 0
    ENDIF
ENDFUNCTION
```

```
FUNCTION save_service_order(order)
    OPEN "service_orders.txt" AS filehandler IN APPEND MODE
    IF filehandler FOUND THEN
        SET services_str = EMPTY STRING
        FOR each service IN order['services']
            CONCATENATE service['name'], service['price'] AND ADD TO services_str
        ENDFOR
        CONCATENATE order['service_id'], order['username'], order['status'], order['total_price'], services_str
        WRITE THE CONCATENATED ORDER DETAILS TO filehandler
        CLOSE filehandler
    ELSE
        OUTPUT "File not found"
    ENDIF
ENDFUNCTION
```

```

FUNCTION create_service_order(username)
    service_id = CALL generate_service_id
    SET service_items = []
    SET total_price = 0
    services = CALL load_data(SERVICE_ORDERS_FILE)
    FOR each service AND details IN services
        DISPLAY service INDEX, service NAME, AND details['tentative_time']
        DISPLAY "Price: RM" AND details['service_price']
    ENDFOR

    WHILE True
        OUTPUT "Please select the service you want to request (Enter '0' to finish)"
        INPUT service_option
        IF service_option == 0 THEN
            BREAK WHILE LOOP
        ENDIF
        service_name = CALL get_service(service_option)
        IF service_name IS None THEN
            DISPLAY "Invalid choice, please try again"
            CONTINUE WHILE LOOP
        ENDIF
        IF ANY SERVICE IN service_items HAS service['name'] == service_name THEN
            OUTPUT "This service has been added to your order."
            CONTINUE WHILE LOOP
        ENDIF
        service_details = services.get(service_name)
        IF service_details EXISTS THEN
            ADD service_name, service_details['service_price'], service_details['tentative_time'] TO service_items
            total_price = service_details['service_price']
            DISPLAY "Added service to your order."
        ENDIF
    ENDWHILE
    IF service_items IS NOT EMPTY THEN
        status = "Unpaid"
        order = CREATE AN ORDER DICTIONARY WITH service_id, username, service_items, total_price, AND status
        CALL save_service_order(order)
        DISPLAY "Service Order placed successfully."
        CALL log_action(username, "Place Service/Repair Order")
        CALL customer_interface(username)
    ELSE
        DISPLAY "No service added to the order."
    ENDIF
    RETURN service_id, service_items, total_price
END FUNCTION

```



```

FUNCTION load_orders_details(filename)
    OPEN filename AS filehandler IN READ MODE
    IF filehandler FOUND THEN
        SET data TO EMPTY DICTIONARY
        WHILE NOT END OF FILE
            READ LINE FROM filehandler
            SET order_id TO PART BEFORE FIRST SEMICOLON
            REMOVE PART BEFORE FIRST SEMICOLON AND SEMICOLON FROM LINE
            SET username TO PART BEFORE FIRST SEMICOLON
            REMOVE PART BEFORE FIRST SEMICOLON AND SEMICOLON FROM LINE
            SET status TO PART BEFORE FIRST SEMICOLON
            REMOVE PART BEFORE FIRST SEMICOLON AND SEMICOLON FROM LINE
            SET total_price TO PART BEFORE FIRST SEMICOLON
            REMOVE PART BEFORE FIRST SEMICOLON AND SEMICOLON FROM LINE
            SET items_str TO LINE

            SET items TO EMPTY LIST
            WHILE items_str IS NOT EMPTY
                SET item_part TO PART BEFORE FIRST SEMICOLON
                REMOVE PART BEFORE FIRST SEMICOLON AND SEMICOLON FROM items_str
                SET item_details TO PART SPLIT BY COMMA
                SET item_name TO FIRST PART
                SET item_price TO SECOND PART
                SET item_quantity TO 1
                IF item_details HAS MORE THAN 2 PARTS THEN
                    SET item_quantity TO THIRD PART
                ENDIF
                ADD item_name, item_quantity, AND item_price TO items
            ENDWHILE

            SET data[order_id] TO DICTIONARY WITH username, status, total_price, AND items
        ENDWHILE
        CLOSE filehandler
        RETURN data
    ELSE
        OUTPUT "File not found."
        RETURN EMPTY DICTIONARY
    ENDIF
ENDFUNCTION

FUNCTION display_order_details(username)
    SET purchase_orders TO CALL load_order_details('orders.txt')
    SET service_orders TO CALL load_order_details('service_orders.txt')
    FOR each order_id AND order IN purchase_orders
        IF order['username'] == username THEN
            OUTPUT "Order ID: " + order_id + ", Status: " + order['status'] + ", Total Price: RM" + order['total_price'] + ", Items: " + order['items']
        ENDIF
    END FOR

    FOR each order_id AND order IN service_orders
        IF order['username'] EQUALS username THEN
            OUTPUT "Order ID: " + order_id + ", Status: " + order['status'] + ", Total Price: RM" + order['total_price'] + ", Services: " + order['items']
        ENDIF
    END FOR
END FUNCTION

```

```

FUNCTION modify_order(username)
    SET user_purchase_orders, user_service_orders TO CALL display_order_details(username)
    IF user_purchase_orders IS EMPTY AND user_service_orders IS EMPTY THEN
        OUTPUT "You have no orders to modify."
    ENDIF

    WHILE True
        OUTPUT "1. Modify Purchase Order"
        OUTPUT "2. Modify Service/Repair Order"
        OUTPUT "3. Exit"
        INPUT choice
        IF choice == 1 THEN
            SET order_file TO 'orders.txt'
            SET orders TO user_purchase_orders
            SET modify_type TO "item"
        ELSEIF choice == 2 THEN
            SET order_file TO 'service_orders.txt'
            SET orders TO user_service_orders
            SET modify_type TO "service"
        ELSEIF choice == 3 THEN
            OUTPUT "Exiting modification menu."
            CALL customer_interface(username)
        ELSE
            OUTPUT "Invalid choice. Please try again."
            CONTINUE WHILE LOOP
        ENDIF

        IF orders IS empty THEN
            OUTPUT "You have no orders of this type."
            CONTINUE WHILE LOOP
        ENDIF

        OUTPUT "Order Summary:"
        FOR each order_id AND order IN orders
            OUTPUT "Order ID: " + order_id + ", Status: " + order['status'] + ", Total Price: RM" + order['total_price']
            OUTPUT "Items: " + CONCATENATE ITEMS
        ENDFOR

        OUTPUT "Enter the Order ID you want to modify: "
        INPUT order_id
        IF order_id NOT IN orders THEN
            OUTPUT "Invalid Order ID."
            CONTINUE WHILE LOOP
        ENDIF

        IF orders[order_id]['status'] == 'paid' THEN
            OUTPUT "Order cannot be modified as it has been paid."
            CONTINUE WHILE LOOP
        ENDIF

        WHILE True
            OUTPUT "1. Add " + modify_type
            OUTPUT "2. Remove " + modify_type
            OUTPUT "3. Finish modification"
            INPUT option
            IF option == 1 THEN
                IF modify_type == "item" THEN
                    SET stock TO CALL get_stock()
                    OUTPUT "Available Items:"
                    FOR each item AND details IN stock
                        IF details['for_staff_only'] IS False THEN
                            OUTPUT item_name AND details['price']
                        ENDIF
                    ENDFOR
                ENDIF
                OUTPUT "Please enter the number of item you want to add (Enter '0' to finish): "
                INPUT item_choice
                IF item_choice EQUALS 0 THEN
                    BREAK WHILE LOOP
                ENDIF
                SET item_name TO CALL get_item(item_choice)
                IF item_name IS None THEN
                    OUTPUT "Invalid choice, please try again"
                    CONTINUE WHILE LOOP
                ENDIF
                OUTPUT "Enter quantity"
                INPUT quantity
                IF quantity <= 0 THEN
                    OUTPUT "Quantity must be greater than zero."
                    CONTINUE WHILE LOOP
                ENDIF
                SET item_stock TO stock[item_name]
                IF item_stock['quantity'] >= quantity THEN
                    ADD item_name, quantity, AND item_stock['price'] TO orders[order_id]['items']
                    INCREASE orders[order_id]['total_price'] BY item_stock['price'] * quantity
                    DECREASE item_stock['quantity'] BY quantity
                    CALL save_orders(order_file, orders)
                ENDIF
            ENDIF
        ENDIF
    ENDIF

```

```

        OUTPUT "Order has been modified successfully."
        CALL log_action(username, "Modify Purchase Order - Add Item")
    ELSE
        OUTPUT "Insufficient stock."
    ENDIF
ENDWHILE
ELSEIF modify_type == "service" THEN
    SET services TO CALL load_data(SERVICE_ORDERS_FILE)
    OUTPUT "Available Service/Repair Option:"
    FOR each service AND details IN services
        OUTPUT service_name AND details['service_price']
    ENDFOR
    WHILE True
        INPUT service_option
        IF service_option == 0 THEN
            BREAK WHILE LOOP
        ENDIF
        SET service_name TO CALL get_service(service_option)
        IF service_name IS None THEN
            OUTPUT "Invalid choice, please try again"
            CONTINUE WHILE LOOP
        ENDIF
        IF ANY service IN orders[order_id]['items'] HAS 'name' EQUAL TO service_name THEN
            OUTPUT "This service has been added to your order."
            CONTINUE WHILE LOOP
        ENDIF
        SET service_details TO services[service_name]
        IF service_details EXISTS THEN
            ADD service_name, service_details['service_price'], AND service_details['tentative_time'] TO orders[order_id]['items']
            INCREASE orders[order_id]['total_price'] BY service_details['service_price']
            OUTPUT "Added service to your order."
            CALL save_orders(order_file, orders)
            OUTPUT "Order has been modified successfully."
            CALL log_action(username, "Modify Service/Repair Order - Add Service")
        ENDIF
    ENDWHILE
ENDIF
ELSEIF option == 2 THEN
    IF modify_type == "item" THEN
        OUTPUT "Enter item name to remove: "
        INPUT item_name
        SET item_to_remove TO None
        FOR each item IN orders[order_id]['items']
            IF item['name'] == item_name THEN
                SET item_to_remove TO item
                BREAK
            ENDIF
        ENDFOR
        IF item_to_remove EXISTS THEN
            REMOVE item_to_remove FROM orders[order_id]['items']
            DECREASE orders[order_id]['total_price'] BY item_to_remove['price']
            CALL save_orders(order_file, orders)
            OUTPUT "Item removed"
            CALL log_action(username, "Modify Purchase Order - Remove Item")
        ELSE
            OUTPUT "Item not found in the order."
        ENDIF
    ELSEIF modify_type == "service" THEN
        OUTPUT "Enter the service name to remove: "
        INPUT service_name
        SET service_to_remove TO None
        FOR each service IN orders[order_id]['items']
            IF service['name'] == service_name THEN
                SET service_to_remove TO service
                BREAK
            ENDIF
        ENDFOR
        IF service_to_remove EXISTS THEN
            REMOVE service_to_remove FROM orders[order_id]['items']
            DECREASE orders[order_id]['total_price'] BY service_to_remove['price']
            CALL save_orders(order_file, orders)
            OUTPUT "Service removed."
            CALL log_action(username, "Modify Service/Repair Order - Remove Service")
        ELSE
            OUTPUT "Service not found in the order."
        ENDIF
    ENDIF
ELSEIF option == 3 THEN
    CALL customer_interface(username)
ELSE
    OUTPUT "Invalid choice. Please try again."
ENDIF
ENDWHILE
ENDFUNCTION

```

```

FUNCTION save_orders(filename, order)
    OPEN filename AS filehandler IN WRITE MODE
    IF filehandler FOUND THEN
        FOR each order_id AND order IN orders
            IF filename IS 'orders.txt' THEN
                SET items_str TO EMPTY STRING
                FOR each item IN order['items']
                    CONCATENATE item['name'], ",", item['quantity'], ",", item['price'] AND ADD TO items_str
                    CONCATENATE ";" AND ADD TO items_str
                ENDFOR
                REMOVE LAST CHARACTER ";" FROM items_str
            ELSEIF filename IS 'service_orders.txt' THEN
                SET items_str TO EMPTY STRING
                FOR each service IN order['items']
                    CONCATENATE service['name'], ",", service['price'] AND ADD TO items_str
                    CONCATENATE ";" AND ADD TO items_str
                ENDFOR
                REMOVE LAST CHARACTER ";" FROM items_str
            ELSE
                OUTPUT "Unsupported file type."
                CLOSE filehandler
                RETURN
            ENDIF
            CONCATENATE order_id, ";", order['username'], ";", order['status'], ";", order['total_price'], ";", items_str AND STORE IN line
            WRITE line TO filehandler
        ENDFOR
        CLOSE filehandler
    ELSE
        OUTPUT "File not found."
    ENDIF
ENDFUNCTION

FUNCTION make_payment(username)
    CALL check_order_details(username) AS user_purchase_orders AND user_service_orders

    SET orders_to_pay = {}

    FOR each order_id AND order IN user_purchase_orders
        IF order['status'] IS NOT 'paid' THEN
            ADD order_id AND order TO orders_to_pay
        ENDIF
    ENDFOR

    FOR each order_id AND order IN user_service_orders
        IF order['status'] IS NOT 'paid' THEN
            ADD order_id AND order TO orders_to_pay
        ENDIF
    ENDFOR

    IF orders_to_pay IS EMPTY THEN
        OUTPUT "You have no unpaid orders to pay for."
    ENDIF

    OUTPUT "Your unpaid orders:"
    FOR each order_id AND order IN orders_to_pay
        OUTPUT "Order ID: " + order_id + ", Total Price: RM" + order['total_price']
    ENDFOR

    OUTPUT "Enter the Order ID you want to pay for:"
    INPUT order_id

    IF order_id IS IN orders_to_pay THEN
        SET orders_to_pay[order_id]['status'] TO 'paid'
        CALL save_orders('orders.txt', user_purchase_orders)
        CALL save_orders('service_orders.txt', user_service_orders)
        OUTPUT "Order has been paid successfully."
        CALL generate_invoice(username)
        CALL log_action(username, "Make Payment")
    ELSE
        OUTPUT "Invalid Order ID."
    ENDIF
ENDFUNCTION

```

```

FUNCTION inquiry_of_order_status(username)
    CALL log_action(username, "Inquiry of Order Status")
    CALL check_order_details(username) AS user_purchase_orders AND user_service_orders

    IF user_purchase_orders IS EMPTY AND user_service_orders IS EMPTY THEN
        OUTPUT "Order Status Inquiry:"
        OUTPUT "You have no orders to inquire about."
    ENDIF

    IF user_purchase_orders IS NOT EMPTY THEN
        OUTPUT "Purchase Orders:"
        FOR each order_id AND order IN user_purchase_orders
            OUTPUT "Order ID: " + order_id
            OUTPUT "Status: " + order['status']
            OUTPUT "Total Price: RM" + order['total_price']
            OUTPUT "Items:"
            FOR each item IN order['items']
                OUTPUT " - " + item['name'] + " (RM" + item['price'] + ")"
            ENDFOR
        ENDFOR
    ELSE
        OUTPUT "No purchase orders found."
    ENDIF

    IF user_service_orders IS NOT EMPTY THEN
        OUTPUT "Service/Repair Orders:"
        FOR each order_id AND order IN user_service_orders
            OUTPUT "Order ID: " + order_id
            OUTPUT "Status: " + order['status']
            OUTPUT "Total Price: RM" + order['total_price']
            OUTPUT "Services:"
            FOR each item IN order['items']
                OUTPUT " - " + item['name'] + " (RM" + item['price'] + ")"
            ENDFOR
        ENDFOR
    ELSE
        OUTPUT "No service/repair orders found."
    ENDIF
ENDFUNCTION

```

```

FUNCTION cancel_order(username)
    CALL check_order_details(username) AS user_purchase_orders AND user_service_orders
    SET orders_to_cancel = {}

    FOR each order_id AND order IN user_purchase_orders
        IF order['status'] IS NOT 'paid' THEN
            ADD order_id AND order TO orders_to_cancel
        ENDIF
    ENDFOR

    FOR each order_id AND order IN user_service_orders
        IF order['status'] IS NOT 'paid' THEN
            ADD order_id AND order TO orders_to_cancel
        ENDIF
    ENDFOR

    IF orders_to_cancel IS EMPTY THEN
        OUTPUT "You have no unpaid orders to cancel."
        RETURN
    ENDIF

    OUTPUT "Your unpaid orders:"
    FOR each order_id AND order IN orders_to_cancel
        OUTPUT "Order ID: " + order_id + ", Total Price: RM" + order['total_price']
    ENDFOR

    OUTPUT "Enter the Order ID you want to cancel: "
    INPUT order_id
    IF order_id EXISTS IN orders_to_cancel THEN
        IF order_id EXISTS IN user_purchase_orders THEN
            REMOVE order_id FROM user_purchase_orders
            CALL save_orders('orders.txt', user_purchase_orders)
        ELSE IF order_id EXISTS IN user_service_orders THEN
            REMOVE order_id FROM user_service_orders
            CALL save_orders('service_orders.txt', user_service_orders)
        ENDIF
        CALL log_action(username, "Cancel Order")
        OUTPUT "Order has been canceled successfully."
    ELSE
        OUTPUT "Invalid Order ID."
    ENDIF
ENDFUNCTION

FUNCTION customer_reports(username)
    CALL log_action(username, "Generate Customer Report")
    CALL load_data(REPORTS_FILE) AS reports
    SET user_reports AS reports.GET(username, EMPTY_LIST)

    IF user_reports IS EMPTY THEN
        OUTPUT "No reports found for user: " + username
    ELSE
        OUTPUT "Reports:"
        OUTPUT "    Date    | Time |    Actions"
        FOR EACH action IN user_reports
            OUTPUT " - " + action['time'] + ": " + action['action']
        ENDFOR
    ENDIF
ENDFUNCTION

```

```

FUNCTION collect_feedback(username)
    OUTPUT "Please provide your feedback."
    SET rating TO None
    While rating IS None
        OUTPUT "Rate our service (1-5): "
        INPUT rating
        IF rating IS INTEGER THEN
            IF rating < 1 OR rating > 5 THEN
                OUTPUT "Please enter a number between 1 and 5."
                SET rating TO None
            ENDIF
        ELSE
            OUTPUT "Invalid input. Please try again."
            SET rating TO None
        ENDIF
    ENDWHILE

    OUTPUT "Please leave your comments: "
    INPUT comments

    SET feedback TO DICTIONARY WITH
        'username' AS username,
        'rating' AS rating
        'comments' AS comments,
        'date' AS CURRENT DATE AND TIME

    OPEN 'feedback.json' AS filehandler IN READ MODE
    IF file FOUND THEN
        SET feedback_data TO CONTENTS OF 'feedback.json'
        APPEND feedback TO feedback_data
        SAVE feedback_data TO 'feedback.json'
    ELSE
        OUTPUT "File not found"
    ENDIF

    OUTPUT "Thank you for your feedback!"
    CALL log_action(username, "Give Feedback")
ENDFUNCTION

```


Superuser Create Data Function

```
FUNCTION CreateSuperUser_data
    DEFINE a list named SuperUser_data with single string 'superuser,superpass'
    OPEN file 'SuperUser.txt' in write mode
        FOR each line in SuperUser_data
            WRITE the item to the file followed by a newline
        CLOSE file
    PRINT "Super User data file created with initial data."

#read_file
FUNCTION read_file
TRY
    OPEN file 'SuperUser.txt' in read mode
    READ content of file
    PRINT "File content:"
    PRINT content
    CLOSE the file
EXCEPT FileNotFoundError
    PRINT "File not found."
```

Superuser Login Function

```
FUNCTION SuperUser_login
    SET max_attempts TO 3
    SET attempts TO 0

    WHILE attempts < max_attempts
        PROMPT user for Super User ID
        READ superuser_id
        PROMPT user for password
        READ password

        SET valid_login TO False

        TRY
            OPEN file 'SuperUser.txt' in read mode
                FOR each line in file
                    SPLIT line into stored_superuser_id AND stored_password
                    IF stored_superuser_id matches superuser_id AND stored_password matches password
                        SET valid_login TO True
                        BREAK loop
        EXCEPT FileNotFoundError
            PRINT "User data file not found."
            RETURN

        IF valid_login
            PRINT "Login successful."
            CALL SuperUser_menu with superuser_id
            RETURN
        ELSE
            INCREMENT attempts
            PRINT number of attempts left

    PRINT "Too many failed attempts. Access denied."
    CALL reset_superuser_password with superuser_id
```

Superuser Reset Password Function

```
FUNCTION reset_superuser_password(superuser_id)

    TRY
        OPEN file 'SuperUser.txt' in read mode
        READ all lines into superusers
    EXCEPT FileNotFoundError
        PRINT "User data file not found."
        RETURN

    FOR each line in superusers with index i
        SPLIT line into stored_superuser_id
        IF stored_superuser_id matches superuser_id
            WHILE True
                PROMPT user for new password
                READ new_password

                CHECK if new_password meets length and character requierments
                IF new_password is valid
                    UPDATE superusers[i] with new_password
                    OPEN file 'SuperUser.txt' in write mode
                    WRITE updated superusers
                    PRINT "Password reset successfully."
                    RETURN
                ELSE
                    PRINT validation error messages
            PRINT "SuperUser ID not found."
```

Superuser Verify New Users Function

```
FUNCTION Verify_New_Users
    TRY
        OPEN 'users.txt' in read mode
        READ all lines into users
    EXCEPT FileNotFoundError
        RETURN

    SET pending_users TO users where status is "pending"

    IF pending_users is empty
        PRINT "No pending users to verify."
        RETURN

    FOR each user in pending_users with index i
        PRINT user details

    LOG action with superuser_id and "Verify New Users"

    WHILE True
        PROMPT user for decision (Y/N)
        IF decision is valid
            BREAK loop
        PRINT invalid choice message

    IF decision is 'Y'
        CALL Add_User
    ELSE
        PRINT "No users were processed."
```

Superuser Add Users Function

```
FUNCTION Add_User
    LOG action with superuser_id and "Add User"
    TRY
        OPEN 'users.txt' in read mode
        READ all lines into users
    EXCEPT FileNotFoundError
        RETURN

    SET pending_users TO users where status is "pending"

    IF pending_users is empty
        PRINT "No pending users to add."
        RETURN

    FOR each user in pending_users with index i
        PRINT user details

        WHILE True
            PROMPT user for decision (A/R)
            IF decision is valid
                BREAK loop
            PRINT invalid choice message

        IF decision is 'A'
            WHILE True
                PROMPT user for role (S/A)
                IF role is valid
                    BREAK loop
                PRINT invalid role message

            UPDATE user status to "approved" and save user
            CALL save_user_to_file
            PRINT success message
        ELSE
            UPDATE user status to "rejected"
            PRINT rejection message

    TRY
        OPEN 'users.txt' in write mode
        WRITE updated users
    EXCEPT Exception as e
        PRINT error message
```

Superuser Save User to File Function

```
FUNCTION save_user_to_file(username, password, ic, role)
    IF role is 'staff'
        SET file_path TO 'staff.txt'
    ELSE IF role is 'admin'
        SET file_path TO 'admin.txt'
    ELSE IF role is 'customer'
        SET file_path TO 'customers.txt'
    ELSE
        RETURN

    TRY
        OPEN file_path in append mode
        WRITE user details
    EXCEPT Exception as e
        PRINT error message
```

Superuser Modify User Function

```
FUNCTION Modify_User_Details
    LOG action with superuser_id and "Modify User Details"
    TRY
        OPEN 'users.txt' in read mode
        READ all lines into users
    EXCEPT FileNotFoundError
        RETURN

    SET user_dict TO dictionary of username to user details

    IF user_dict is empty
        PRINT "No valid users found."
        RETURN

    FOR each user in user_dict
        PRINT user details

    PROMPT user for selection
    IF selection is valid
        UPDATE user details
        WRITE updated user details to 'users.txt'
        PRINT success message
    ELSE
        PRINT invalid selection message
```

Superuser Disable User Access Function

```
#Disable_User_access
FUNCTION Disable_User_Access
    SET file_map TO dictionary of role choices to file paths

    PRINT role choices

    WHILE True
        PROMPT user for role choice
        IF choice is valid
            BREAK loop
        ELSE
            PRINT invalid choice message

    SET file_path TO selected file path
    TRY
        OPEN file_path in read mode
        READ all lines into users
    EXCEPT FileNotFoundError
        RETURN

    IF users is empty
        PRINT "No users found."
    ELSE
        RETURN

    PRINT user details

    WHILE True
        PROMPT user for user choice
        IF choice is valid
            BREAK loop
        ELSE
            PRINT invalid choice message

    UPDATE selected user status to "disabled"
    TRY
        OPEN file_path in write mode
        WRITE updated users
        PRINT success message
        LOG action with superuser_id and role
        PRINT error message
```

Superuser Inquiry System Usage Function

```
FUNCTION Inquiry_System_Usage(superuser_id)
    LOAD data from 'reports.txt'

    IF reports is empty
        PRINT "No reports found."
        RETURN

    FOR each username in reports
        PRINT username and their reports

    LOG action with superuser_id and "Inquired system usage report"
```

Superuser Check Customer Order Function

```
#Check_Customer_Order_Status
FUNCTION Check_Customer_Order_Status(superuser_id)
    LOG action with superuser_id and "Check customer order status"
    LOAD orders from 'orders.txt'

    IF orders is empty
        PRINT "No orders found."
        RETURN

    PROMPT user for order ID

    IF order ID is in orders
        PRINT order details
        LOG action with username and order ID
    ELSE
        PRINT "Order not found."
```

Superuser View Report Function

```
FUNCTION Superuser_View_Reports(superuser_id)
    LOG action with superuser_id and "View Reports"
    LOAD data from 'reports.txt'

    IF reports is empty
        PRINT "No reports found."
        RETURN

    FOR each superuser_id in reports
        PRINT superuser_id and their reports

    PROMPT user to return to main menu
```

Superuser View Feedback Function

```
FUNCTION view_all_feedback(superuser_id)
  LOG action with superuser_id and "View Feedback"
  LOAD data from 'feedback.json'

  FOR each feedback in feedback_data
    IF feedback is valid
      PRINT feedback details
    ELSE
      PRINT unexpected data format message
```

Superuser Menu Function

```
FUNCTION SuperUser_menu(superuser_id)
  DISPLAY menu options for the Super User
  PROMPT user for menu choice
  READ choice

  IF choice is valid
    PERFORM action based on choice
  ELSE
    PRINT invalid choice message
```


Admin Interface Function

```
FUNCTION admin_interface
    WHILE True
        DISPLAY "Admin Interface Menu"
        DISPLAY "1. Login"
        DISPLAY "2. Signup"
        DISPLAY "3. Exit"

        INPUT choice

        IF choice IS '1' THEN
            CALL Admin_Login

        ELSE IF choice IS '2' THEN
            CALL Admin_Signup

        ELSE IF choice IS '3' THEN
            DISPLAY "Exiting..."
            CALL homepage
            BREAK

        ELSE
            DISPLAY "Invalid option. Please choose again."
        END IF
    END WHILE
END FUNCTION
```

Admin Login Function

```
FUNCTION Admin_Login
    SET max_attempts TO 3
    SET attempts TO 0

    DISPLAY "Admin Login Page"

    WHILE attempts < max_attempts
        INPUT username
        INPUT password

        DISPLAY "Attempting login with username: username"

        IF verify_Admincredentials(username, password, active, "admin.txt") IS True THEN
            DISPLAY "Login successful! Welcome, username"
            CALL admin_menu(username)
            RETURN
        ELSE
            INCREMENT attempts
            DISPLAY "Wrong username or password. attempts_left attempts left."
        END IF
    END WHILE

    DISPLAY "Too many failed attempts. Please reset your password."
    CALL reset_admin_password(username, "admin.txt")

    CALL main_menu
END FUNCTION
```

Admin Signup Function

```
FUNCTION Admin_Signup
    SET role TO "admin"
    SET status TO "pending"

    DISPLAY "Admin Registration Page"

    WHILE True
        INPUT username

        IF NOT check_username(username, "admin.txt") THEN
            BREAK
        ELSE
            DISPLAY "Username already exists. Please try another one."
        END IF

        WHILE True
            INPUT password

            CALL check_password(password) RETURNS valid, message

            IF valid THEN
                INPUT verify_pw

                IF verify_pw EQUALS password THEN
                    DISPLAY "-Password match-"
                    BREAK
                ELSE
                    DISPLAY "Passwords do not match. Please try again."
                END IF
            ELSE
                DISPLAY message
            END IF
        END WHILE

        CALL save_pendingAdmin(username, password, role, status, "users.txt")
        DISPLAY "Your registration is pending to be approved."
        DISPLAY "You will be returned to Homepage."

        CALL admin_interface
    END WHILE
END FUNCTION
```

Admin Save Pending Function

```
FUNCTION save_pendingAdmin(username, password, role, status, filename)
    TRY
        OPEN 'users.txt' IN APPEND MODE
        WRITE username, password, role, status TO file
        CLOSE file
    CATCH FileNotFoundError
        RETURN False
    END TRY
END FUNCTION
```

Admin Save Pending Function

```
FUNCTION reset_admin_password(username, filename)
    DISPLAY "Reset Password Page"

    SET attempt_count TO 0
    SET max_attempts TO 5

    WHILE attempt_count < max_attempts
        INPUT new_password

        IF password DOES NOT MEET CRITERIA THEN
            DISPLAY CRITERIA_MESSAGE
            INCREMENT attempt_count
            CONTINUE

        INPUT confirm_password

        IF new_password NOT EQUAL TO confirm_password THEN
            DISPLAY "Passwords do not match. Please try again."
            INCREMENT attempt_count
            CONTINUE

        CALL modify_admin_password(username, new_password, filename) RETURNS valid, message

        IF valid THEN
            RETURN True, "Password has been reset successfully."
        ELSE
            RETURN False, "Failed to reset password. Please try again."

    RETURN False, "Too many failed attempts. Please contact the system administrator."
END FUNCTION
```

Admin Verify Credentials Function

```
FUNCTION verifys_Admincredentials(username, password, active, filename)
    DISPLAY "Verifying Admin Credentials..."

    TRY
        OPEN filename IN READ MODE

        FOR EACH line IN file
            SPLIT line INTO stored_username, stored_password, stored_active

            IF username EQUALS stored_username AND password EQUALS stored_password AND active EQUALS stored_active THEN
                DISPLAY "Credentials verified!"
                RETURN True
            END FOR
        CATCH FileNotFoundError
            RETURN False
    END TRY
END FUNCTION
```

Admin Verify New Users Function

```
FUNCTION
Verify_New_Users
    SET username TO "default_user"
    CALL log_action(username, "Verify New Users")

    TRY
        OPEN 'users.txt' IN READ MODE
        SET users TO READ ALL LINES FROM file
        CLOSE file
    CATCH FileNotFoundError
        DISPLAY "No users to verify."
        RETURN

    SET pending_users TO FILTER users WHERE status IS "pending"

    IF pending_users IS EMPTY THEN
        DISPLAY "No pending users to verify."
        RETURN

    FOR EACH user IN pending_users
        SPLIT user INTO username, password, ic, role, status
        DISPLAY user details

    WHILE True
        INPUT decision
        IF decision IS 'Y' OR 'N' THEN
            BREAK
        ELSE
            DISPLAY "Invalid choice. Please enter 'Y' for Yes or 'N' for No."

    IF decision EQUALS 'Y' THEN
        CALL Add_User
    ELSE
        DISPLAY "No users were processed."
END FUNCTION
```

Log_Action Function

```
FUNCTION log_action(username, action)
    SET reports TO Load data from REPORTS_FILE

    SET current_time TO Get current date and time in 'YYYY-MM-DD HH:MM:SS' format

    IF username NOT IN reports THEN
        SET reports[username] TO an empty list
    ENDIF

    APPEND a dictionary with 'time' as current_time and 'action' as action TO reports[username]

    CALL save_data with reports and REPORTS_FILE as arguments
ENDFUNCTION
```

Modify Staff Password Function

```
FUNCTION modify_staff_password(username, new_password, ic, active, filename)
    OPEN filename AS filehandler IN READ MODE
    IF filename FOUND THEN
        OPEN filename FOR writing AS file
        SET found TO False

        FOR each line IN lines DO
            SPLIT line INTO user_data USING ','

            IF user_data[0] == username AND user_data[2] == ic AND user_data[3] == active THEN
                WRITE username, new_password, ic, active TO file
                SET found TO True
            ELSE
                WRITE line TO file
            ENDIF
        ENDIF

        IF found IS False THEN
            RETURN False, "Username, Identity Card, or account status does not match."
        ENDIF

        RETURN True, "Password updated successfully."
    ELSE
        PRINT "File not found"
    ENDFIF
ENDFUNCTION
```

Staff Login Function

```
FUNCTION StaffLogIn()  
    SET max_attempts = 3  
    SET attempts = 0  
  
    PRINT "Inventory Staff Log In Page"  
  
    WHILE attempts < max_attempts  
        username = INPUT "Enter Username: "  
        password = INPUT "Enter Password: "  
        ic = INPUT "Enter Identity Card:"  
        active = "active"  
  
        IF verify_Staffcredentials(username, password, ic, active, "staff.txt") THEN  
            PRINT "Login successful! Welcome, " + username  
            CALL staff_menu(username)  
        ELSE  
            INCREMENT attempts  
            PRINT "Wrong username or password. You have " + (max_attempts - attempts) + " attempts left."  
  
            IF attempts == max_attempts THEN  
                PRINT "Too many failed attempts. Please reset your password."  
                valid, message = CALL reset_staff_password(username, ic, active, "staff.txt")  
                PRINT message  
            ENDIF  
        ENDIF  
    ENDWHILE  
ENDFUNCTION
```

Reset Staff Password Function

```
FUNCTION reset_staff_password(username, ic, active, filename)
    PRINT "Reset Password"
    SET attempt_count = 0
    SET max_attempts = 5

    WHILE attempt_count < max_attempts
        PRINT "Enter new_password"
        GET new_password

        IF new_password LENGTH < 8 THEN
            PRINT "Password must be at least 8 characters."
            INCREMENT attempt_count
            CONTINUE LOOP
        ENDIF

        SET has_upper TO CHECK IF new_password CONTAINS UPPERCASE LETTER
        SET has_digit TO CHECK IF new_password CONTAINS DIGIT
        SET has_special TO CHECK IF new_password CONTAINS SPECIAL CHARACTER

        IF NOT has_upper THEN
            PRINT "Password must contain at least 1 uppercase letter."
            INCREMENT attempt_count
            CONTINUE LOOP
        ENDIF

        IF NOT has_digit THEN
            PRINT "Password must contain at least 1 number."
            INCREMENT attempt_count
            CONTINUE LOOP
        ENDIF

        IF NOT has_special THEN
            PRINT "Password must contain at least 1 special character."
            INCREMENT attempt_count
            CONTINUE LOOP
        ENDIF

        PRINT "Confirm new password"
        GET confirm_password
        IF new_password != confirm_password THEN
            PRINT "Passwords do not match."
            INCREMENT attempt_count
            CONTINUE LOOP
        ENDIF

        SET valid, message TO CALL modify_staff_password(username, new_password, ic, active, filename)
        PRINT "Modify password result: " + valid + ", Message: " + message

        IF valid THEN
            RETURN TRUE, "Password reset successfully."
        ELSE
            RETURN FALSE, "Failed to reset password. Please try again."
        ENDIF
    ENDWHILE

    RETURN FALSE, "Too many failed attempts. Contact system administrator."
ENDFUNCTION
```

Verify Staff Credentials Function

```
FUNCTION verify_Staffcredentials(username, password, ic, active, filename)
  OPEN filename AS filehandler IN READMODE
  IF file FOUND THEN
    FOR EACH line IN file DO
      SPLIT line INTO stored_username, stored_password, stored_ic, stored_active

      IF username == stored_username AND
         password == stored_password AND
         ic == stored_ic AND
         active == stored_active THEN
        RETURN TRUE
      ENDIF
    ENDFOR
  ELSE
    PRINT "File not found"
  ENFIF
ENDFUNCTION
```


Staff Signup Function

```
FUNCTION StaffSignUp()
    SET role = "staff"
    SET status = "pending"

    PRINT "Inventory Staff Registration Page"

    // Create Username
    WHILE TRUE
        PRINT "Create username:"
        GET username
        IF check_username(username, "staff.txt") IS TRUE
            PRINT "Username already exists. Please try another one."
        ELSE
            BREAK
        ENDIF
    ENDWHILE

    // Create Password
    WHILE TRUE
        PRINT "Create password (At least 8 characters, including uppercase letter, number, and special character):"
        GET password
        SET valid, message TO check_password(password)
        IF valid IS FALSE
            PRINT message
            CONTINUE
        ENDIF

        PRINT "Re-enter password:"
        IF verify_pw == password
            PRINT "-Password match-"
            BREAK
        ELSE
            PRINT "Password do not match. Please try again."
            CONTINUE
        ENDIF
    ENDWHILE

    // Check Identity Card
    WHILE TRUE
        PRINT "Identity Card:"
        GET ic
        SET ic TO INTEGER(user_input)
        IF check_ic(ic, "staff_ic.txt") IS FALSE
            PRINT "Invalid Identity Card. Please try again."
        ELSE
            PRINT "Your registration is pending to be approved."
            PRINT "You will be returned to Homepage."
            CALL save_pendingStaff(username, password, ic, role, status, "users.txt")
            CALL homepage()
            BREAK
        ENDIF
    ENDWHILE
ENDFUNCTION
```

Save Pending Staff Function

```

FUNCTION save_pendingStaff(username, password, ic, role, status, filename)
    OPEN file named `users.txt` in APPEND MODE
    WRITE lines to the file:
        username, password, ic, role, status
    CLOSE file
ENDFUNCTION

FUNCTION check_ic(ic, filename)
    OPEN file named filename in read mode
    IF file FOUND THEN
        existing_ic =[]
        FOR EACH line in the file
            SPLIT line by ","
            ADD the third element (IC) to existing_ic
        END FOR
        CLOSE file
        IF ic in existing_ic
            RETURN True
        ELSE
            RETURN False
        ENDIF
    ELSE
        PRINT "File not found"
    ENFIF
ENDFUNCTION

```

Staff Homepage Function

```
FUNCTION Staffhomepage()  
    PRINT "    Inventory Staff Homepage    "  
    PRINT "1. Login"  
    PRINT "2. Signup"  
    INPUT "Please enter your option:"  
    GET homepageoption  
  
    IF homepageoption == "1"  
        CALL StaffLogIn()  
    ELSE IF homepageoption == "2"  
        CALL StaffSignUp()  
    ELSE  
        PRINT "Invalid options. Please try again"  
        CALL Staffhomepage()  
    ENDIF  
ENDFUNCTION  
  
FUNCTION load_data(filename)  
    OPEN filename AS filehandler IN READ MODE  
    IF file FOUND THEN  
        RETURN JSON LOAD FROM file  
    ELSE  
        PRINT "File not found"  
    ENDIF  
ENDFUNCTION  
  
FUNCTION save_data(data, filename)  
    OPEN filename AS filehandler IN WRITING MODE  
    IF file FOUND THEN  
        WRITE data to JSON DUMP, WITH INDENT=4  
    ENDIF  
ENDFUNCTION
```

```

FUNCTION initialize_files()
    # Initialize STOCK_FILE with default values
    IF load_data(STOCK_FILE) IS EMPTY
        SET initial_stock TO
        {
            'Desktop': { 'quantity': 17, 'price': 2500, 'for_staff_only': FALSE },
            'Laptop': { 'quantity': 20, 'price': 4000, 'for_staff_only': FALSE },
            'Mouse': { 'quantity': 10, 'price': 80, 'for_staff_only': FALSE },
            'Keyboard': { 'quantity': 20, 'price': 150, 'for_staff_only': FALSE },
            'Monitor': { 'quantity': 10, 'price': 500, 'for_staff_only': FALSE },
            'Charger': { 'quantity': 10, 'price': 100, 'for_staff_only': FALSE },
            'CPU': { 'quantity': 10, 'price': 500, 'for_staff_only': TRUE }
        }
        CALL save_data(initial_stock, STOCK_FILE)
    ENDIF

    # Initialize SERVICE_ORDERS_FILE with default values
    IF load_data(SERVICE_ORDERS_FILE) IS EMPTY
        SET initial_service TO
        {
            'Computer Repair': { 'tentative_time': '3-5 days', 'service_price': 250 },
            'Device Repair': { 'tentative_time': '2-4 days', 'service_price': 100 },
            'Software Installation': { 'tentative_time': '1 day', 'service_price': 200 },
            'Hardware Upgrade': { 'tentative_time': '2-3 days', 'service_price': 250 }
        }
        CALL save_data(initial_service, SERVICE_ORDERS_FILE)
    ENDIF

    # Check if PURCHASED_ITEMS_FILE is empty
    IF load_data(PURCHASED_ITEMS_FILE) IS EMPTY
        CALL save_data([], PURCHASED_ITEMS_FILE)
    ENDIF

    # Initialize FEEDBACK_FILE with default values
    IF load_data(FEEDBACK_FILE) IS EMPTY
        SET initial_feedback TO
        {
            'username': 'JohnDoe',
            'date': '2024-08-14 10:15:30',
            'rating': 5,
            'comments': 'Great service!'
        }
        CALL save_data(initial_feedback, FEEDBACK_FILE)
    ENDIF
ENDFUNCTION

```

Update Stock Function

```

FUNCTION update_stock(item, quantity)
    # Load current stock data
    SET stock TO load_data(STOCK_FILE)

    # Check if the item exists in the stock
    IF item EXISTS IN stock
        # Update the stock quantity for the item
        stock[item]['quantity'] = stock[item]['quantity'] + quantity

        # Save the updated stock data
        CALL save_data(stock, STOCK_FILE)

        # Display the updated stock information
        PRINT "Stock updated: " + item + " now has " + stock[item]['quantity'] + " units."
    ELSE
        PRINT "Invalid item for stock update."
    ENDIF
ENDFUNCTION

```

Inventory Modify Order Function

```
FUNCTION inventory_modify_order(username, order_id, item, new_quantity)
    # Load existing orders and stock data
    SET orders TO load_data(PURCHASED_ITEMS_FILE)
    SET stock TO load_data(STOCK_FILE)

    # Check if the order ID is valid
    IF order_id IS BETWEEN 1 AND LENGTH(orders) THEN
        SET order TO orders[order_id - 1]

        # Check if the order status is "Paid"
        IF order['status'] IS EQUAL TO 'Paid' THEN
            PRINT "Cannot modify a paid order."
            RETURN

        SET items TO order['items'] OR []

        # Find the item in the order
        SET item_to_modify TO NULL
        FOR EACH i IN items DO
            IF i['item'] IS EQUAL TO item THEN
                SET item_to_modify TO i
                BREAK
            ENDIF
        ENDFOR

        # If item not found, exit the function
        IF item_to_modify IS NULL THEN
            PRINT "Item not found in the order."
            RETURN
        ENDIF

        SET old_quantity TO item_to_modify['quantity']
        SET quantity_difference TO new_quantity - old_quantity

        # Update stock levels based on the difference in quantity
        CALL update_stock(item, quantity_difference)

        # Update item with new quantity and recalculate its total cost
        SET item_to_modify['quantity'] TO new_quantity
        SET item_to_modify['total_cost'] TO stock[item]['price'] * new_quantity

        # Recalculate the total cost for the entire order
        SET order['total_cost'] TO SUM(
            i['total_cost'] OR (stock[i['item']]['price'] * i['quantity']) FOR EACH i IN items
        )

        PRINT "Order modified."
        PRINT "Item:", item
        PRINT "Old Quantity:", old_quantity, "New Quantity:", new_quantity
        PRINT "New Total Cost for Item:", item_to_modify['total_cost']
        PRINT "New Total Cost for Order:", order['total_cost']

        # Save the updated orders
        CALL save_data(orders, PURCHASED_ITEMS_FILE)
    ELSE
        PRINT "Invalid order ID."
    ENDIF

    # Log the action performed
    CALL log_action(username, "Stock Check - Modify Purchase Order")
ENDFUNCTION
```

Inventory Cancel Order Function

```

FUNCTION inventory_cancel_order(username, order_id)
  OPEN filename AS filehandler IN READ MODE
  IF file FOUND THEN
    SET orders TO LOAD_DATA(PURCHASED_ITEMS_FILE)

    IF order_id >= 1 AND order_id <= LENGTH(orders) THEN
      SET order TO orders[order_id - 1]
      SET items TO order.GET('items', [])

      IF order.GET('status') == "paid" THEN
        PRINT "Paid orders are non-refundable unless the product is unavailable."
      ELSE
        // Update stock for each item in the order
        FOR EACH item IN items
          SET item_name TO item.GET('item')
          SET quantity TO item.GET('quantity')
          CALL update_stock(item_name, quantity)

        // Remove the order from the list
        REMOVE orders[order_id - 1]

        // Save the updated orders list
        CALL save_data(orders, PURCHASED_ITEMS_FILE)

        PRINT "Order ID: " + order_id + " canceled."
        CALL log_action(username, "Cancel Purchase Order")
      ENDIF
    ELSE
      PRINT "Invalid order ID."
    ENDIF
  ELSE
    PRINT "File not found"
  ENDIF
ENDFUNCTION

```

Inventory Stock Check Function

```
FUNCTION inventory_stock_check(username, for_staff=False)
    SET stock TO CALL get_stock()

    WHILE True
        PRINT "Current stock:"
        FOR EACH idx, (item, details) IN ENUMERATE(stock.items(), start=1) DO
            SET quantity TO details.GET('quantity', 0)
            SET price TO details.GET('price', 0.0)
            SET item_type TO details.GET('type', 'Unknown')
            SET for_staff_only TO details.GET('for_staff_only', False)
            SET visibility TO 'Staff Only' IF for_staff_only ELSE 'Staff and Customer'

            IF NOT for_staff OR (for_staff AND for_staff_only) THEN
                PRINT idx + ". " + item
                PRINT "    Quantity: " + quantity
                PRINT "    Price: RM" + FORMAT(price, 2)
                PRINT "    Available for: " + visibility

        PRINT "Options:"
        PRINT "1. Update Quantity"
        PRINT "2. Update Price"
        PRINT "3. Add New Item"
        PRINT "4. Remove Item"
        PRINT "5. Exit"

        INPUT("Select an option (1-5): ")
        GET option

        IF option == '1' THEN
            SET item_index TO INPUT("Enter the item number to update quantity: ") - 1
            IF 0 <= item_index < LENGTH(stock) THEN
                SET item TO LIST(stock.keys())[item_index]
                SET new_quantity TO INPUT("Enter the new quantity: ")
                stock[item]['quantity'] = new_quantity
                CALL save_data(stock, STOCK_FILE)
                PRINT "Updated quantity for " + item + " to " + new_quantity
            ELSE
                PRINT "Invalid item number."
            ENDIF
        ENDIF
        CALL log_action(username, "Stock Check - Update Quantity")
```

```

ELSE IF option == '2' THEN
    SET item_index TO INPUT("Enter the item number to update price: ") - 1
    IF 0 <= item_index < LENGTH(stock) THEN
        SET item TO LIST(stock.keys())[item_index]
        SET new_price TO INPUT("Enter the new price: RM")
        stock[item]['price'] = new_price
        CALL save_data(stock, STOCK_FILE)
        PRINT "Updated price for " + item + " to RM" + FORMAT(new_price, 2)
    ELSE
        PRINT "Invalid item number."
    ENDIF
    CALL log_action(username, "Stock Check - Update Price")

ELSE IF option == '3' THEN
    SET item TO INPUT("Enter the name of the new item: ")
    SET initial_quantity TO INPUT("Enter the initial quantity of the new item: ")
    SET initial_price TO INPUT("Enter the price of the new item: RM")
    SET for_staff_only TO INPUT("Is this item available for staff only? (Y for Yes | N for No): ").STRIP().UPPER()

    IF item IN stock THEN
        PRINT item + " already exists in stock."
    ELSE
        IF for_staff_only == 'Y' THEN
            SET for_staff_only TO True
        ELSEIF for_staff_only == 'N' THEN
            SET for_staff_only TO False
        ELSE
            PRINT "Invalid input. Please try again."
            RETURN // Exit the function if input is invalid
        ENDIF
    ENDIF
    stock[item] = {'quantity': initial_quantity, 'price': initial_price, 'for_staff_only': for_staff_only}
    CALL save_data(stock, STOCK_FILE)

    PRINT "New item registered: " + item
    PRINT "    Quantity: " + initial_quantity
    PRINT "    Price per unit: RM" + FORMAT(initial_price, 2)
    PRINT "    For staff only: " + for_staff_only

    CALL log_action(username, "Stock Check - Add New Item")

ELSEIF option == '4' THEN
    SET item_index TO INPUT("Enter the item number to remove: ") - 1
    IF 0 <= item_index < LENGTH(stock) THEN
        SET item TO LIST(stock.keys())[item_index]
        DELETE stock[item]
        CALL save_data(stock, STOCK_FILE)
        PRINT "Removed item: " + item
    ELSE
        PRINT "Invalid item number."
    ENDIF
    CALL log_action(username, "Stock Check - Remove Item")

ELSEIF option == '5' THEN
    PRINT "Exiting stock check."
    CALL log_action(username, "Stock Check - Exit")
    BREAK // Exit the loop

ELSE
    PRINT "Invalid option. Please try again."
ENDIF

ENDWHILE
ENDFUNCTION

```


Inventory Purchase Order Function

```
FUNCTION inventory_purchase_order(username)
    SET stock TO CALL get_stock()
    SET purchased_items TO CALL load_data(PURCHASED_ITEMS_FILE)
    SET user_orders TO [order FOR EACH order IN purchased_items IF order.GET('username') == username]

    PRINT "Available Items for Inventory Staff:"
    FOR EACH idx, (item, details) IN ENUMERATE(stock.items(), start=1) DO
        PRINT idx + ". " + item
        PRINT "    Quantity: " + details['quantity']
        PRINT "    Price: RM" + FORMAT(details['price'], 2)

    SET order_items TO EMPTY_LIST()

    OPEN filename AS filehandle IN READ MODE
    IF file FOUND THEN
        WHILE True
            SET item_idx TO INPUT("Enter the item number to order (or 0 to finish): ")

            IF item_idx == 0 THEN
                BREAK // Exit the loop if finished ordering
            ENDIF

            IF item_idx < 1 OR item_idx > LENGTH(stock) THEN
                PRINT "Invalid item number."
                CONTINUE // Skip to the next iteration
            ENDIF

            SET item TO LIST(stock.keys())[item_idx - 1]
            SET quantity TO INPUT("Enter quantity for " + item + ": ")

            SET total TO stock[item]['price'] * quantity
            CALL APPEND(order_items, {'item': item, 'quantity': quantity, 'price': stock[item]['price'], 'total_cost': total})

            CALL update_stock(item, +quantity)

        IF LENGTH(order_items) == 0 THEN
            PRINT "No items were ordered."
            RETURN // Exit if no items were ordered
        ENDIF

        SET total_cost TO SUM(item['total_cost'] FOR EACH item IN order_items)
        SET status TO "Unpaid"
        CALL save_purchase_record(username, order_items, total_cost, status)
        CALL save_data(stock, "stock.json")

        PRINT "Purchase Order Placed Successfully!"
        FOR EACH idx, item IN ENUMERATE(order_items, start=1) DO
            PRINT idx + ". " + item['item']
            PRINT "    Quantity: " + item['quantity']
            PRINT "    Price: RM" + FORMAT(item['price'], 2)

        PRINT "Total Price: RM" + FORMAT(total_cost, 2)
        PRINT "Status: " + status
        ENDFOR
        CALL log_action(username, "Make Purchase Order")
        ENDWHILE

    ELSE
        PRINT "File not found"
    ENDIF

    CALL log_action(username, "Fail to Make Purchase Order")
ENDFUNCTION
```

Inventory Check Purchase Order Status Function

```
FUNCTION inventory_check_purchase_order_status(username)
    CALL log_action(username, "Check Purchase Order Status")

    OPEN filename AS filehandler IN READ MODE
    IF file FOUND THEN
        SET orders TO CALL load_data(PURCHASED_ITEMS_FILE)

        SET user_orders TO [order FOR EACH order IN orders IF order.GET('username') == username]

        IF LENGTH(user_orders) == 0 THEN
            PRINT "No orders found."
            CALL staff_menu(username)
        ELSE
            PRINT "Purchase orders:"
            FOR EACH index, order IN ENUMERATE(user_orders, start=1) DO
                SET status TO IF order.GET('status', 'Unpaid') == 'Paid' THEN 'Paid' ELSE 'Unpaid'
                SET total_cost TO order.GET('total_cost', 0.0)

                PRINT "-----Order ID: " + index + "-----"

                IF 'items' IN order AND ISINSTANCE(order['items'], list) THEN
                    FOR EACH idx, item IN ENUMERATE(order['items'], start=1) DO
                        SET quantity TO item.GET('quantity', 'Unknown')
                        SET item_name TO item.GET('item', 'Unknown')
                        SET price TO item.GET('price', 0.0)
                        PRINT idx + ". " + item_name
                        PRINT "    Quantity: " + quantity
                        PRINT "    Price per unit: RM" + FORMAT(price, 2)
                    ENDFOR
                ELSE
                    PRINT "    - No items found in this order."
                ENDIF
                PRINT "Total Price: RM" + FORMAT(total_cost, 2)
                PRINT "Status: " + status
            ENDFOR
        ENDIF
    ELSE
        PRINT "File not found"
    ENDIF
ENDFUNCTION
```

Service Order Function

```
FUNCTION service_order(username)
    SET services TO CALL load_data(SERVICE_ORDERS_FILE)

    PRINT "Available Service/Repairs:"
    FOR EACH idx, (service, details) IN ENUMERATE(services.items(), start=1) DO
        PRINT idx + ". " + service + " : " + details['tentative_time']
        PRINT "    Price: RM" + FORMAT(details['service_price'], 2)
    ENDFOR
    SET service_options TO INPUT("Enter Y to modify or N to exit.")

    IF service_options == 'Y' THEN
        SET service_name TO INPUT("Enter the service name you want to modify: ").strip()

        IF service_name IN services THEN
            SET new_name TO INPUT("Enter new service name (or press Enter to keep the current name): ").strip()
            SET new_time TO INPUT("Enter new tentative time (or press Enter to keep the current time): ").strip()
            SET new_price TO INPUT("Enter new service price (or press Enter to keep the current price): RM ").strip()

            IF new_name THEN
                services[new_name] = services.POP(service_name)
                SET service_name TO new_name
            ENDIF
            IF new_time THEN
                services[service_name]['tentative_time'] = new_time
            ENDIF
            IF new_price THEN
                OPEN filename AS filehandler IN READ MODE
                IF file FOUND THEN
                    services[service_name]['service_price'] = CONVERT_TO_FLOAT(new_price)
                ELSE
                    PRINT "Invalid price input. Keep the old price."
                ENDIF
            ENDIF
            CALL save_data(services, SERVICE_ORDERS_FILE)
            PRINT "Service updated successfully."
            CALL log_action(username, "Modify Service Order")
        ELSE
            PRINT "Service not found."
        ENDIF
    ENDIF
    CALL staff_menu(username)
ENDFUNCTION
```

Inventory Payment Function

```
FUNCTION inventory_payment(username)
    CALL log_action(username, "Make Payment")

    # Load purchase orders and filter for the specific user
    SET orders TO CALL load_data(PURCHASED_ITEMS_FILE)
    SET user_orders TO [order FOR EACH order IN orders IF order.get('username') == username]

    IF user_orders IS EMPTY THEN
        PRINT "No purchase orders found for " + username + ". No payment needed."
        RETURN
    ENDIF

    PRINT "Your Purchase Orders:"
    FOR EACH index, order IN ENUMERATE(user_orders, start=1) DO
        # Get the order status, default to 'Unpaid' if not set
        SET status TO "Paid" IF order.get('status', 'Unpaid') == 'Paid' ELSE "Unpaid"
        SET total_cost TO order.get('total_cost', 0.0)

        PRINT "-----Order ID: " + index + "-----"

        # Display Purchase Order
        IF 'items' IN order AND IS_LIST(order['items']) THEN
            FOR EACH idx, item IN ENUMERATE(order['items'], start=1) DO
                SET item_name TO item.get('item', 'Unknown')
                SET quantity TO item.get('quantity', 'Unknown')
                SET price TO item.get('price', 0.0)
                PRINT idx + ". " + item_name
                PRINT "    Quantity: " + quantity
                PRINT "    Price per unit: RM" + FORMAT(price, 2)
            ENDFOR
            PRINT "Total Price: RM" + FORMAT(total_cost, 2)
            PRINT "Status: " + status
        ELSE
            PRINT "    - No items found in this order."
        ENDIF
    ENDFOR

    # Ask for Order ID to make payment
    OPEN filename AS filehandler IN READ MODE
    IF file FOUND THEN
        SET payment_id TO CONVERT_TO_INT(INPUT("Select your Order ID to make payment: "))

        IF 1 <= payment_id <= LENGTH(user_orders) THEN
            SET selected_order TO user_orders[payment_id - 1]

            # Check current status
            IF selected_order.get('status', 'Unpaid') == 'Paid' THEN
                PRINT "This order is already paid."
                RETURN
            ENDIF

            SET answer TO INPUT("Do you want to make payment (Y for Yes, N for No): ").strip().upper()
            IF answer == 'Y' THEN
                selected_order['status'] = 'Paid'
                CALL save_data(orders, PURCHASED_ITEMS_FILE) # Save the updated orders
                PRINT "Payment successful. You will receive your goods in 3-4 working days."
            ELSE IF answer == 'N' THEN
                PRINT "Payment not completed. Don't forget to pay."
            ELSE
                PRINT "Invalid input. Please try again."
            ENDIF
        ELSE
            PRINT "Invalid Order ID."
        ENDIF
    ELSE
        PRINT "Invalid input. Please enter a valid number."
    ENDIF

    CALL log_action(username, "Attempted to Make Payment")
ENDFUNCTION
```

Save View Reports Function

```
FUNCTION staff_view_reports(username)
    SET reports TO CALL load_data(REPORTS_FILE)
    SET user_reports TO reports.get(username, [])

    IF user_reports IS EMPTY THEN
        PRINT "No reports found for user: " + username + "."
    ELSE
        PRINT "Reports for " + username + ":"
        PRINT "    Date    | Time    |    Actions"
        FOR EACH action IN user_reports DO
            PRINT " - " + action['time'] + ": " + action['action']
        ENDIF
        INPUT("Press Enter to return to the main menu...")
    ENDFUNCTION
```

Staff View Service Orders Function

```
FUNCTION staff_view_service_orders()
    SET service_orders TO CALL load_orders_details('service_orders.txt')

    IF service_orders IS EMPTY THEN
        PRINT "No service orders found."
        RETURN
    ENDIF

    PRINT "Order Summary"

    # Display each service order
    FOR EACH order_id, order IN service_orders DO
        PRINT "Order ID: " + order_id + ", Customer: " + order['username'] + ", Status: " + order['status'] + ", Total Price: RM" + order['total_price']
        FOR EACH item IN order['items'] DO
            PRINT " - " + item['name'] + " (RM" + item['price'] + ")"
        ENDFOR
    ENDFOR

    # Prompt for status update
    WHILE TRUE DO
        SET order_id TO INPUT("Enter the Order ID you want to modify (or type '0' to quit): ")
        IF order_id EQUALS '0' THEN
            BREAK
        ENDIF

        IF order_id NOT IN service_orders THEN
            PRINT "Invalid Order ID."
            CONTINUE
        ENDIF

        SET current_status TO service_orders[order_id]['status']
        PRINT "Current status for Order ID " + order_id + ": " + current_status

        SET new_status TO INPUT("Enter new status (Done/Processing): ").CAPITALIZE()
        IF new_status NOT IN ['Done', 'Processing'] THEN
            PRINT "Invalid status. Please enter 'Done' or 'Processing'."
            CONTINUE
        ENDIF

        # Update the order status
        service_orders[order_id]['status'] TO new_status
        PRINT "Order ID " + order_id + " status updated to " + new_status + "."

        # Save updated orders back to the file
        CALL save_orders_details('service_orders.txt', service_orders)
    ENDWHILE
ENDFUNCTION
```

Save Orders Details Function

```
FUNCTION save_orders_details(filename, orders)
    OPEN file WITH filename IN WRITE MODE

    FOR EACH order_id, order IN orders DO
        SET items_str TO EMPTY LIST

        FOR EACH item IN order['items'] DO
            IF 'quantity' IN item THEN
                APPEND item['name'] + ", " + item['quantity'] + ", " + item['price'] TO items_str
            ELSE
                APPEND item['name'] + ", " + item['price'] TO items_str
            ENDIF
        ENDIF

        SET items_line TO JOIN items_str WITH " "
        SET line TO order_id + "; " + order['username'] + "; " + order['status'] + "; " + FORMAT(order['total_price'], 2) + "; " + items_line + NEWLINE

        WRITE line TO file
    ENDIF

    CLOSE file
ENDFUNCTION
```

Staff Menu Function

```
FUNCTION staff_menu(username)
    CALL initialize_files()
    CALL log_action(username, "Login")

    WHILE TRUE DO
        PRINT "\n*****"
        PRINT "      Inventory Menu      "
        PRINT "*****"
        PRINT "1. Place Purchase Order"
        PRINT "2. Modify Purchase Order"
        PRINT "3. Cancel Purchase Order"
        PRINT "4. Check Purchase Order Status"
        PRINT "5. Stock Check/Adjustments"
        PRINT "6. Payment"
        PRINT "7. Service/ Repairs"
        PRINT "8. Report"
        PRINT "9. View Service Order"
        PRINT "10. Logout"

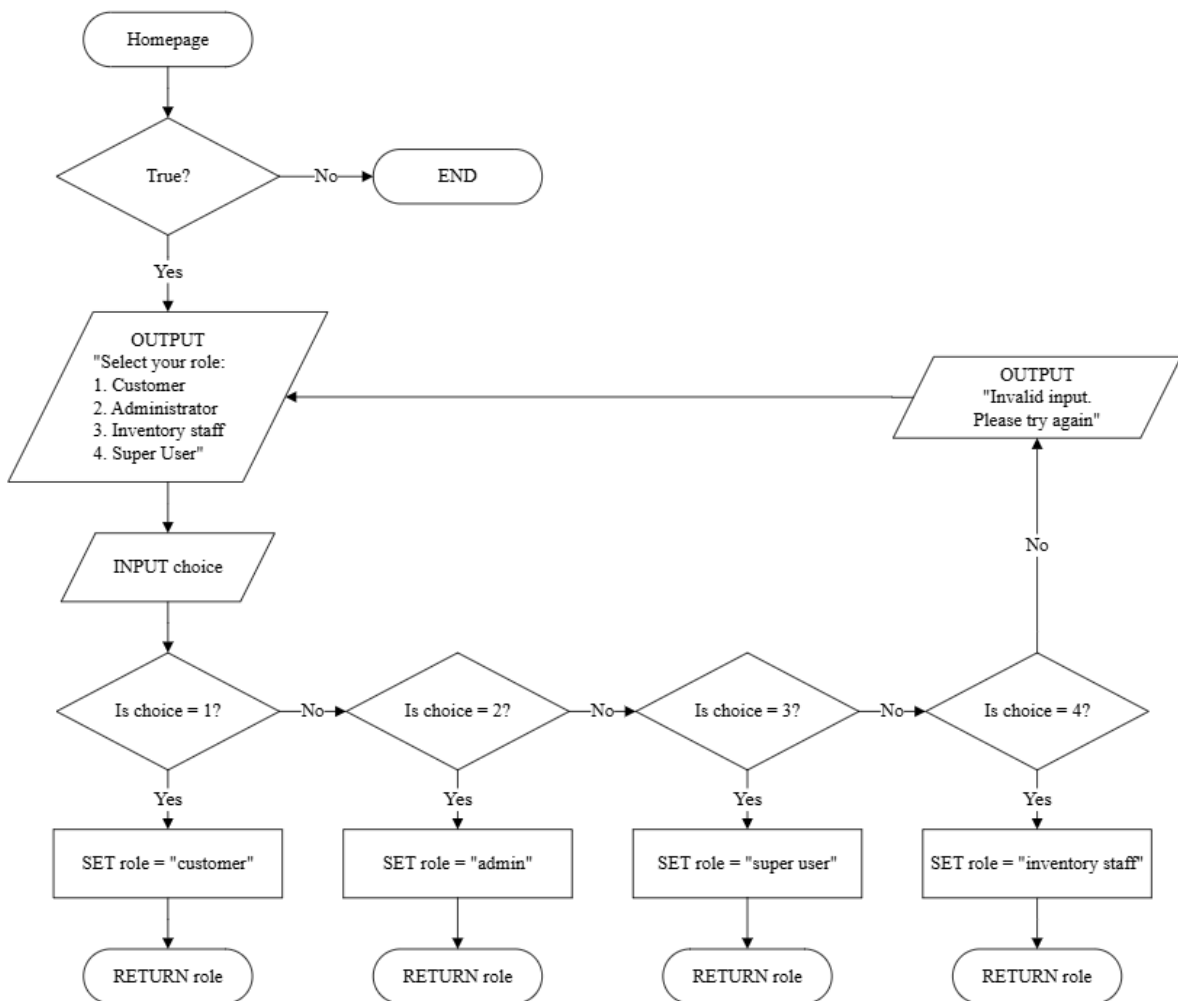
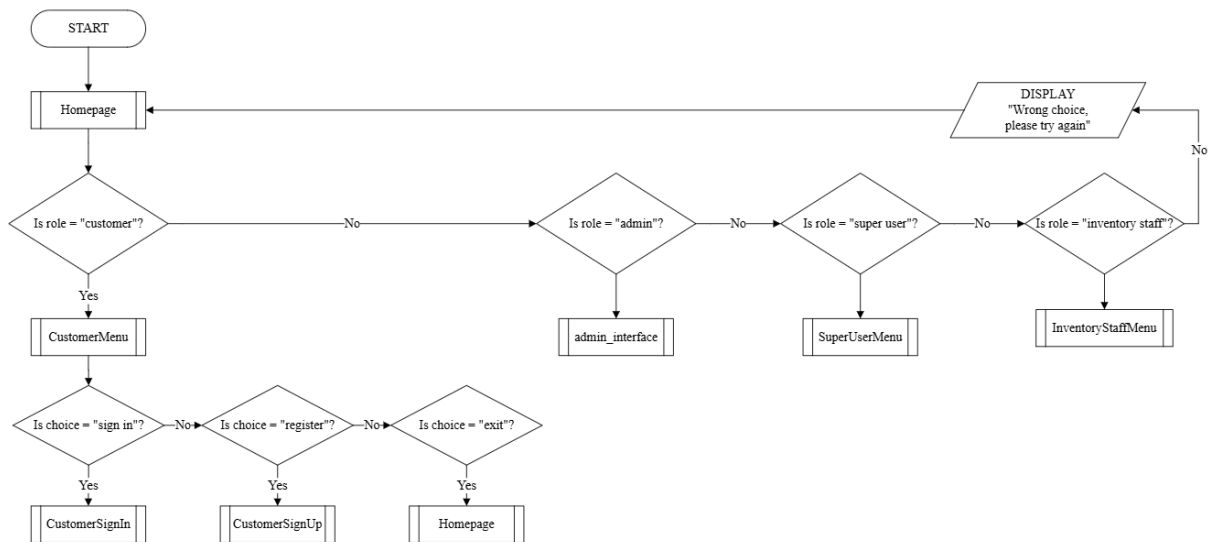
        SET choice TO INPUT "Choose an option: "

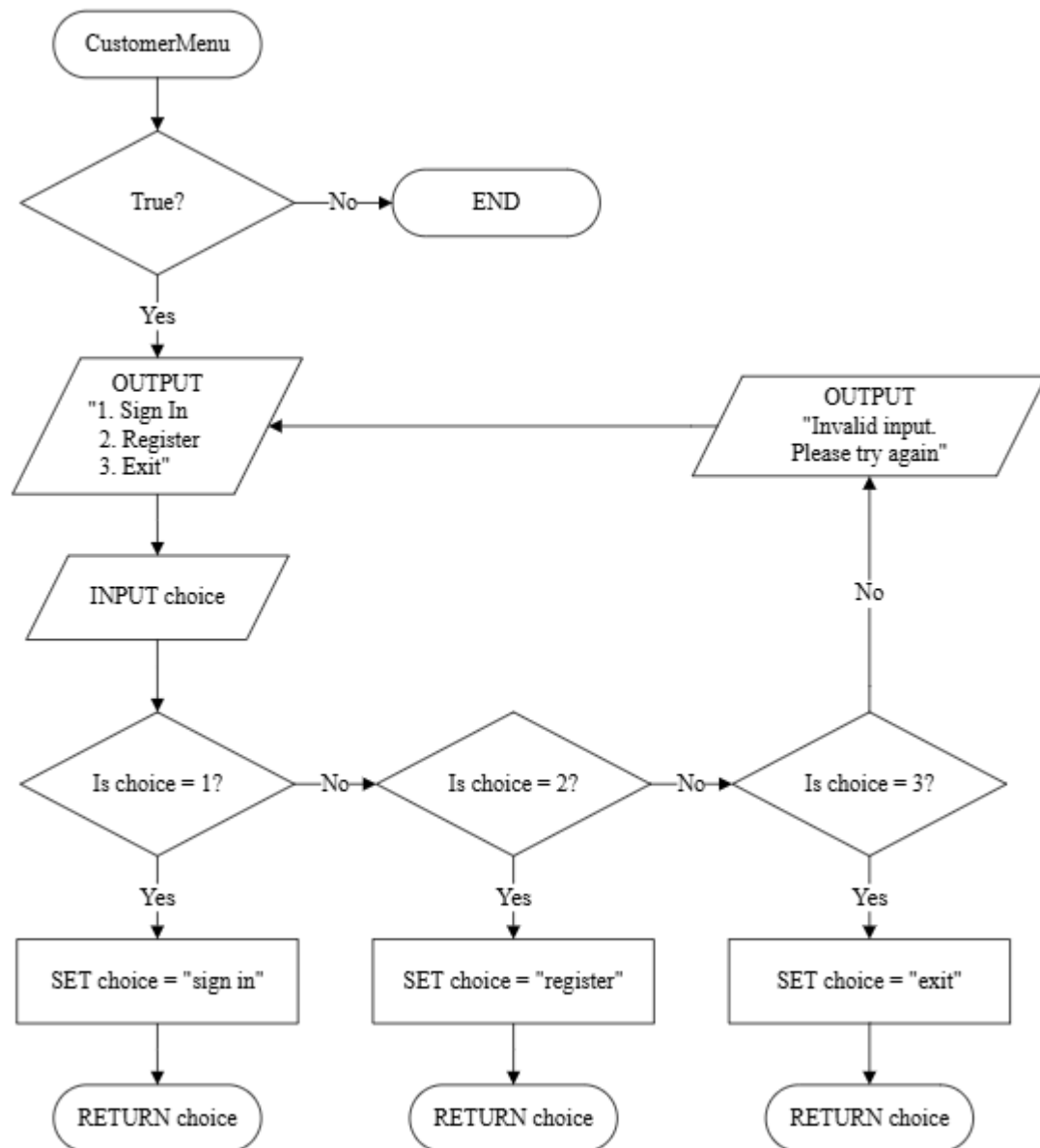
        IF choice == '1' THEN
            CALL inventory_purchase_order(username)
        ELSEIF choice == '2' THEN
            CALL inventory_check_purchase_order_status(username) // Show orders to find the ID
            SET order_id TO INPUT "Enter Order ID to modify: "
            SET item TO INPUT "Enter Item Name to modify: "
            SET new_quantity TO INPUT "Enter new quantity: "
            CALL inventory_modify_order(username, order_id, item, new_quantity)
        ELSEIF choice == '3' THEN
            CALL inventory_check_purchase_order_status(username) // Show orders to find the ID
            SET order_id TO INPUT "Enter order ID to cancel: "
            CALL inventory_cancel_order(username, order_id)
        ELSEIF choice == '4' THEN
            CALL inventory_check_purchase_order_status(username)
        ELSEIF choice == '5' THEN
            CALL inventory_stock_check(username, for_staff=FALSE)
        ELSEIF choice == '6' THEN
            CALL inventory_payment(username)
        ELSEIF choice == '7' THEN
            CALL service_order(username)
        ELSEIF choice == '8' THEN
            CALL log_action(username, "View Reports")
            CALL staff_view_reports(username)

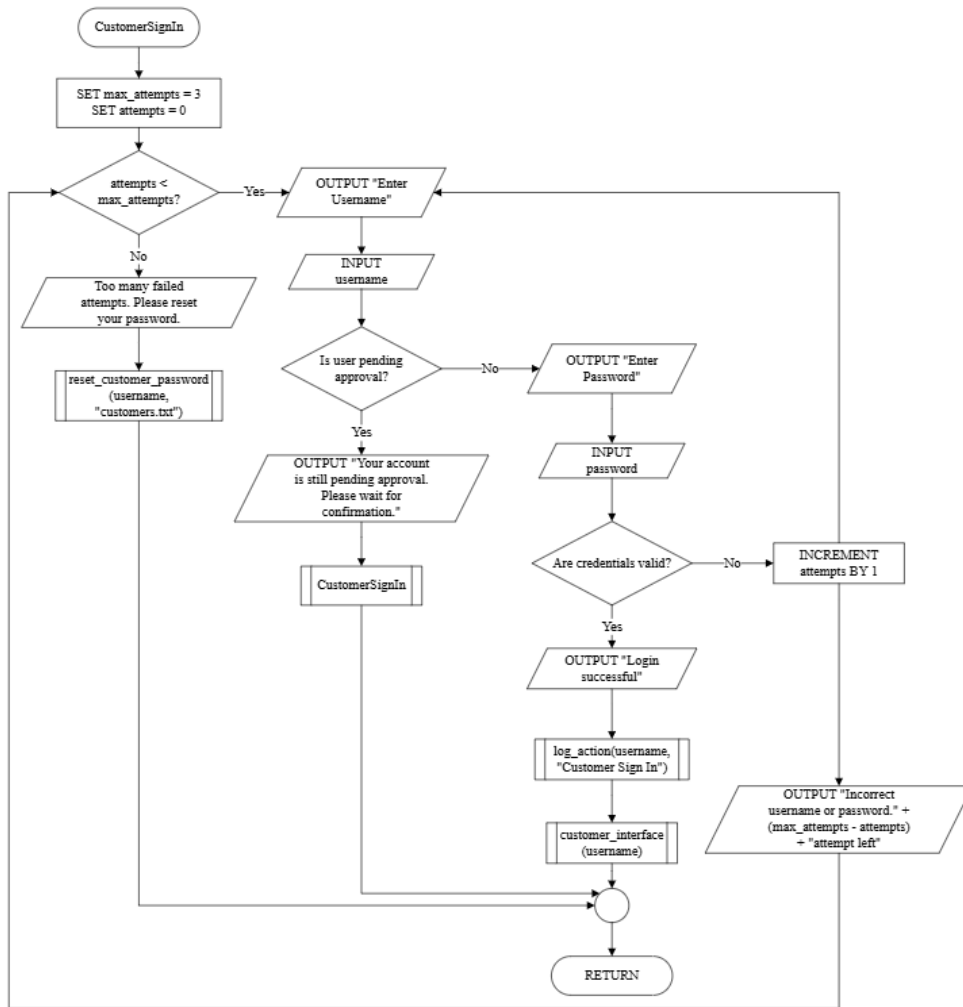
            ELSEIF choice == '9' THEN
                CALL log_action(username, "View Service Orders")
                CALL staff_view_service_orders()
            ELSEIF choice == '10' THEN
                CALL log_action(username, "Logout")
                CALL homepage()
            ELSE
                PRINT "Invalid choice. Please try again."
                CONTINUE
            ENDIF
        ENDIF
    ENDWHILE
ENDFUNCTION
```

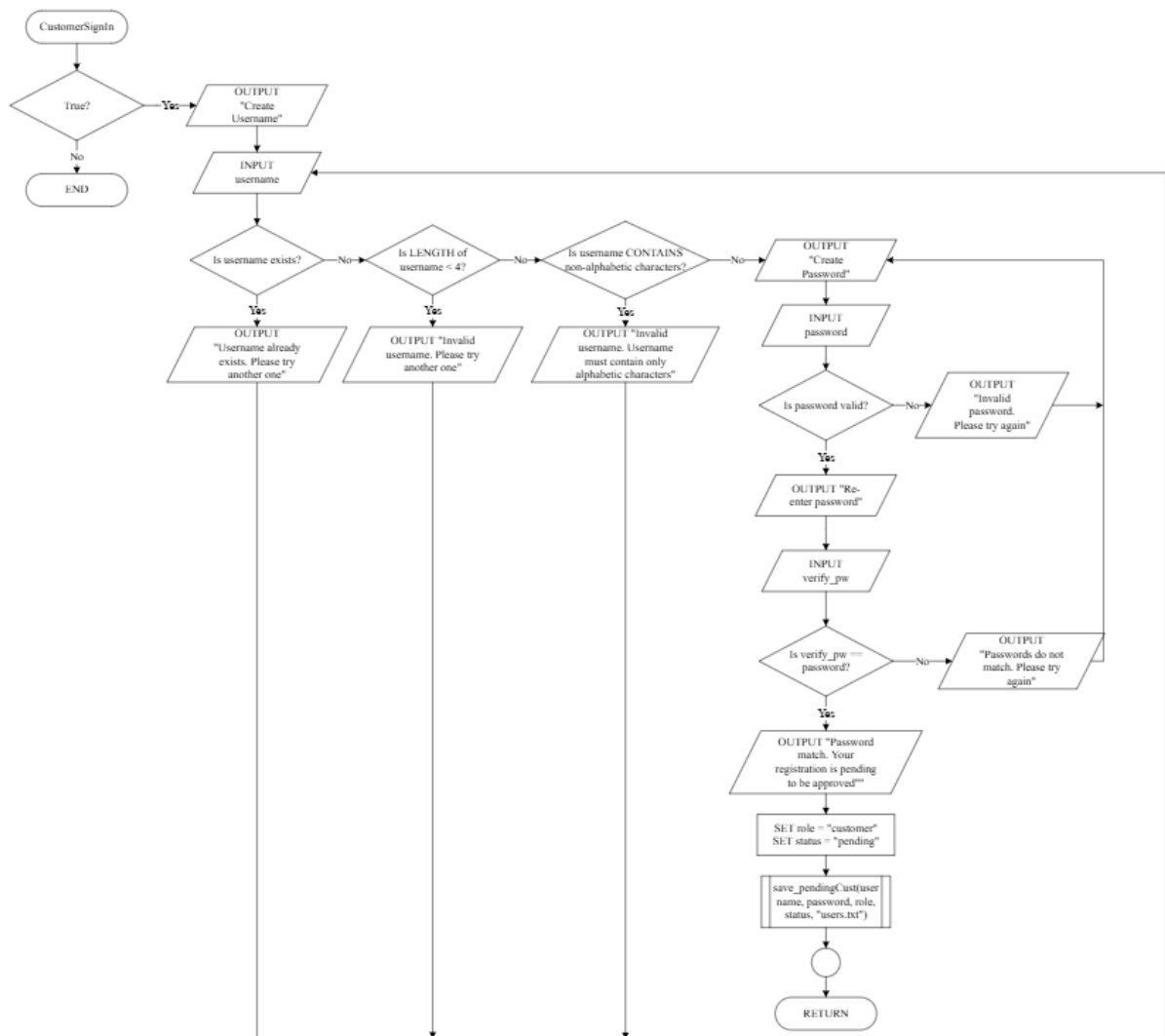
Flowchart

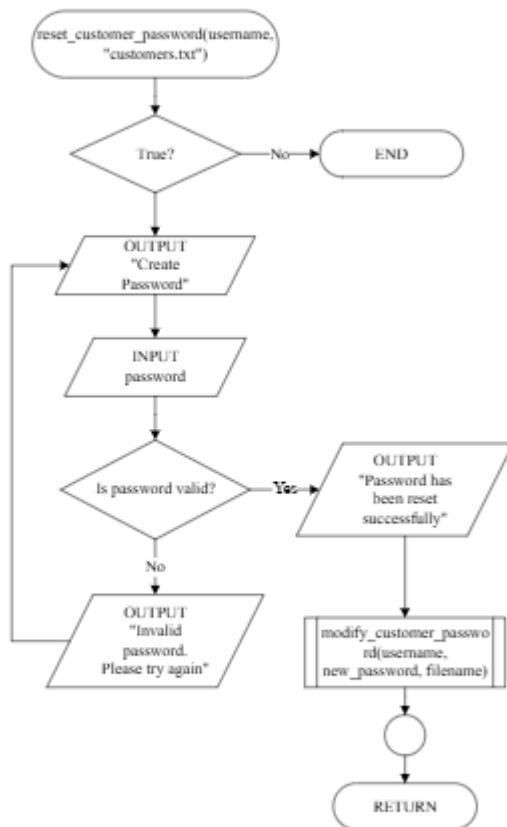
Customer flowchart







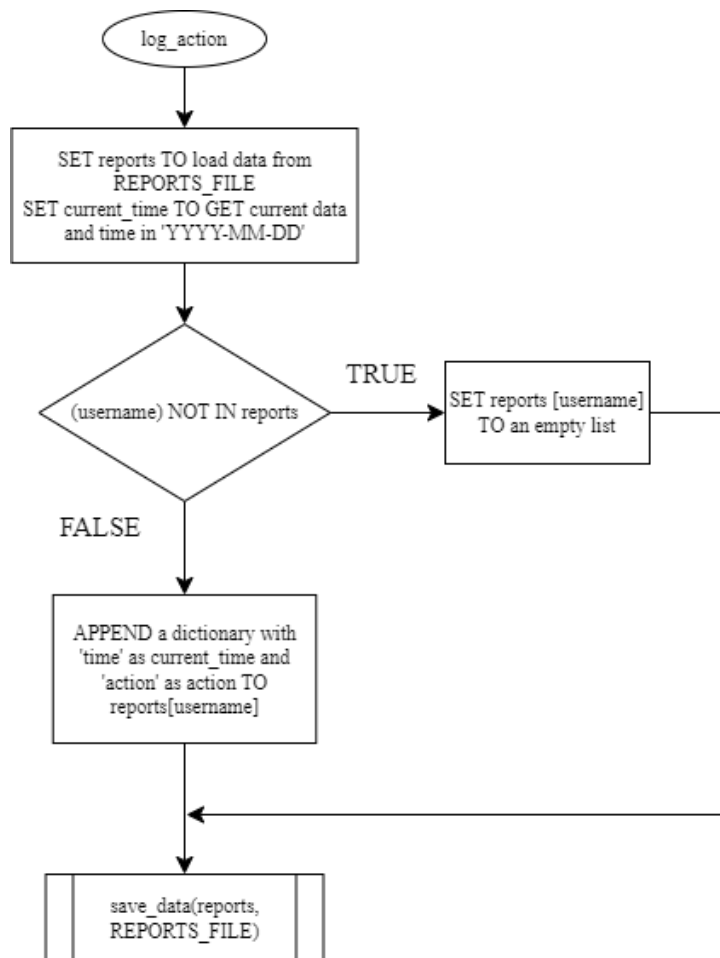




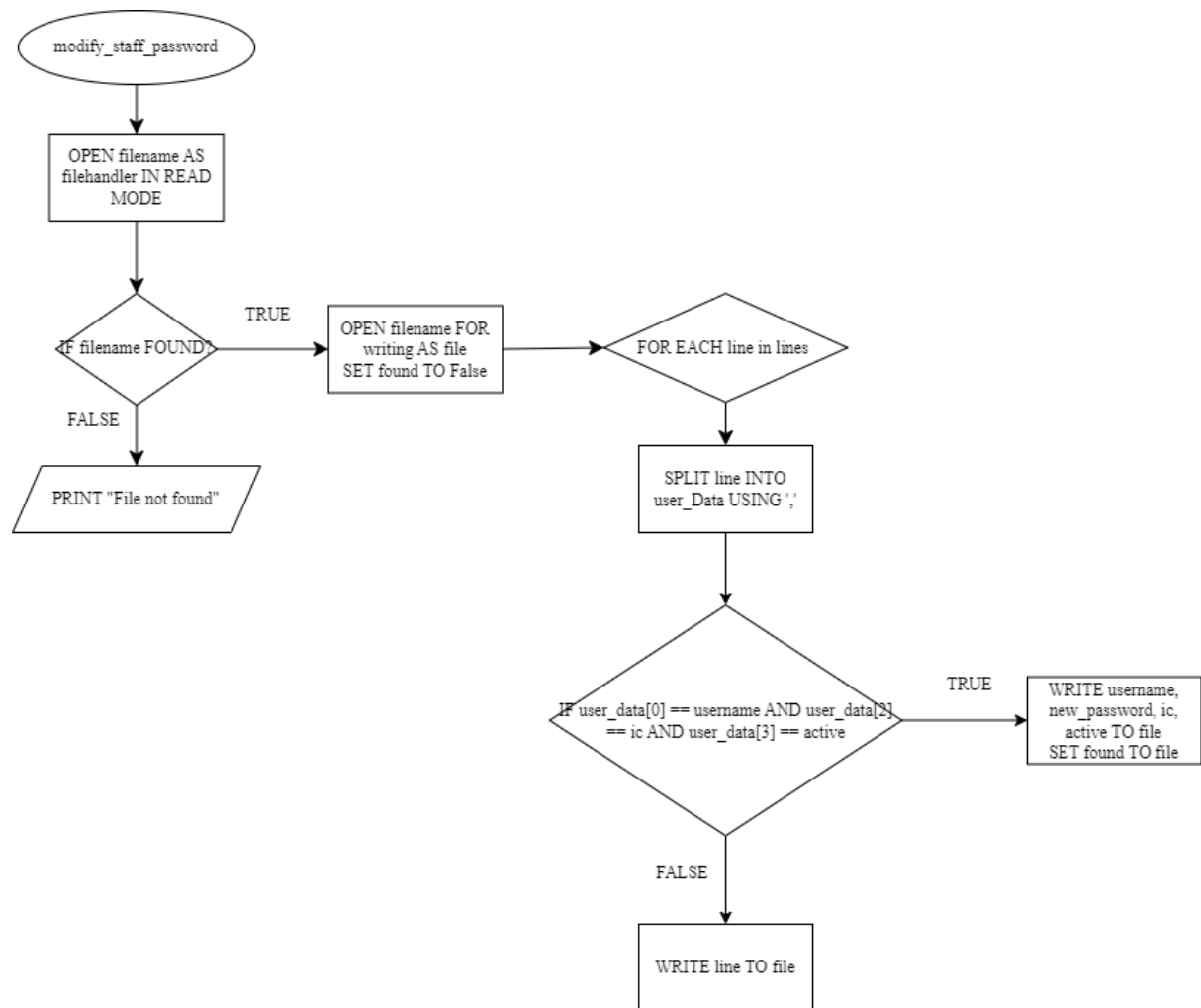
customer

Superuser

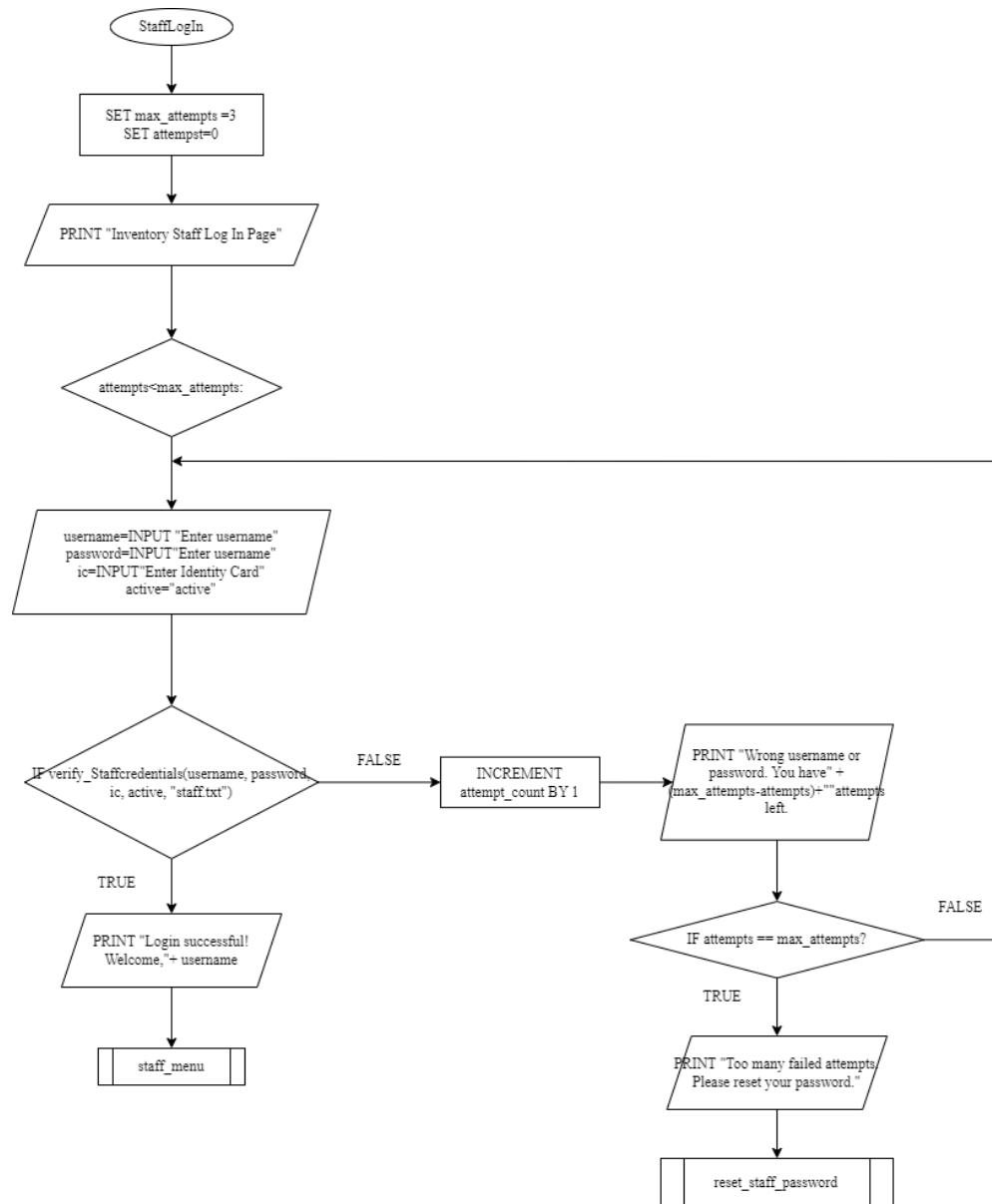
Log_action



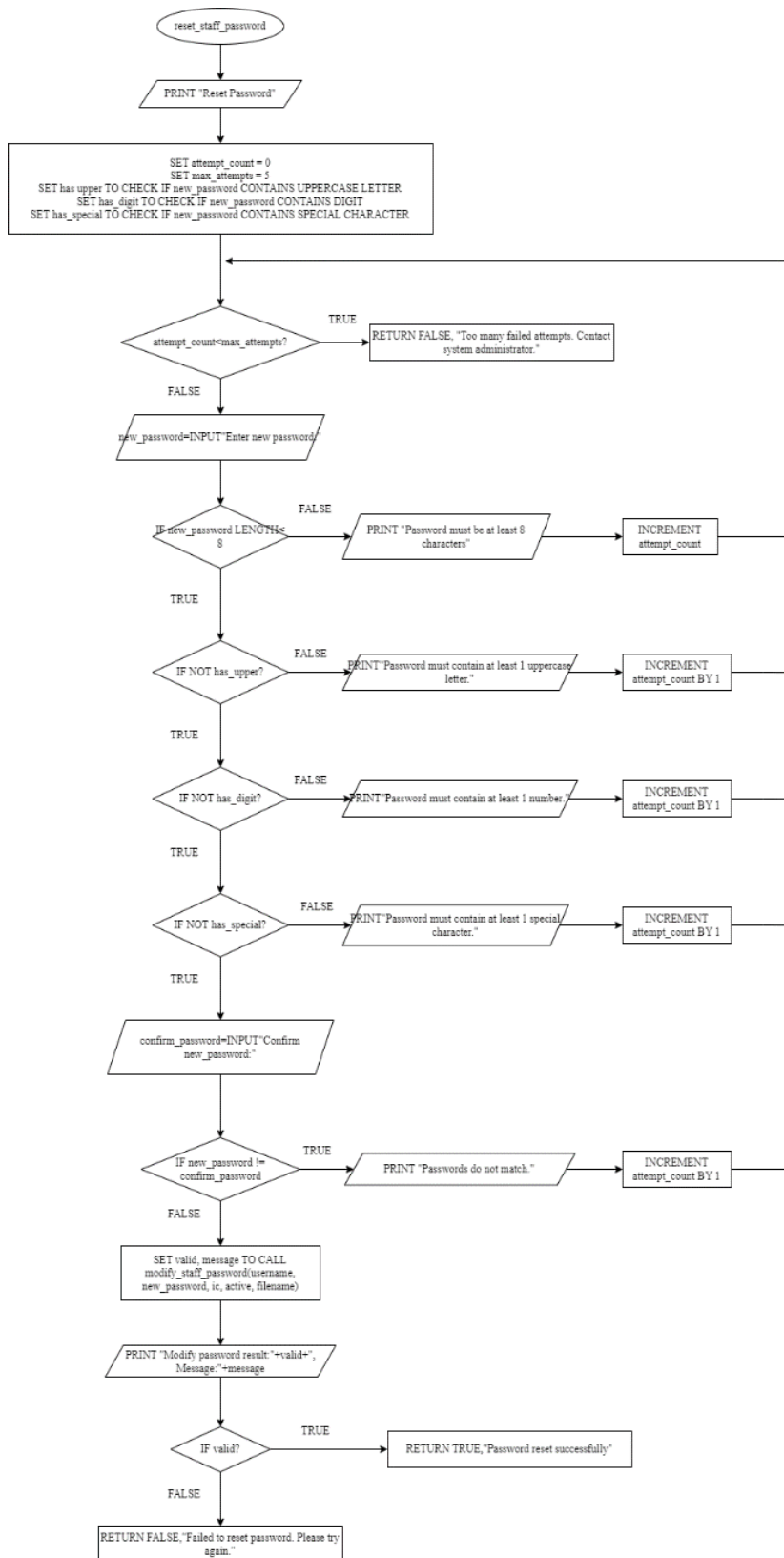
Modify Staff Password



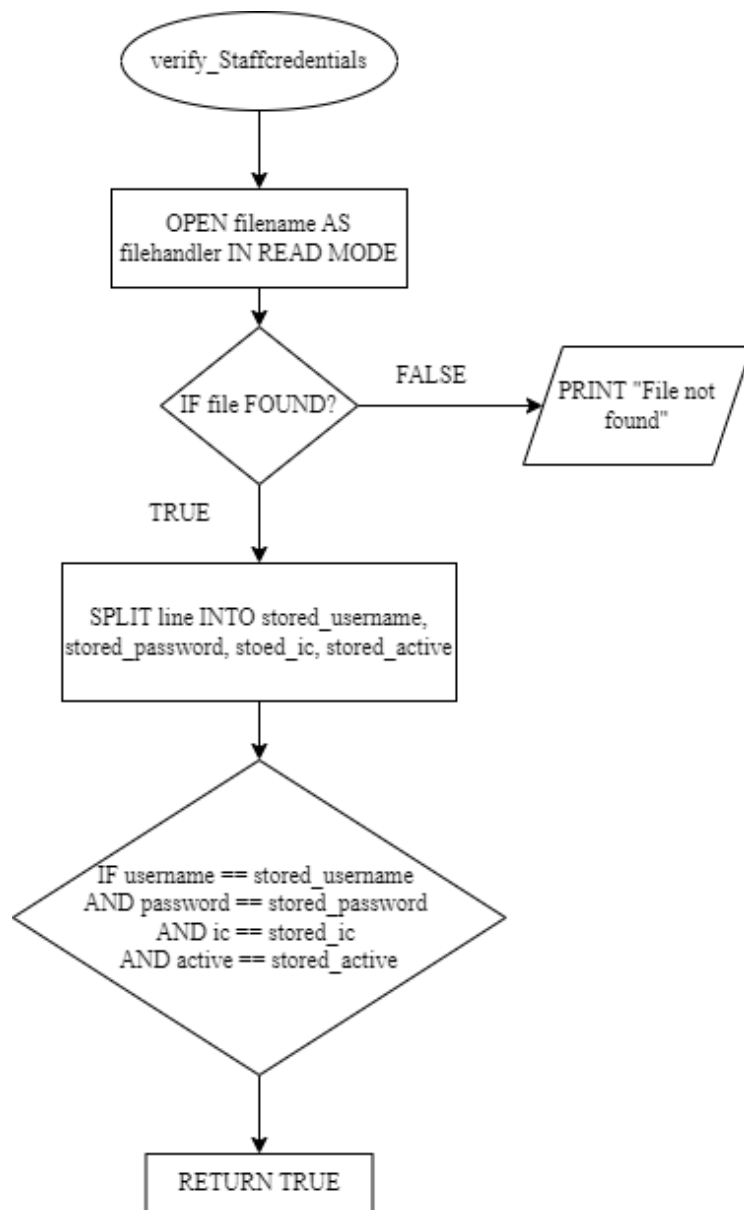
Staff Login



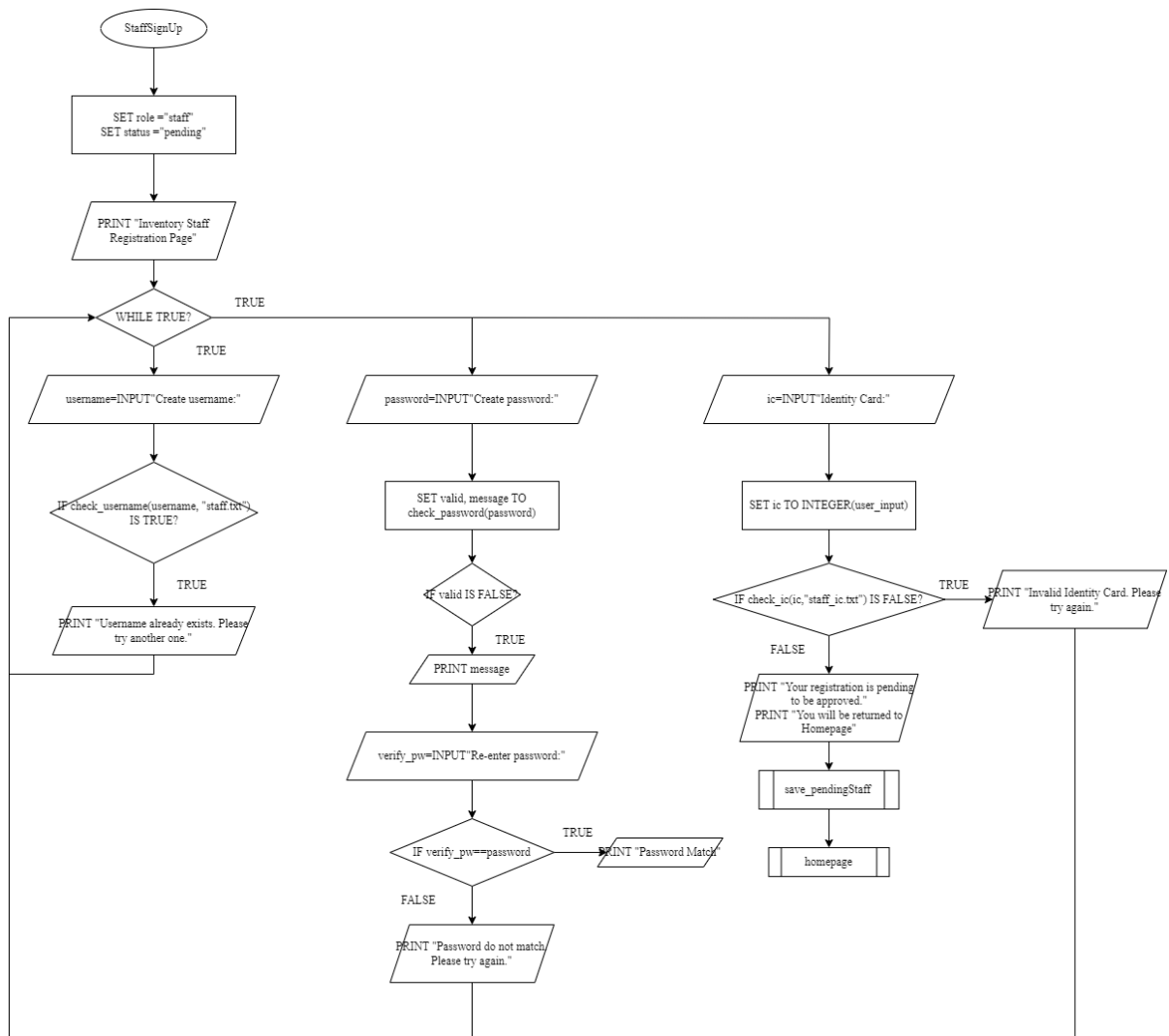
Reset Staff Password



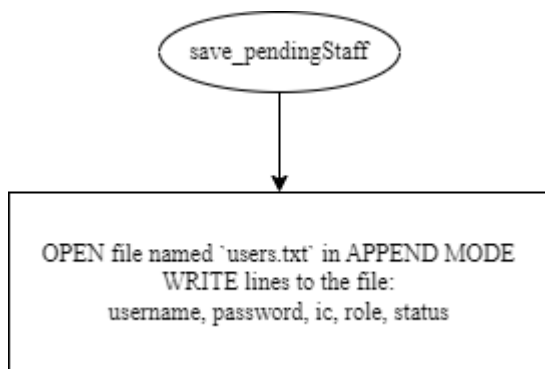
Verify Staff Credentials



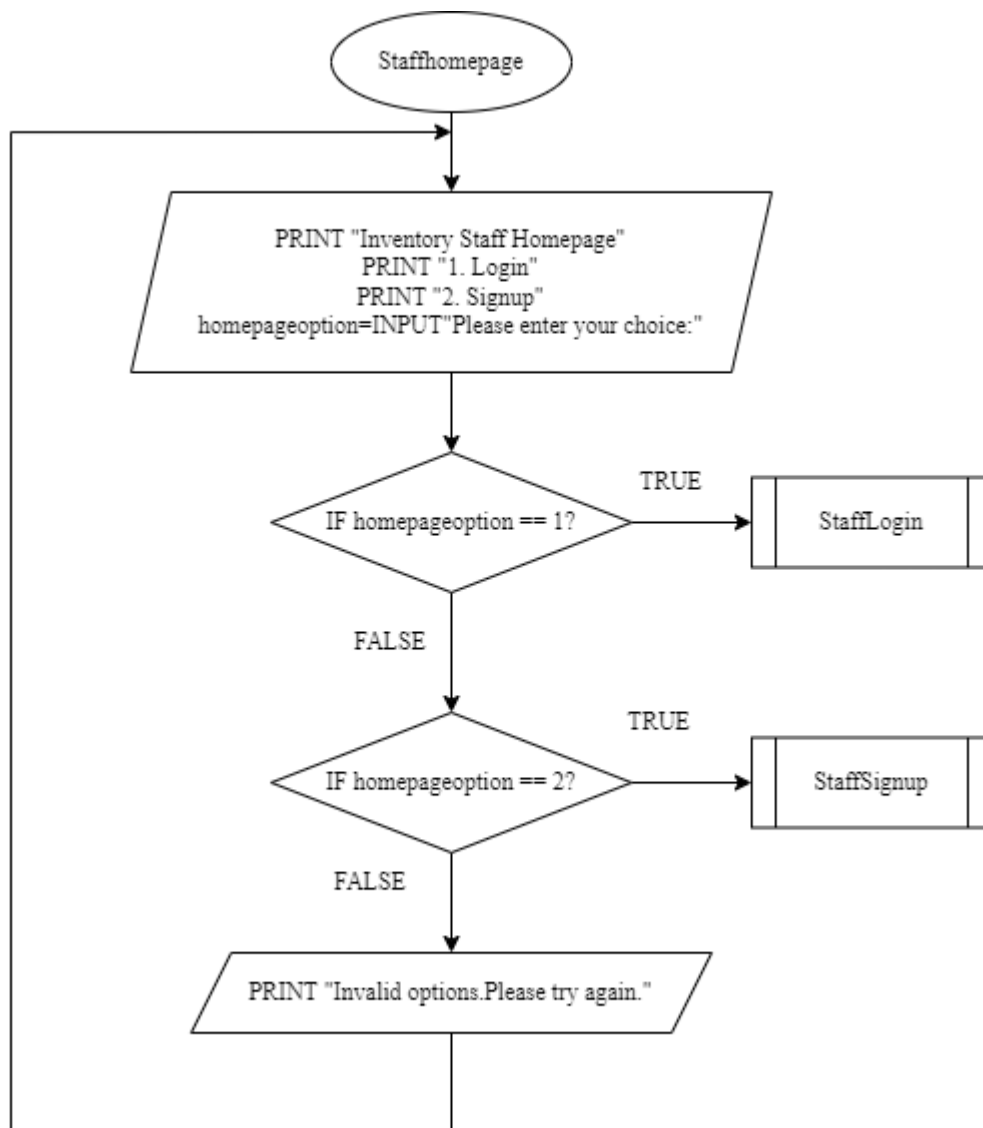
Staff Signup



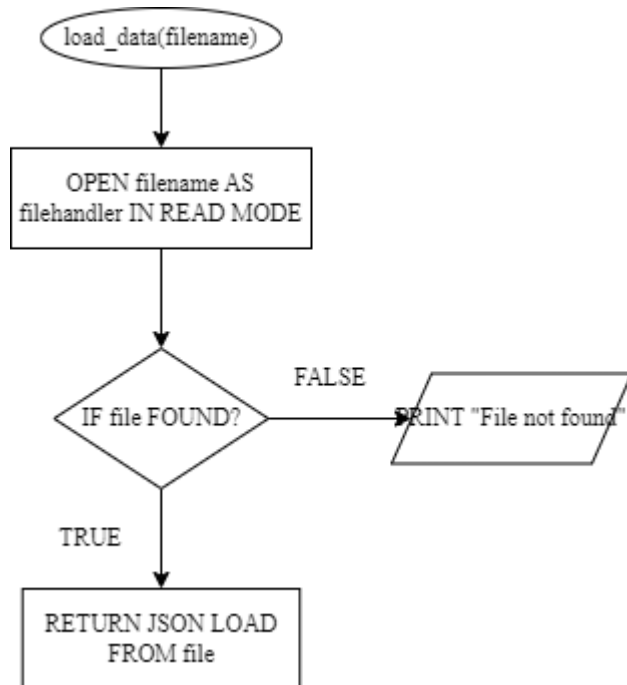
Save Pending Staff



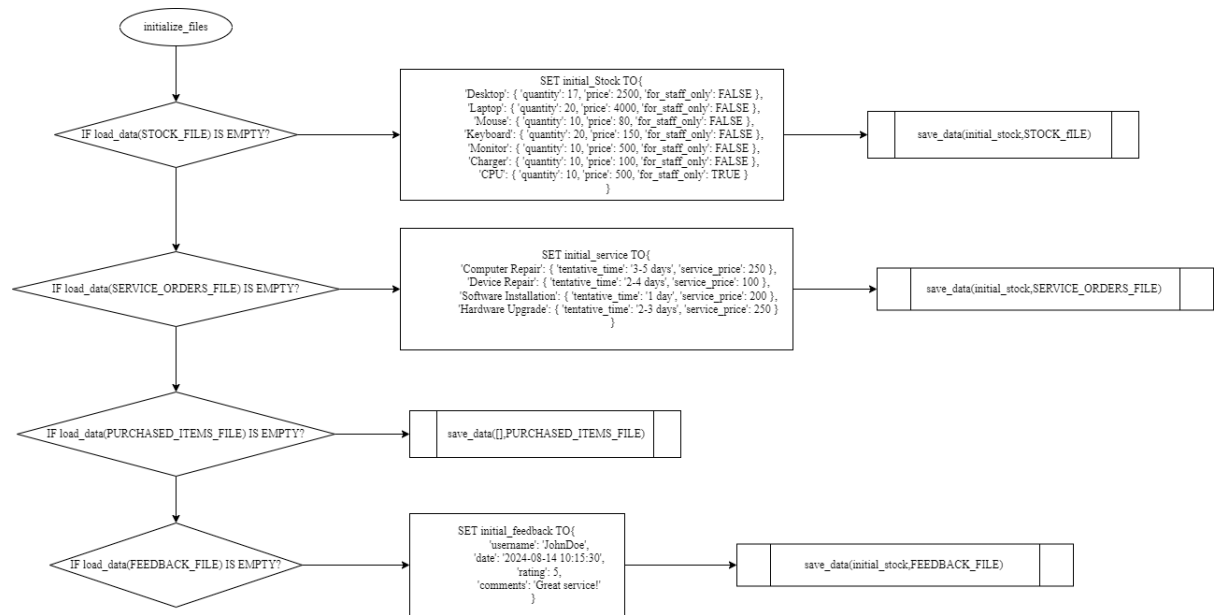
Staff homepage



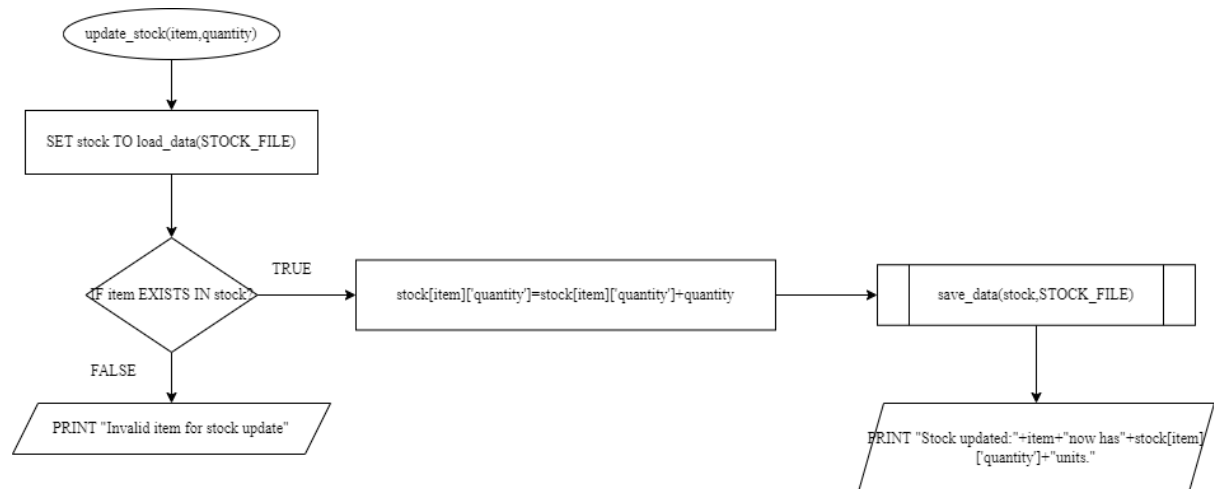
Load_data(filename)



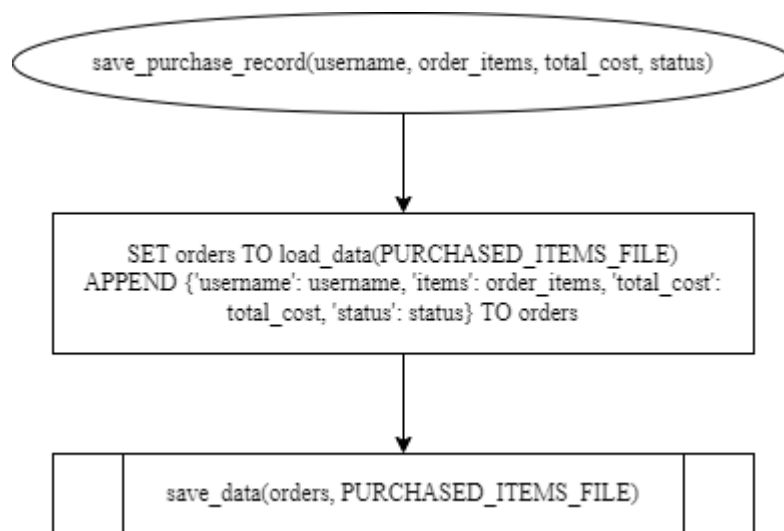
Initialize_files



Update stock



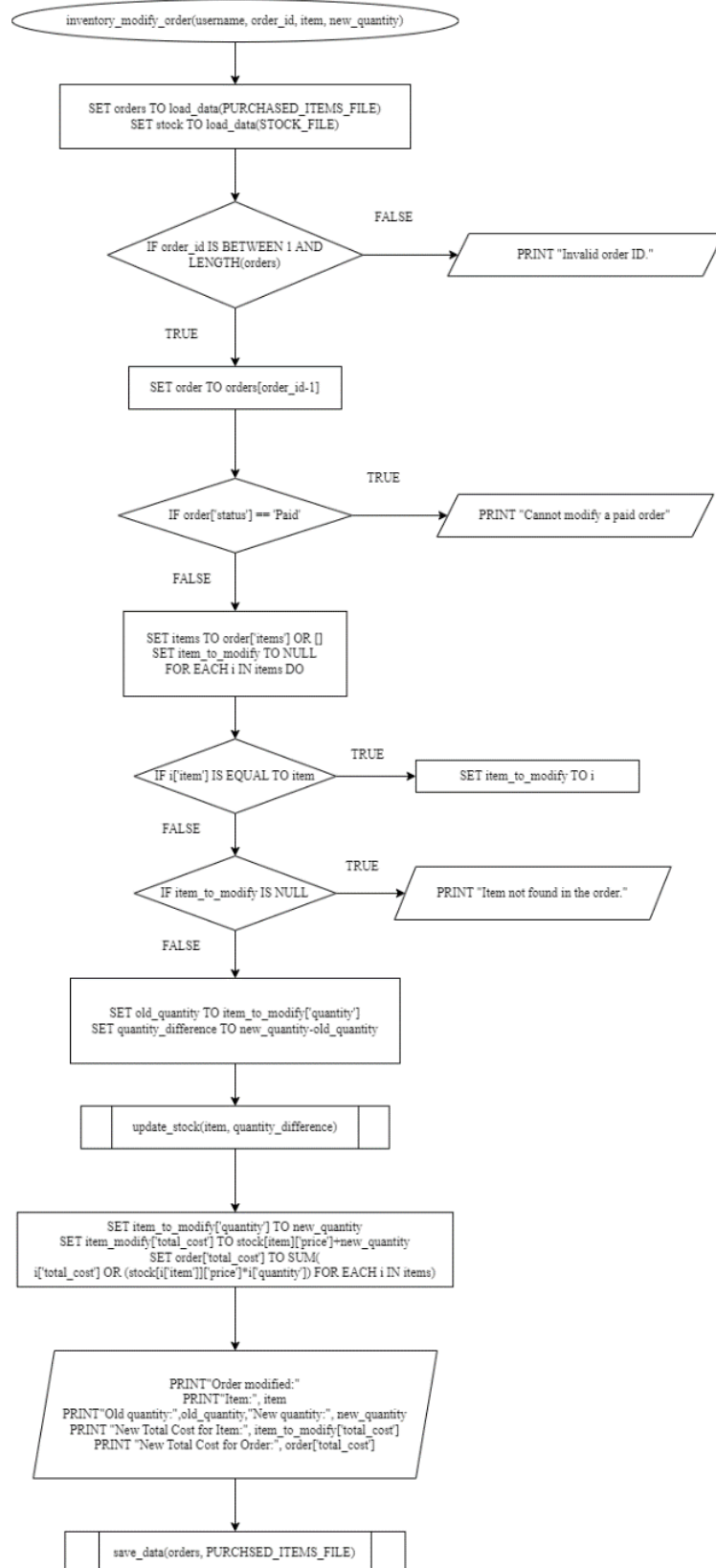
Save purchase record



Get stock



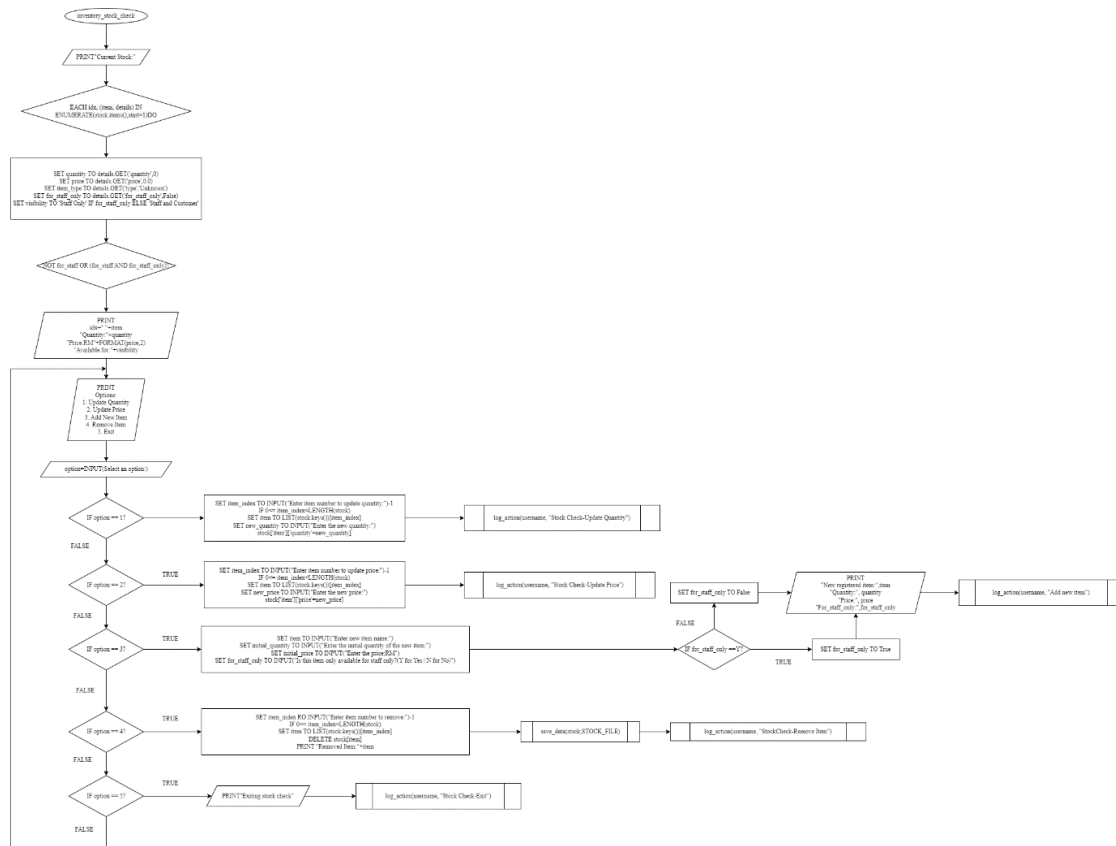
Inventory Modify Order



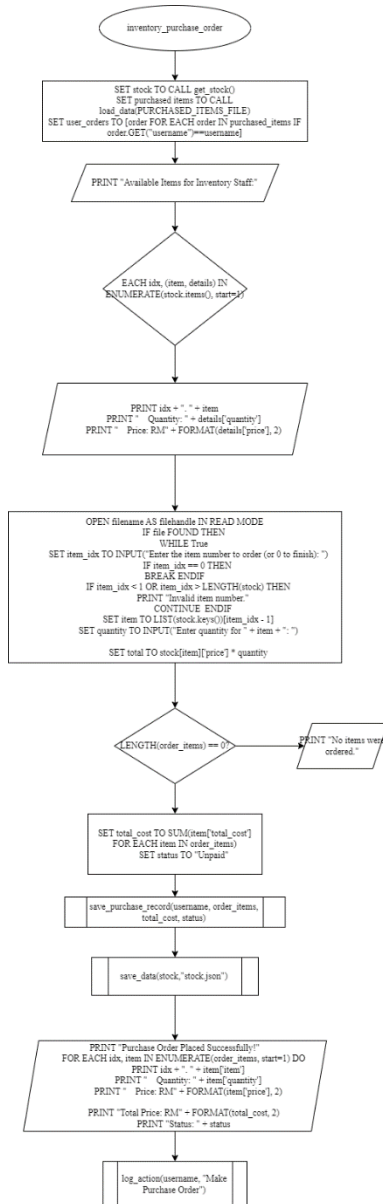
Log action



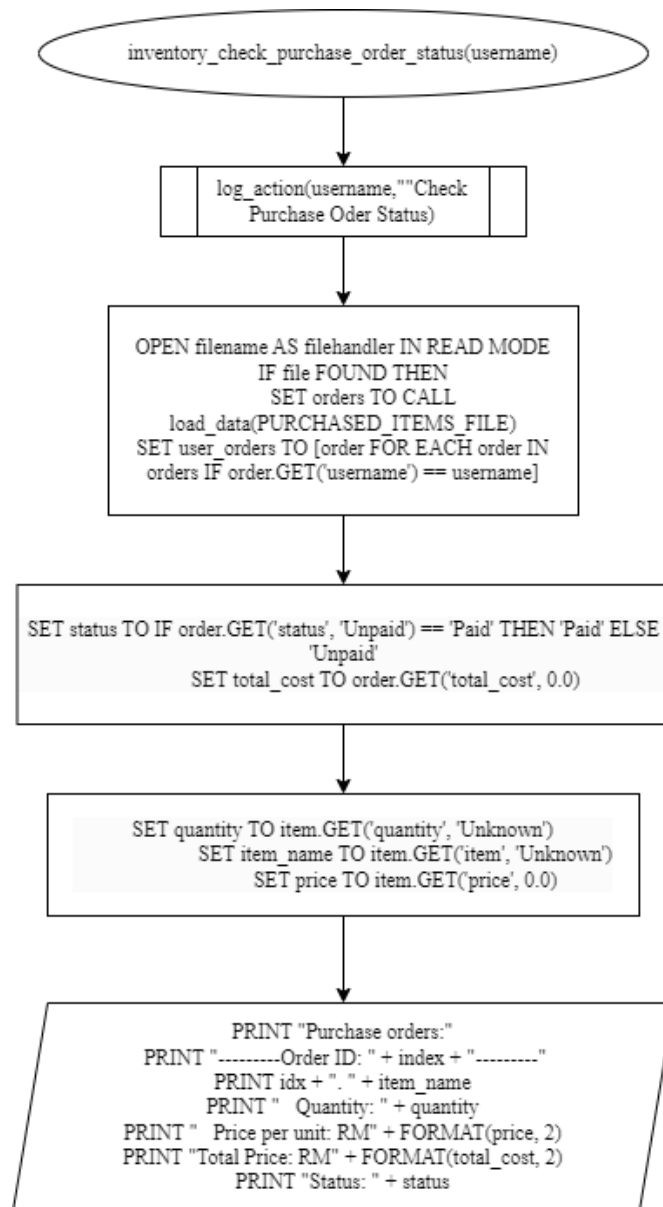
Inventory Stock Check



Inventory Purchase Order



Inventory Check Purchase Order Status



Source Code with Explanation

Import Files

```
import json
from datetime import datetime

# File paths used in Inventory
STOCK_FILE = 'stock.json'
PURCHASED_ITEMS_FILE = 'purchased_items.json'
REPORTS_FILE = 'reports.json'
SERVICE_ORDERS_FILE = 'service_orders.json'
FEEDBACK_FILE = 'feedback.json'
ORDERS_FILE = 'orders.txt'
CUST_SERVICE_ORDERS_FILE = 'service_orders.txt'
```

Figure 1: Import json files and datetime from Python

Source Code	Explanation
<code>'import json'</code>	Import dictionary, which is json helps to write and read the json files.
<code>'from datetime import datetime'</code>	Import datetime from Python to record dates and time for reports and
<code>'STOCK_FILE = 'stock.json''</code>	Create a json file named stock to store the details of stocks including item name, price, quantity and identity the visibility of users which some of the items will only show and able to buy by staff.
<code>PURCHASED_ITEMS_FILE = 'purchased_items.json'</code>	create a json file named purchased_items and store the details of purchased items by staff including usernames, items, quantity, price, total cost and status.
<code>REPORTS_FILE = 'reports.json'</code>	create a json file named reports to store the details when users are accessing the system. The reports will store username, date, time, and actions.
<code>'SERVICE_ORDERS_FILE = 'service_orders.json'</code>	create a json file named service_orders to store the details of service or repairs provided by Inventory Staff. The details include type of services and repairs, tentative time and service price.
<code>'FEEDBACK_FILE = 'feedback.json'</code>	Create a json file named feedback to store the feedback given by the customers. The details include username, rating, comments, date and time.

ORDERS_FILE=orders.txt	Create a text file to store the details of orders
CUST_SERVICE_ORDERS_FILE=service_orders.txt	Create a text file to store the details of service orders placed by the customers.

Log Functions

```
#Function used to record every actions users did
def log_action(username, action):
    #Log actions performed by users.
    reports = load_data(REPORTS_FILE)
    current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')

    if username not in reports:
        reports[username] = []

    reports[username].append({
        'time': current_time,
        'action': action
    })

    save_data(reports, REPORTS_FILE)
```

Figure 2: Function for Record Actions Done by Users

SourceCode	Explanation
def log_action(username, action):	Define a function named log_action that take two parameters which are username and action.
reports = load_data(REPORTS_FILE)	reports will hold the data that read data from REPORTS_FILE
current_time = datetime.now().strftime('%Y-%m-%d %H:%M:%S')	Obtain the current date and time as a formatted string (Year-Month-Day Hour-Minute-Second)
if username not in reports:	Check the presence of username in reports which hold the data that read from REPORTS_FILE
reports[username]=[]	Add the details in the reports and saved it according to username
reports[username].append({ 'time': current_time, 'action': action })	Add the details which are time and action according to the username
save_data(reports, REPORTS_FILE)	Get the data from reports and save it into REPORTS_FILE

check_password Function

```
def check_password(password):
    if len(password) < 8:
        return False, "Password must be at least 8 characters."

    has_upper = any(char.isupper() for char in password)
    has_digit = any(char.isdigit() for char in password)
    special_characters = "!@#$%^&*_{+()}.?;:\\"<>[]{}|"
    has_special = any(char in special_characters for char in password)

    if not has_upper:
        return False, "Password must contain at least 1 uppercase letter."
    if not has_digit:
        return False, "Password must contain at least 1 number."
    if not has_special:
        return False, "Password must contain at least 1 special character."
    return True, "Password is valid"
```

Figure : Function for Checking the Password of the Users

check_username Function

```
def check_username(username, filename):
    try:
        with open(filename, "r") as file:
            existing_username = [line.strip().split(",")[0] for line in file.readlines()]
            if username in existing_username:
                return True
    except FileNotFoundError:
        return False
    return False
```

Figure : Function for Checking the Username of the Users

Source Code	Explanation
def check_username(username, filename):	Define a function named check_username that take two parameters which are username and filename
try:	Attempts to open and read a file.
with open(filename, "r") as file: existing_username = [line.strip().split(",")[0] for line in file.readlines()]	Read a file and extract the first column of data from each line into a list
if username in existing_username: return True	Use IF statement to check if username exists in existing username
except FileNotFoundError: return False return False	If the file does not exist and cannot be opened, a FileNotFoundError is raised. Returns False to indicate that the operation could not be completed due to the missing file.

homepage Function

```
def homepage():
    while True:
        print("\n*****")
        print(" Welcome to KL Central Computer Company (KLCCC) website! ")
        print("*****")
        print("Please select a role to access to the system")
        print("1. Customer")
        print("2. Admin")
        print("3. Super User")
        print("4. Inventory")
        try:
            role = int(input("Please enter your role: "))
            if role == 1:
                role="customer"
                CustomerMenu()
            elif role == 2:
                role="admin"
                AdminMenu()
            elif role == 3:
                role="superuser"
                SuperUser_login()
            elif role == 4:
                role="staff"
                Staffhomepage()
            else:
                print("Invalid input. Please try again.")
        except ValueError:
            print("Invalid input. Please try again.")
```

Figure: Function for Users to choose role

Source Code	Explanation
def homepage():	Define a function named homepage
while True:	Use a loop to continuously an action
<pre>print("\n*****") print(" Welcome to KL Central Computer Company (KLCCC) website! ") print("*****") print("Please select a role to access to the system") print("1. Customer") print("2. Admin") print("3. Super User") print("4. Inventory")</pre>	Differentiate the KL Central Computer Company website with other section. Show a list of role to provide a better view for the user

	to choose their role
try:	Attempts to open and read a file.
role = int(input("Please enter your role: "))	Prompts user to get role.
<pre> if role == 1: role="customer" CustomerMenu() elif role == 2: role="admin" AdminMenu() elif role == 3: role="superuser" SuperUser_login() elif role == 4: role="staff" Staffhomepage() else: print("Invalid input. Please try again.") </pre>	Use IF statement to redirect users to different Menu according to their chosen role. Hence, an error message will be displayed to the user when they entered an invalid role number.
<pre> except ValueError: print("Invalid input. Please try again.") </pre>	An error message will be displayed to the user when they entered an invalid input.

generate_service_id Function

```
def generate_service_id():
    #Y = Year, full version
    #m = Month as a number 01-12
    #d = Day of month 01-31
    #H = Hour 00-23
    #M = Minute 00-59
    #S = Second 00-59
    return "SR" + datetime.now().strftime("%Y%m%d%H%M%S")
```

Figure: Function for Auto-generated service_id

generate_order_id Function

```
def generate_order_id():
    #Y = Year, full version
    #m = Month as a number 01-12
    #d = Day of month 01-31
    #H = Hour 00-23
    #M = Minute 00-59
    #S = Second 00-59
    return "OR" + datetime.now().strftime("%Y%m%d%H%M%S")
```

Figure: Function for Auto-generated order_id

Source Code	Explanation
def generate_service_id(): def generate_order_id():	Define function named generate_service_id and generate_order_id
return "SR" + datetime.now().strftime("%Y%m%d%H%M%S") return "OR" + datetime.now().strftime("%Y%m%d%H%M%S")	Used to generate service_id and order_id by using datetime.

CustomerSignIn Function

```
def CustomerSignIn():
    print("*****")
    print("Customer Log In Page")
    print("*****\n")
    max_attempts = 3
    attempts = 0

    while attempts < max_attempts:
        username = input("Enter Username: ")
        if is_user_pending(username, "pendingCust.txt"):
            print("Your account is still pending approval. Please wait for confirmation.")
            CustomerSignIn()
        password = input("Enter Password: ")
        if verify_customer_credentials(username, password, "customers.txt"):
            print(f"Login successful! Welcome, {username}")
            customer_interface(username)
        else:
            attempts += 1
            print(f"Incorrect username or password. You have {max_attempts - attempts} attempt(s) left.")

    print("Too many failed attempts. Please reset your password.")
    valid, message = reset_customer_password(username, "customers.txt")
    print(message)
```

Figure : Function for Customer to Login

StaffLogIn Function

```
def StaffLogIn():
    max_attempts=3
    attempts=0
    print("\n*****")
    print(" Inventory Staff Log In Page ")
    print("*****")
    while True:
        username = input("\nEnter Username: ")
        password = input("Enter Password: ")
        ic = input("Enter Identity Card:")
        active='active'
        if verify_Staffcredentials(username, password, ic, active, "staff.txt"):
            print(f"Login successful! Welcome, {username}")
            staff_menu(username)
        else:
            attempts+=1
            print(f"Wrong username or password. You have {max_attempts-attempts} attempts left.")
            if attempts==3:
                print("Too many failed attempts. Please reset your password.")
                valid, message = reset_staff_password(username, "staff.txt")
                print(message)
```

Figure : Function for Staff to Login

Source Code	Explanation
def StaffLogIn():	Define a function named StaffLogin
max_attempts=3 attempts=0	Set max_attempts = 3 and attempts = 0 to define and initialize variables that control how many attempts can be down by the user within a loop
print("\n*****") print(" Inventory Staff Log In Page ") print("*****")	Act as a header of the section to differentiate “Inventory Staff Log In Page” section with other sections.
while True: username = input("\nEnter Username: ") password = input("Enter Password: ")	Use a loop to continuously ask the staff to enter his username and password until certain condition is met(Entering correct username and password).

<pre> if verify_Staffcredentials(username, password, ic, active,"staff.txt"): print(f'Login successful! Welcome, {username}') staff_menu(username) </pre>	Use IF statement to check whether the entered username, password, ic and active is match with the credentials in staff.txt. As the credentials is met, a message “Login successful! Welcome{username}” will be shown. The f before the string is used to directly embed variables within the string using curly braces. Hence, {username} will be replaced with the actual username entered by the user. After that, a funtion named staff_menu(username) will be called, and the user will be led to staff_menu.
<pre> else: attempts+=1 print(f'Wrong username or password. You have {max_attempts- attempts} attempts left.") </pre>	When the entered username and password aren’t match with the credentials in staff.txt, attempts will be count by 1 to track the numbers of attempts by the user. User will get feedback to inform them that the entered username or password was incorrect and how many attempts they have left before they are locked out.
<pre> if attempts==3: print("Too many failed attempts. Please reset your password.") valid, message = reset_password(username, "staff.txt") print(message) </pre>	When the number of attempts reach three, users will get feedback that asked them to reset their password. A function named reset_password will be called and lead the users to the reset their password.

is_user_pending Function

```
def is_user_pending(username, filename):
    try:
        with open(filename, "r") as file:
            for line in file:
                pending_username = line.strip().split(",")[0]
                if pending_username == username:
                    return True
    except FileNotFoundError:
        return False
    return False
```

Figure: Function that Verify the Credentials of Customer

verify_customer_credentials Function

```
def verify_customer_credentials(username, password, filename):
    try:
        with open(filename, "r") as file:
            for line in file.readlines():
                stored_username, stored_password = line.strip().split(",")

                if username == stored_username and password == stored_password:
                    return True
    except FileNotFoundError:
        return False
    return False
```

Figure: Function that Verify the Credentials of Customer

verify_Staffcredentials Function

```
def verify_Staffcredentials(username, password, filename):
    try:
        with open(filename, "r") as file:
            for line in file.readlines():
                stored_username, stored_password = line.strip().split(",")

                if username == stored_username and password == stored_password:
                    return True
    except FileNotFoundError:
        return False
    return False
```

Figure: Function that Verify the Credentials of Staff

Source Code	Explanation
def verify_Staffcredentials(username, password, filename):	Define a function named verify_Staffcredentials that take three parameters which are username, password and filename.
try:	Attempts to open and read a file.
with open(filename, "r") as file: for line in file.readlines(): stored_username, stored_password = line.strip().split(",")	Using with statement to open the file specified by filename in read mode, read all the lines from the file and return them as a list of strings. for line in file.readlines(): iterates over each line in the list returned by file.readlines(). line.strip().split(",") ensure that any extra spaces or newline characters will not affect further processing and splits and stripped line into a list of substrings using a comma to separate them. Hence, the two variables which are stored_username and stored_password will be stored in a line and separated by a comma.
if username == stored_username and password == stored_password: return True	Compare the username and password provided by the user is same with stored_username and stored_password that read from the file. Use the logical AND to ensure both username and password must be matched. When both username and password match with the credentials from the file, the function will return True.
except FileNotFoundError: return False	If the file does not exist and cannot be opened, a FileNotFoundError is raised. Returns False to indicate that the operation could not be completed due to the missing file.
return False	return False when no matching credentials are found in the file

CustomerSignUp Function

```
def CustomerSignUp():
    print("*****")
    print("                                Create an account")
    print("*****\n")
    while True:
        username = input("Create username (At least 4 characters and must contain only alphabetic character): ")
        if check_username(username, "customers.txt"):
            print("Username already exists. Please try another one.")
        elif len(username) < 4:
            print("Invalid username. Please try another one.")
        elif not username.isalpha():
            print("Invalid username. Username must contain only alphabetic character.")
        else:
            break

    while True:
        password = input("Create password (At least 8 characters, including uppercase letter, number, and special character): ")
        valid, message = check_password(password)
        if not valid:
            print(message)
            continue

        verify_pw = input("Re-enter password: ")
        if verify_pw == password:
            print("Password match")
            print("Your registration is pending to be approved.")

            role = "customer"
            status = "pending"

            save_pendingCust(username, password, role, status, "pendingCust.txt")
            break
        else:
            print("Password do not match. Please try again.")
```

Figure: Function for Customers to Signup

StaffSignUp Function-Part1

```
def StaffSignUp():
    role="staff"
    status="pending"
    print("\n*****")
    print("    Inventory Staff Registration Page    ")
    print("*****")
    while True:
        username = input("\nCreate username: ")
        if check_username(username, "staff.txt"):
            print("Username already exists. Please try another one.")
        else:
            break
```

Figure : Function for the Staff to Signup

Source Code	Explanation
def StaffSignUp():	Define a function named StaffSignUp
role="staff" status="pending"	Set the variable role to staff to specify user role in this function. Set the variable status to pending to indicate that a process or request is in awating action.
print("\n***** *****") print(" Inventory Staff Registration Page ")	Act as a header of the section to differentiate “Inventory Staff Registration Page” section with other sections.

<pre>print("***** *****")</pre>	
<pre>while True: username = input("\nCreate username: ")</pre>	Use while statement to create a infinite loop and the loop will be keep running until a condition is met. Hence, prompts the user to enter a username.
<pre>if check_username(username, "staff.txt"): print("Username already exists. Please try another one.")</pre>	Use IF statement to check if the username already exists in the file which is staff.txt. When the condition met, user will be inform that the username is already taken.
<pre>else: break</pre>	Exits the loop if the username is unique and valid, allowing the process to continue

StaffSignUp Function-Part2

```
while True:
    password = input("Create password (At least 8 characters, including uppercase letter, number, and special character): ")
    valid, message = check_password(password)
    if not valid:
        print(message)
        continue
```

Figure : Function for the Staff to Signup

Source Code	Explanation
<pre>while True: password = input("Create password (At least 8 characters, including uppercase letter, number, and special character): ")</pre>	Use while statement to create a infinite loop until a valid password is entered. password is a variable that will prompts the user to enter a password. Hence, the user will be notify with the requirements of password when entering the password.
<pre>valid, message = check_password(password)</pre>	Indicate whether the password is valid and provides information about the password's validity when the function check_password(password) validates the password.
<pre>if not valid: print(message) continue</pre>	Use IF statement to check if the validation is False and print feedback on why the validation failed. User continue to restarts the loop and allow the user to re-enter their input.

StaffSignUp Function-Part3

```
verify_pw = input("Re-enter password: ")
if verify_pw == password:
    print("-Password match-")
    break

else:
    print("Password do not match. Please try again.")
    continue
```

Figure : Function for the Staff to Signup

Source Code	Explanation
verify_pw = input("Re-enter password: ")	Verify_pw is used to accept input from the user by asking them to re-enter the password.
if verify_pw == password: print("-Password match-") break	Use IF statement to ensure that verify_pw is matched with password. IF the password is match, user will be informed by a password match message. Break is used to exit the loop if the input is valid
else: print("Password do not match. Please try again.") continue	When the passwords doesn't match, the else block is executed and an error message will be show to the staff. Hence, the loop will be restart with continue

StaffSignUp Function-Part4

```
while True:
    ic=int(input("Identity Card:"))
    if check_ic(ic,"staff_ic.txt"):
        print("Invalid Identity Card. Please try again.")
    else:
        print("Your registration is pending to be approved.")
        print("You will be returned to Homepage.")
        save_pendingStaff(username, password, ic, role, status, "users.txt")
        homepage()
        break
```

Figure : Function for the Staff to Signup

Source Code	Explanation
while True:	Use while statement to create a infinite loop and the loop will be keep running until a condition is met.
ic=int(input("Identity Card:"))	Prompt users to get input which is ic and limit the input to only accept integer
if check_ic(ic,"staff_ic.txt"): print("Invalid Identity Card. Please try again.")	Validate an Identity Card number against a list of existing ICs stored in a file
else: print("Your registration is pending to be approved.") print("You will be returned to Homepage.") save_pendingStaff(username, password, ic, role, status, "users.txt") homepage() break	Inform the user of pending status, save registration details and redirect the user to homepage

save_pendingCust Function

```
def save_pendingCust(username, password, role, status, filename):
    with open(filename, "a") as file:
        file.write(f"{username},{password},{role},{status}\n")
```

Figure : Function for Saving the Details of the New Signup Staff

save_pendingStaff Function

```
def save_pendingStaff(username, password, ic, role, status, filename):
    with open('users.txt', "a") as file:
        file.writelines(f"{username},{password},{ic},{role},{status}\n")
```

Figure : Function for Saving the Details of the New Signup Staff

Source Code	Explanation
def save_pendingStaff(username, password, ic, role, status, filename):	Define a function named save_pendingStaff that take six parameters which are username, password, ic, role, status, filename
with open('users.txt', "a") as file: file.writelines(f"{username},{password},{ic},{role},{status}\n")	Add data which are username, password, ic, role and status in users.txt

check_ic Function

```
def check_ic(ic, filename):
    try:
        with open(filename, "r") as file:
            existing_username = [line.strip().split(",")[0] for line in file.readlines()]
            if ic in existing_ic:
                return True
    except FileNotFoundError:
        return False
    return False
```

Figure : Function for Checking the Identity Card of the Staff

def check_ic(ic, filename):	Define a function named check_ic that take two parameters which are ic and filename.
try:	Attempts to open and read a file.
with open(filename, "r") as file: existing_username = [line.strip().split(",")[0] for line in file.readlines()]	Read data from a file
if ic in existing_ic: return True	Return value which True if the ic is in the existing_ic list
except FileNotFoundError: return False return False	If the file does not exist and cannot be opened, a FileNotFoundError is raised. Returns False to indicate that the operation could not be completed due to the missing file.

CustomerMenu Function

```
def CustomerMenu():
    while True:
        print("*****\n*****")
        print("SIGN IN/SIGN UP")
        print("*****\n*****")
        print("Dear customer, do you have an account?")
        print("1. Yes, I would like to SIGN IN.")
        print("2. No, I would like to REGISTER for an account.")
        print("3. Exit")
        try:
            choice = int(input("Please enter your choice: "))
            if choice == 1:
                CustomerSignIn()
            elif choice == 2:
                CustomerSignUp()
            elif choice == 3:
                homepage()
            else:
                print("Invalid input. Please try again.")
                CustomerMenu()
        except ValueError:
            print("Invalid input. Please try again.")
```

Figure: Function for the Customer to Login or Signup

Staffhomepage Function

```
def Staffhomepage():
    print("\n*****\n*****")
    print("Inventory Staff Homepage")
    print("*****\n*****")
    print("1. Login")
    print("2. Signup")
    homepageoption=input("Please enter your option:")
    if homepageoption == "1":
        StaffLogIn()
    elif homepageoption == "2":
        StaffSignUp()
    else:
        print("Invalid options. Please try again")
        Staffhomepage()
```

Figure: Function for the Staff to Login or Signup

Source Code	Explanation
def CustomerMenu():	Define a function named CustomerMenu
def Staffhomepage():	Define a function named Staffhomepage
print("*****\n*****\n*****")	Differentiate SignIn and SignUp page for Customer with other section.
print("SIGN IN/SIGN UP")	Differentiate Inventory Staff Homepage with other section.

<pre> print("***** ***** *****\n") print("\n***** **") print(" Inventory Staff Homepage ") print("***** ") </pre>	
<pre> print("1. Login") print("2. Signup") </pre>	Print two option for user to choose which are Login and Signup
<pre> homepageoption=input("Please enter your option:") </pre>	Homepageoption is a variable used to store the option from the user.
<pre> if homepageoption == "1": StaffLogIn() </pre>	Check if user's input is equal to 1, redirect user to StaffLogin page
<pre> elif homepageoption == "2": StaffSignUp() </pre>	Check if user's input is equal to 2, redirect user to StaffSignUp page
<pre> else: print("Invalid options. Please try again") Staffhomepage() </pre>	Display an error message if user have entered an invalid options

load_data and save_data Function

```

def load_data(filename):
    #Load JSON data from a file
    try:
        with open(filename, 'r') as file:
            return json.load(file)
    except (FileNotFoundError, json.JSONDecodeError):
        return {}

def save_data(data, filename):
    #Save JSON data to a file
    with open(filename, 'w') as file:
        json.dump(data, file, indent=4)

```

Figure: Function for Read Data from a File and Save Data in a File

Source Code	Explanation
<pre> def load_data(filename): #Load JSON data from a file </pre>	Define a function named load_data that take one parameter which are filename
<pre> try: </pre>	Attempts to open and read a file.
<pre> with open(filename, 'r') as file: return json.load(file) except (FileNotFoundError, json.JSONDecodeError): </pre>	Read data from a json file

return {}	
def save_data(data, filename):	Define a function named save_data that take two parameter which are data and filename
with open(filename, 'w') as file: json.dump(data, file, indent=4)	Write data in a json file with an indent of 4

get_item and get_item_price Function

```
def get_item(item):  
    if item == 1:  
        return "Desktop"  
    elif item == 2:  
        return "Laptop"  
    elif item == 3:  
        return "Mouse"  
    elif item == 4:  
        return "Keyboard"  
    elif item == 5:  
        return "Monitor"  
    elif item == 6:  
        return "Charger"  
    else:  
        return None  
  
def get_item_price(item_name):  
    if item_name == "Desktop":  
        return 2500  
    elif item_name == "Laptop":  
        return 4000  
    elif item_name == "Mouse":  
        return 80  
    elif item_name == "Keyboard":  
        return 150  
    elif item_name == "Monitor":  
        return 500  
    elif item_name == "Charger":  
        return 100  
    else:  
        return 0
```

Figure: Function for Initialize Data (item and item_price)

get_service and get_service_price Function

```
def get_service(service_option):
    if service_option == 1:
        return "Device Repair"
    elif service_option == 2:
        return "Software Installation"
    elif service_option == 3:
        return "Hardware Upgrade"
    elif service_option == 4:
        return "Computer Repair"
    else:
        return None

def get_service_price(service_name, services):
    return services[service_name]['service_price']
```

Figure: Function for Initialize Data (service_option)

initialize_files Function-Part1

```
def initialize_files():
    #Initialize stock and orders files with default values
    if not load_data(STOCK_FILE):
        initial_stock={
            'Desktop':{'quantity':17,'price':2500,"for_staff_only": False},
            'Laptop':{'quantity':20,'price':4000,"for_staff_only": False},
            'Mouse':{'quantity':10,'price':80,"for_staff_only": False},
            'Keyboard':{'quantity':20,'price':150,"for_staff_only": False},
            'Monitor':{'quantity':10,'price':500,"for_staff_only": False},
            'Charger':{'quantity':10,'price':100,"for_staff_only": False},
            'CPU':{'quantity':10,'price':500,"for_staff_only": True},
        }
        save_data(initial_stock, STOCK_FILE)

    if not load_data(SERVICE_ORDERS_FILE):
        initial_service={
            'Computer Repair':{'tentative_time':'3-5 days','service_price':250},
            'Device Repair':{'tentative_time':'2-4 days','service_price':100},
            'Software Installation':{'tentative_time':'1 days','service_price':200},
            'Hardware Upgrade':{'tentative_time':'2-3 days','service_price':250}
        }
        save_data(initial_service, SERVICE_ORDERS_FILE)
```

Figure: Function for Initialize Data in the Files

Source Code	Explanation
def initialize_files():	Define a function named initialize_files
if not load_data(STOCK_FILE):	Checks if the STOCK_FILE exists and is not empty
initial_stock={ 'Desktop':{'quantity':17,'price':2500,"for_staff_only": False}, 'Laptop':{'quantity':20,'price':4000,"for_staff_only": False},	Save the initial stock in a file

'Mouse': {'quantity':10,'price':80,"for_staff_only": False}, 'Keyboard': {'quantity':20,'price':150,"for_staff_only": False}, 'Monitor': {'quantity':10,'price':500,"for_staff_only": False}, 'Charger': {'quantity':10,'price':100,"for_staff_only": False}, 'CPU': {'quantity':10,'price':500,"for_staff_only": True}, }	
save_data(initial_stock, STOCK_FILE)	Save the initial_stock into the STOCK_FILE
if not load_data(SERVICE_ORDERS_FILE):	Checks if the SERVICE_ORDERS_FILE exists and is not empty
initial_service={ 'Computer Repair': {'tentative_time':'3-5 days','service_price':250}, 'Device Repair': {'tentative_time':'2-4 days','service_price':100}, 'Software Installation': {'tentative_time':'1 days','service_price':200}, 'Hardware Upgrade': {'tentative_time':'2-3 days','service_price':250} }	Save some initial service in the file
save_data(initial_service, SERVICE_ORDERS_FILE)	Save the initial_service into the SERVICE_ORDERS_FILE

initialize_files Function-Part2

```
if not load_data(PURCHASED_ITEMS_FILE):
    save_data([], PURCHASED_ITEMS_FILE)

if not load_data(FEEDBACK_FILE):
    initial_feedback={
        'username': 'JohnDoe',
        'date': '2024-08-14 10:15:30',
        'rating': 5,
        'comments': 'Great service!'
    }
    save_data(initial_feedback, FEEDBACK_FILE)
```

Figure: Function for Initialize Data in the Files

Source Code	Explanation
if not load_data(PURCHASED_ITEMS_FILE): save_data([], PURCHASED_ITEMS_FILE)	initializes the PURCHASE_ITEMS_FILE if it does not exist or is empty
if not load_data(FEEDBACK_FILE): initial_feedback={ 'username': 'JohnDoe', 'date': '2024-08-14 10:15:30', 'rating': 5, 'comments': 'Great service!' } save_data(initial_feedback, FEEDBACK_FILE)	Handles the initialization of feedback data if the feedback file is empty or doesn't exist

update_stock Function

```
def update_stock(item, quantity):
    #Update stock levels in the stock file.
    stock = load_data(STOCK_FILE)

    if item in stock:
        stock[item]['quantity'] += quantity
        save_data(stock, STOCK_FILE)
        print(f"\nStock updated: {item} now has {stock[item]['quantity']} units.")
    else:
        print("Invalid item for stock update.")
```

Figure: Function for Updating the Stock into the File

Source Code:	Explanation:
def update_stock(item, quantity):	Define a function named update_stock that take 2 parameters which are item and quantity.
stock = load_data(STOCK_FILE)	Load data from STOCK_FILE and store it into a variable named stock
if item in stock: stock[item]['quantity'] += quantity save_data(stock, STOCK_FILE) print(f"\nStock updated: {item} now has {stock[item]['quantity']} units.")	Update the quantity of an item in the stock and then save the updated stock data to a file
else: print("Invalid item for stock update.")	Display an error message when the user has entered an invalid item for stock update

save_orders Function

```
def save_orders(filename, orders):
    try:
        with open(filename, "w") as file:
            for order_id, order in orders.items():
                if filename == 'orders.txt':
                    items_str = ".join([
                        f"{item['name']},{item['quantity']},{item['price']:.2f}"
                        for item in order['items']
                    ])
                elif filename == 'service_orders.txt':
                    items_str = ".join([
                        f"{item['name']},{item['price']:.2f}"
                        for item in order['items']
                    ])
                else:
                    raise ValueError("Unsupported file type.")

                line = f"{order_id};{order['username']};{order['status']};{order['total_price']:.2f};{items_str}\n"
                print(f"Writing to file: {line}")
                file.write(line)
    except Exception as e:
        print(f"An error occurred while saving orders: {e}")
```

Figure: Function for Storing the orders into the File

save_purchase_order Function

```
def save_purchase_order(order):
    with open("orders.txt", "a") as file:
        items_str = '.join([f"{item['name']},{item['quantity']},{item['price']:.2f}" for item in order['items']])
        file.write(f"{order['order_id']};{order['username']};{order['status']};{order['total_price']:.2f};{items_str}\n")
```

Figure: Function for Storing the purchase_order into the File

save_service_order Function

```
def save_service_order(order):  
    with open("service_orders.txt", "a") as file:  
        services_str = ';'.join([f"{service['name']},{service['price']:.2f}" for service in order['services']])  
        file.write(f"{order['service_id']};{order['username']};{order['status']};{order['total_price']:.2f};{services_str}\n")
```

Figure: Function for Storing the service_order into the File

save_purchase_record Function

```
def save_purchase_record(username, order_items, total_cost, status):  
    # Save a record of the purchased items  
    orders = load_data(PURCHASED_ITEMS_FILE)  
    orders.append({'username': username, 'items': order_items, 'total_cost': total_cost, 'status': status})  
    save_data(orders, PURCHASED_ITEMS_FILE)
```

Figure: Function for Storing the Purchase_Record into the File

Source Code	Explanation
def save_purchase_record(username, order_items, total_cost, status):	Define a function named save_purchase_record that take four parameter which are username, order_items, total cost and status
orders = load_data(PURCHASED_ITEMS_FILE)	Load data from a file and store it into a variable named orders
orders.append({'username': username, 'items': order_items, 'total_cost': total_cost, 'status': status})	Add new order into the orders list
save_data(orders, PURCHASED_ITEMS_FILE)	Save all data in orders dictionary into PURCHASED ITEMS FILE

get_stock Function

```
def get_stock():  
    #Retrieve current stock levels  
    return load_data(STOCK_FILE)
```

Figure: Function for Retrieve the Stock into the File

Source Code	Explanation
-------------	-------------

def get_stock(): return load_data(STOCK_FILE)	Define a function named get_stock to retrieve current stock levels
--	--

modify_order Function-Part1

```
def modify_order(username):
    user_purchase_orders, user_service_orders = check_order_details(username)

    if not user_purchase_orders and not user_service_orders:
        print("You have no orders to modify.")
        return

    while True:
        print("*****")
        print("                        Modify Purchase/Service/Repair Order")
        print("*****\n")
        print("1. Modify Purchase Order")
        print("2. Modify Service/Repair Order")
        print("3. Exit")
        try:
            choice = int(input("Please enter what type of order that you want to modify: "))
            if choice == 1:
                order_file = 'orders.txt'
                orders = user_purchase_orders
                modify_type = "item"
            elif choice == 2:
                order_file = 'service_orders.txt'
                orders = user_service_orders
                modify_type = "service"
            elif choice == 3:
                print("Exiting modification menu.")
                return
            else:
                print("Invalid choice. Please try again.")
                continue
        except ValueError:
            print("Invalid choice. Please try again.")
            continue

        if not orders:
            print("You have no orders of this type.")
            continue
```

Figure: Function for Modify Purchase Order and Service Order of Customer

inventory_modify_order Function-Part1

```
def inventory_modify_order(username, order_id, item, new_quantity):
    #Modify the quantity of a specific item in an existing purchase order.
    orders = load_data(PURCHASED_ITEMS_FILE)
    stock = load_data(STOCK_FILE)

    if 1 <= order_id <= len(orders):
        order = orders[order_id - 1]

        if order.get('status') == 'Paid':
            print("Cannot modify a paid order.")
            return
```

Figure: Function for Modify Purchase Order of Inventory Staff

Source Code	Explanation
def inventory_modify_order(username, order_id, item, new_quantity):	Define a function named inventory_modify_order that take 4

	parameter which are username, order_id, item and new_quantity
orders = load_data(PURCHASED_ITEMS_FILE) stock = load_data(STOCK_FILE)	Load data from a file and store it into a variable named orders
if 1 <= order_id <= len(orders): order = orders[order_id - 1]	Ensure that the order_id provided by the user is within the valid range of order indices.
if order.get('status') == 'Paid': print("Cannot modify a paid order.") return	Determine if an order has already been marked as "Paid"

modify_order Function-Part2

```

print("*****")
print("
Order Summary")
print("*****\n")
for order_id, order in orders.items():
    print(f"Order ID: {order_id}, Status: {order['status']}, Total Price: RM{order['total_price']:.2f}")
    for item in order['items']:
        print(f" - {item['name']} (RM{item['price']:.2f})")

order_id = input("Enter the Order ID you want to modify: ")
if order_id not in orders:
    print("Invalid Order ID.")
    continue

if orders[order_id]['status'] == 'paid':
    print("Order cannot be modified as it has already been paid.")
    continue

while True:
    print("1. Add " + modify_type)
    print("2. Remove " + modify_type)
    print("3. Finish modification")
    try:
        option = int(input("Please enter your choice: "))
        if option == 1:
            if modify_type == "item":
                stock = get_stock()
                print("*****")
                print("
Available Items")
                print("*****\n")
                for idx, (item, details) in enumerate(stock.items(), start=1):
                    if not details.get('for_staff_only', False):
                        print(f"{idx}. {item}")
                        print(f"    Price: RM{details['price']:.2f}")
                while True:
                    try:
                        item_choice = int(input("Please enter the number of item you want to add (Enter '0' to finish): "))
                        if item_choice == 0:
                            break
                        item_name = get_item(item_choice)
                        if item_name is None:
                            print("Invalid choice, please try again")
                            continue

                        quantity = int(input(f"Please enter the quantity for {item_name}: "))
                        if quantity <= 0:
                            print("Quantity must be greater than zero.")
                            continue

                        item_stock = stock[item_name]

```

Figure: Function for Modify Purchase Order and Service Order of Customer

inventory_modify_order Function-Part2

```
items = order.get('items', [])

# Find the item in the order
item_to_modify = None
for i in items:
    if i['item'] == item:
        item_to_modify = i
        break

if not item_to_modify:
    print(f"Item '{item}' not found in the order.")
    return

old_quantity = item_to_modify['quantity']
quantity_difference = new_quantity - old_quantity

# Update stock based on the difference in quantity
update_stock(item, quantity_difference)
```

Figure: Function for Modify Purchase Order of Inventory Staff

Source Code	Explanation
items = order.get('items', [])	Safely retrieve the value associated with the key which is items from the order dictionary
item_to_modify = None	provides a clear starting value indicating that no item has been selected or found.
for i in items:	Create a loop to go through each item in the items list
if i['item'] == item: item_to_modify = i break	Use to find a specific item in a list of items
if not item_to_modify: print(f"Item '{item}' not found in the order.") return	Use to verify that an item was found and is valid before attempting further operations.
old_quantity = item_to_modify['quantity'] quantity_difference = new_quantity - old_quantity	Retrieve the item from the stock and get its current quantity. Calculate the difference between the new and old quantities. Hence, update the stock.
update_stock(item, quantity_difference)	Update the stock level of an item based on the quantity difference.

modify_order Function-Part3

```
        if item_stock['quantity'] >= quantity:
            orders[order_id]['items'].append({
                "name": item_name,
                "quantity": quantity,
                "price": item_stock['price'] * quantity
            })
            orders[order_id]['total_price'] += item_stock['price'] * quantity
            item_stock['quantity'] -= quantity
            save_orders(order_file, orders)
            print(f"Order {order_id} has been modified successfully.")
            log_action(username, "Modify Purchase Order - Add Item")
        else:
            print(f"Insufficient stock for {item_name}. Only {item_stock['quantity']} available.")
    except ValueError:
        print("Invalid input. Please enter a number.")
elif modify_type == "service":
    services = load_data(SERVICE_ORDERS_FILE)
    print("*****")
    print("                Available Service/Repair Option")
    print("*****\n")
    for idx, (service, details) in enumerate(services.items(), start=1):
        print(f"{idx}. {service} : {details['tentative_time']}")
        print(f"    Price: RM{details['service_price']:.2f}")

    while True:
        try:
            service_option = int(input("Please select the service you want to request (Enter '0' to finish): "))

            if service_option == 0:
                break

            service_name = get_service(service_option)
            if service_name is None:
                print("Invalid choice, please try again")
                continue

            if any(service['name'] == service_name for service in orders[order_id]['items']):
                print(f"{service_name} has already been added to your order.")
                continue
```

Figure: Function for Modify Purchase Order and Service Order of Customer

modify_order Function-Part4

```
        service_details = services.get(service_name)
        if service_details:
            orders[order_id]['items'].append({
                "name": service_name,
                "price": service_details["service_price"],
                "tentative_time": service_details["tentative_time"]
            })
            orders[order_id]['total_price'] += service_details["service_price"]
            print(f"Added {service_name} to your order.")
            save_orders(order_file, orders)
            print(f"Order {order_id} has been modified successfully.")
            log_action(username, "Modify Service/Repair Order - Add Service")
        except ValueError:
            print("Invalid input. Please enter a number")

    elif option == 2:
        if modify_type == "item":
            item_name = input(f"Enter the name of the {modify_type} to remove: ")
            item_to_remove = None
            for item in orders[order_id]['items']:
                if item['name'] == item_name:
                    item_to_remove = item
                    break
            if item_to_remove:
                orders[order_id]['items'].remove(item_to_remove)
                orders[order_id]['total_price'] -= item_to_remove['price']
                print(f"{item_name} removed from the order.")
                log_action(username, "Modify Purchase Order - Remove Item")
            else:
                print(f"{item_name} not found in the order.")
        elif modify_type == "service":
            service_name = input(f"Enter the name of the {modify_type} to remove: ")
            service_to_remove = None
            for service in orders[order_id]['items']:
                if service['name'] == service_name:
                    service_to_remove = service
                    break
            if service_to_remove:
                orders[order_id]['items'].remove(service_to_remove)
                orders[order_id]['total_price'] -= service_to_remove['price']
                print(f"{service_name} removed from the order.")
                save_orders(order_file, orders)
                print(f"Order {order_id} has been modified successfully.")
                log_action(username, "Modify Service/Repair Order - Remove Service")
            else:
                print(f"{service_name} not found in the order.")
    elif option == '3':
        customer_interface(username)
    else:
        print("Invalid choice. Please try again.")
except ValueError:
    print("Invalid input. Please enter a number.")
```

Figure: Function for Modify Purchase Order and Service Order of Customer

inventory_modify_order Function-Part5

```
# Update the item with the new quantity and recalculate the total cost
item_to_modify['quantity'] = new_quantity
item_to_modify['total_cost'] = stock[item]['price'] * new_quantity

# Recalculate the total cost for the entire order
order['total_cost'] = sum(
    i.get('total_cost', stock[i['item']]['price'] * i['quantity']) for i in items
)

print(f"Order {order_id} modified.")
print(f"Item: {item}")
print(f"Old Quantity: {old_quantity}, New Quantity: {new_quantity}")
print(f"New Total Cost for Item: RM{item_to_modify['total_cost']:.2f}")
print(f"New Total Cost for Order: RM{order['total_cost']:.2f}")
save_data(orders, PURCHASED_ITEMS_FILE)
else:
    print("Invalid order ID.")
log_action(username, "Stock Check - Modify Purchase Order")
```

Figure: Function for Modify Purchase Order of Inventory Staff

Source Code	Explanation
<code>item_to_modify['quantity'] = new_quantity</code>	To update the quantity of an item in dictionary
<code>item_to_modify['total_cost'] = stock[item]['price'] * new_quantity</code>	To calculate and update the total cost for an item based on its new quantity.
<code>order['total_cost'] = sum(i.get('total_cost', stock[i['item']]['price'] * i['quantity']) for i in items)</code>	To calculate the total cost of an order by summing up the total costs of individual items.
<code>print(f"Order {order_id} modified.")</code>	Display a confirmation message
<code>print(f"Item: {item}")</code>	Display the modified item
<code>print(f"Old Quantity: {old_quantity}, New Quantity: {new_quantity}")</code>	Display the old quantity and new quantity after modification
<code>print(f"New Total Cost for Item: RM{item_to_modify['total_cost']:.2f}")</code>	Display new total cost the item
<code>print(f"New Total Cost for Order: RM{order['total_cost']:.2f}")</code>	Display new total cost of the order
<code>save_data(orders, PURCHASED_ITEMS_FILE)</code>	Save the data in the orders dictionary into the file
<code>else: print("Invalid order ID.")</code>	Display an error message when the user has entered an invalid order_id
<code>log_action(username, "Stock Check - Modify Purchase Order")</code>	Record user's action and datetime and named the action as Stock Check – Modify Purchase Order

cancel_order Function

```
def cancel_order(username):
    log_action(username, "Cancel Order")
    user_purchase_orders, user_service_orders = check_order_details(username)

    orders_to_cancel = {}

    for order_id, order in user_purchase_orders.items():
        if not order['status'] == 'paid':
            orders_to_cancel[order_id] = order

    for order_id, order in user_service_orders.items():
        if not order['status'] == 'paid':
            orders_to_cancel[order_id] = order

    if not orders_to_cancel:
        print("You have no unpaid orders to cancel.")
        return

    print("Your unpaid orders:")
    for order_id, order in orders_to_cancel.items():
        print(f"Order ID: {order_id}, Total Price: RM{order['total_price']:.2f}")

    order_id = input("Enter the Order ID you want to cancel: ")
    if order_id in orders_to_cancel:
        if order_id in user_purchase_orders:
            del user_purchase_orders[order_id]
            save_orders('orders.txt', user_purchase_orders)
        elif order_id in user_service_orders:
            del user_service_orders[order_id]
            save_orders('service_orders.txt', user_service_orders)
        print(f"Order {order_id} has been canceled successfully.")
    else:
        print("Invalid Order ID.")
```

Figure: Function for Cancel Purchase Order and Service Order of Customer

inventory_cancel_order Function

```
def inventory_cancel_order(username, order_id):
    #Cancel an existing purchase order.
    try:
        orders = load_data(PURCHASED_ITEMS_FILE)

        if 1 <= order_id <= len(orders):
            order = orders[order_id - 1] # Adjust for 1-based index
            items = order.get('items', [])

            if order.get('status') == 'paid':
                print("Paid orders are non-refundable unless the product is unavailable.")
            else:
                # Update stock for each item in the order
                for item in items:
                    item_name = item['item']
                    quantity = item['quantity']
                    update_stock(item_name, quantity) # Increase stock by the ordered quantity

                # Remove the order from the list
                orders.pop(order_id - 1)

                # Save the updated orders list
                save_data(orders, PURCHASED_ITEMS_FILE)

                print(f"Order ID: {order_id} canceled.")
                log_action(username, "Cancel Purchase Order")
        else:
            print("Invalid order ID.")
    except Exception as e:
        print(f"An error occurred while canceling the order: {e}")
```

Figure: Function for Cancel Purchase Order of Inventory Staff

Source Code	Explanation
def inventory_cancel_order(username, order_id):	Define a function named inventory_cancel_order that take two parameter which are username and order_id
try:	Attempts to open and read a file.
orders = load_data(PURCHASED_ITEMS_FILE)	Load data from a file and store it in a variable named orders
if 1 <= order_id <= len(orders): order = orders[order_id - 1] # Adjust for 1-based index items = order.get('items', [])	To validate the order ID, retrieve the specified order, and access its items.
if order.get('status') == 'paid': print("Paid orders are non-refundable unless the product is unavailable.")	Provide a clear view with the conditions under which refunds can be requested.
else: # Update stock for each item in the order for item in items: item_name = item['item'] quantity = item['quantity'] update_stock(item_name, quantity)	To handle the post-processing of an order by updating stock quantities and removing the order from the list once processing is complete.
orders.pop(order_id - 1)	
save_data(orders, PURCHASED_ITEMS_FILE)	Save the data in the orders dictionary into the PURCHASED_ITEMS_FILE
print(f"Order ID: {order_id} canceled.")	Display a message that has successfully cancel an purchase order
log_action(username, "Cancel Purchase Order")	Record user's action with datetime and named the action as Cancel Purchase Order
else: print("Invalid order ID.")	Display an error message when user has entered an invalid order_id
except Exception as e: print(f"An error occurred while canceling the order: {e}")	To handle and report errors that occur during the cancellation of an order.

inventory_stock_check Function-Part1

```
def inventory_stock_check(username, for_staff=False):
    #Display and adjust current stock levels, optionally filtering by visibility.
    stock = get_stock()

    while True:
        print("\nCurrent stock:")
        for idx, (item, details) in enumerate(stock.items(), start=1):
            quantity = details.get('quantity', 0)
            price = details.get('price', 0.0)
            item_type = details.get('type', 'Unknown')
            for_staff_only = details.get('for_staff_only', False)
            visibility = 'Staff Only' if for_staff_only else 'Staff and Customer'

            # Filter items based on visibility
            if not for_staff or (for_staff and for_staff_only):
                print(f"{idx}. {item} ")
                print(f"    Quantity: {quantity}")
                print(f"    Price: RM{price:.2f}")
                print(f"    Available for: {visibility}")

        print("\nOptions:")
        print("1. Update Quantity")
        print("2. Update Price")
        print("3. Add New Item")
        print("4. Remove Item")
        print("5. Exit")

        option = input("Select an option (1-5): ")
```

Figure: Function for Stock Checking and Making Adjustment of Inventory Staff

Source Code	Explanation
def inventory_stock_check(username, for_staff=False):	Define a function named inventory_stock_check that take 2 parameter which are username and for_staff. Set a default value to `for_staff` which is False.
stock = get_stock()	Fetches and returns the current stock data, handling potential file or data issues.
while True: print("\nCurrent stock:")	Use while statement to create an infinite loop until a certain requirement is met
for idx, (item, details) in enumerate(stock.items(), start=1): quantity = details.get('quantity', 0) price = details.get('price', 0.0) item_type = details.get('type', 'Unknown') for_staff_only = details.get('for_staff_only', False) visibility = 'Staff Only' if for staff only else 'Staff and Customer'	To iterate through stock items, retrieve their details, and determine their visibility
if not for_staff or (for_staff and for_staff_only): print(f"{idx}. {item} ") print(f" Quantity: {quantity}") print(f" Price: RM{price:.2f}")	Display inventory items based on their visibility settings and the user's status.

<code>print(f' Available for: {visibility}')</code>	
<code>print("\nOptions:") print("1. Update Quantity") print("2. Update Price") print("3. Add New Item") print("4. Remove Item") print("5. Exit")</code>	Print the options available for the user to get a clear view of options. <code>print("\nOptions:")</code> to indicates the difference with other sections.
<code>option = input("Select an option (1-5): ")</code>	option is a variable that store the input from the user. User will be asked to select the option between 1-5.

inventory_stock_check Function-Part2

```

if option == '1':
    item_index = int(input("Enter the item number to update quantity: ")) - 1
    if 0 <= item_index < len(stock):
        item = list(stock.keys())[item_index]
        new_quantity = int(input("Enter the new quantity: "))
        stock[item]['quantity'] = new_quantity
        save_data(stock, STOCK_FILE)
        print(f"Updated quantity for {item} to {new_quantity}.")
    else:
        print("Invalid item number.")
    log_action(username, "Stock Check - Update Quantity")

elif option == '2':
    item_index = int(input("Enter the item number to update price: ")) - 1
    if 0 <= item_index < len(stock):
        item = list(stock.keys())[item_index]
        new_price = float(input("Enter the new price: RM"))
        stock[item]['price'] = new_price
        save_data(stock, STOCK_FILE)
        print(f"Updated price for {item} to RM{new_price:.2f}.")
    else:
        print("Invalid item number.")
    log_action(username, "Stock Check - Update Price")

```

Figure: Function for Stock Checking and Making Adjustment of Inventory Staff

Source Code	Explanation
<code>if option == '1':</code> <code>item_index = int(input("Enter the item number to update quantity: ")) - 1</code>	Check if the user's input is equal to 1 Allow the user to select an item by its number for quantity updating
<code>if 0 <= item_index < len(stock):</code> <code>item =</code> <code>list(stock.keys())[item_index]</code> <code>new_quantity = int(input("Enter the new quantity: "))</code> <code>stock[item]['quantity'] =</code> <code>new_quantity</code> <code>save_data(stock, STOCK_FILE)</code> <code>print(f"Updated quantity for {item} to {new_quantity}.")</code>	Ensures that the item quantity is correctly updated in both the stock dictionary and the stored file, with appropriate user feedback.

<pre> else: print("Invalid item number.") </pre>	Display an error message to notify user that they have entered an invalid item number
<pre> log_action(username, "Stock Check - Update Quantity") </pre>	Record user's action with datetime and named the action as Stock Check-Update Quantity
<pre> elif option == '2': item_index = int(input("Enter the item number to update price: ")) - 1 </pre>	Allow the user to select an item by its number for price updating
<pre> if 0 <= item_index < len(stock): item = list(stock.keys())[item_index] new_price = float(input("Enter the new price: RM")) stock[item]['price'] = new_price save_data(stock, STOCK_FILE) print(f"Updated price for {item} to RM{new_price:.2f}.") </pre>	Allows users to modify item prices and ensures that these changes are stored persistently.
<pre> else: print("Invalid item number.") </pre>	Display an error message to notify user that they have entered an invalid item number
<pre> log_action(username, "Stock Check - Update Price") </pre>	Record user's action with datetime and named the action as Stock Check-Update Price

inventory_stock_check Function-Part3

```
elif option == '3':
    item = input("Enter the name of the new item: ")
    initial_quantity = int(input("Enter the initial quantity of the new item: "))
    initial_price = float(input("Enter the price of the new item: RM"))
    for_staff_only = input("Is this item available for staff only? (Y for Yes | N for No): ").strip().upper()

    if item in stock:
        print(f"{item} already exists in stock.")
    else:
        if for_staff_only == 'Y':
            for_staff_only = True
        elif for_staff_only == 'N':
            for_staff_only = False
        else:
            print("Invalid input. Please try again.")
            return # Exit the function if input is invalid
```

Figure: Function for Stock Checking and Making Adjustment of Inventory Staff

Source Code	Explanation
elif option == '3':	Check if the user's input is equal to 3
item = input("Enter the name of the new item: ")	Item is a variable to store the input from the user which is the name of the new item.
initial_quantity = int(input("Enter the initial quantity of the new item: "))	Initial_quantity is a variable to store the initial quantity of the new item from the user. By define the initial_quantity as integer, the input will only accept integer.
initial_price = float(input("Enter the price of the new item: RM"))	Initial_price is a variable to store the price of the new item. By define the initial_price as float, the user may enter decimal number
for_staff_only = input("Is this item available for staff only? (Y for Yes N for No): ").strip().upper()	for_staff_only is a variable to store the visibility of users. The input will only accepted Y for Yes and N for No.
if item in stock: print(f"{item} already exists in stock.")	Use IF statement to check if an item already exists in the stock dictionary and print a message to notify the user
else: if for_staff_only == 'Y': for_staff_only = True elif for_staff_only == 'N': for_staff_only = False	Handle user input for whether an item is restricted to staff only and convert it to Boolean value
else: print("Invalid input. Please try again.") return	Display an error message when the user has entered an invalid input

inventory_stock_check Function-Part4

```
# Save the item to stock with the appropriate availability
stock[item] = {
    'quantity': initial_quantity,
    'price': initial_price,
    'for_staff_only': for_staff_only
}
save_data(stock, STOCK_FILE)

# Display the registered item details
print(f"New item registered: {item}")
print(f"    Quantity: {initial_quantity}")
print(f"    Price per unit: RM{initial_price:.2f}")
print(f"    For staff only: {for_staff_only}")

# Log the action
log_action(username, "Stock Check - Add New Item")
```

Figure: Function for Stock Checking and Making Adjustment of Inventory Staff

Source Code	Explanation
stock[item] = { 'quantity': initial_quantity, 'price': initial_price, 'for_staff_only': for_staff_only }	Update or create an entry in a stock dictionary with details about a newly registered item
save_data(stock, STOCK_FILE)	Save data in a stock dictionary in a STOCK_FILE
print(f"New item registered: {item}") print(f" Quantity: {initial_quantity}") print(f" Price per unit: RM{initial_price:.2f}") print(f" For staff only: {for_staff_only}")	Display a clear and well-organized output of the details for the new registered item.
log_action(username, "Stock Check - Add New Item")	Record user's action with datetime and named the action as Stock Check-Add New Item

inventory_stock_check Function-Part5

```
elif option == '4':
    item_index = int(input("Enter the item number to remove: ")) - 1
    if 0 <= item_index < len(stock):
        item = list(stock.keys())[item_index]
        del stock[item]
        save_data(stock, STOCK_FILE)
        print(f"Removed item: {item}.")
    else:
        print("Invalid item number.")
    log_action(username, "Stock Check - Remove Item")

elif option == '5':
    print("Exiting stock check.")
    log_action(username, "Stock Check - Exit")
    break

else:
    print("Invalid option. Please try again.")
```

Figure: Function for Stock Checking and Making Adjustment of Inventory Staff

Source Code	Explanation
elif option == '4':	Check if the user input is equal to 4
item_index = int(input("Enter the item number to remove: ")) - 1	Prompt user to get item_index to correctly identifies and removes the item from the list or dictionary
if 0 <= item_index < len(stock): item = list(stock.keys())[item_index] del stock[item] save_data(stock, STOCK_FILE) print(f"Removed item: {item}.")	Ensures the index is within the acceptable range, delete item form the dictionary using the index, save changes to a file and confirm the action to the user.
else: print("Invalid item number.")	Display a message when user has entered an invalid item number
log_action(username, "Stock Check - Remove Item")	Record user's action with datetime and named the action as Stock Check-Remove Item
elif option == '5': print("Exiting stock check.")	Exit from stock check and redirect to staff menu
log_action(username, "Stock Check - Exit")	Record user's action with datetime and named the action as Stock Check-Exit
break	Exit from a loop
else: print("Invalid option. Please try again.")	An error message displayed when the entered option is invalid

place_purchase_order Function-Part1

```
def place_purchase_order(username):
    order_id = generate_order_id()
    order_items = []
    total_price = 0
    stock = get_stock()
    print("*****")
    print("                                Available Items")
    print("*****\n")
    for idx, (item, details) in enumerate(stock.items(), start=1):
        if details.get('for_staff_only') == False:
            print(f"{idx}. {item}")
            print(f"    Price: RM{details['price']:.2f}")
    while True:
        try:
            item = int(input("Please enter the number of item you want to purchase (Enter '0' to finish): "))

            if item == 0:
                break

            item_name = get_item(item)
            if item_name is None:
                print("Invalid choice, please try again")
                continue

            quantity = int(input(f"Please enter the quantity for {item_name}: "))

            if quantity <= 0:
                print("Quantity must be greater than zero.")
                continue

            item_stock = stock[item_name]

            if item_stock['quantity'] >= quantity:
                order_items.append({
                    "name": item_name,
                    "quantity": quantity,
                    "price": item_stock['price'] * quantity
                })
                total_price += item_stock['price'] * quantity
                item_stock['quantity'] -= quantity
                print(f"Added {quantity} {item_name}(s) to your order.")
            else:
                print(f"Insufficient stock for {item_name}. Only {item_stock['quantity']} available.")
        except ValueError:
            print("Invalid input. Please enter a number.")
```

Figure: Function for Place Purchase Order by Customer

create_service_order Function-Part1

```
def create_service_order(username):
    service_id = generate_service_id()
    service_items = []
    total_price = 0
    services = load_data(SERVICE_ORDERS_FILE)
    print("*****")
    print("                                Availabe Service/Repair Option")
    print("*****\n")
    for idx, (service, details) in enumerate(services.items(), start=1):
        print(f"{idx}. {service} : {details['tentative_time']}")
        print(f"    Price: RM{details['service_price']:.2f}")

    while True:
        try:
            service_option = int(input("Please select the service you want to request (Enter '0' to finish): "))

            if service_option == 0:
                break

            service_name = get_service(service_option)
            if service_name is None:
                print("Invalid choice, please try again")
                continue

            if any(service['name'] == service_name for service in service_items):
                print(f"{service_name} has already been added to your order.")
                continue

            service_details = services.get(service_name)
            if service_details:
                service_items.append({
                    "name": service_name,
                    "price": service_details["service_price"],
                    "tentative_time": service_details["tentative_time"]
                })
                total_price += service_details["service_price"]
                print(f"Added {service_name} to your order.")
            except ValueError:
                print("Invalid input. Please enter a number")
```

Figure: Function for Create Service Order by Customer

inventory_purchase_order Function-Part1

```
#PURCHASE ORDER FOR STAFF
def inventory_purchase_order(username):
    stock = get_stock()
    purchased_items = load_data(PURCHASED_ITEMS_FILE)
    user_orders = [order for order in purchased_items if order.get('username') == username]

    print("\n-----Available Items for Inventory Staff-----")
    # Display items with index, item name, quantity, price
    for idx, (item, details) in enumerate(stock.items(), start=1):
        print(f"{idx}. {item}")
        print(f"    Quantity: {details['quantity']}")
        print(f"    Price: RM{details['price']:.2f}")

    #Create a list to store the items that staff bought
    order_items = []
```

Figure: Function for Making Purchase Order by Inventory Staff

Source Code	Explanation
-------------	-------------

def inventory_purchase_order(username):	Define a function named inventory_purchase_order that take 1 parameter which is username.
stock = get_stock() purchased_items = load_data(PURCHASED_ITEMS_FILE)	Fetches and returns the current stock data, handling potential file or data issues. Load data from a file and store it in a variable named purchased_items
user_orders = [order for order in purchased_items if order.get('username') == username]	Filters purchase order with those associated with specified users
print("\n-----Available Items for Inventory Staff-----") # Display items with index, item name, quantity, price for idx, (item, details) in enumerate(stock.items(), start=1): print(f'{idx}. {item}') print(f' Quantity: {details['quantity']}") print(f' Price: RM{details['price']:.2f}') order_items = []	Displays a list of available items for inventory staff with their index, name, quantity, and price.
	Create an empty list named order_items to store details of items in a purchase order

place_purchase_order Function-Part2

```

if order_items:
    order = {
        "order_id": order_id,
        "username": username,
        "items": order_items,
        "total_price": total_price,
        "status": "Unpaid"
    }
    save_purchase_order(order)
    print(f"Order {order_id} placed successfully.")
    log_action(username, "Place Purchase Order")
    customer_interface(username)
else:
    print("No items added to the order.")
return order_id, order_items, total_price

```

Figure: Function for Place Purchase Order by Customer

create_service_order Function-Part2

```
if service_items:
    order = {
        "service_id": service_id,
        "username": username,
        "services": service_items,
        "total_price": total_price,
        "status": "Unpaid"
    }
    save_service_order(order)
    print(f"Service Order {service_id} placed successfully.")
    log_action(username, "Place Service/Repair Order")
    customer_interface(username)
else:
    print("No service added to the order.")
return service_id, service_items, total_price
```

Figure: Function for Create Service Order by Customer

inventory_purchase_order Function-Part2

```
try:
    while True:
        item_idx = int(input("\nEnter the item number to order (or 0 to finish): "))
        if item_idx == 0:
            break

        if item_idx < 1 or item_idx > len(stock):
            print("Invalid item number.")
            continue

        item = list(stock.keys())[item_idx - 1]
        quantity = int(input(f"Enter quantity for {item}: "))

        # Calculate total price for the quantity ordered
        total = stock[item]['price'] * quantity
        order_items.append({
            'item': item,
            'quantity': quantity,
            'price': stock[item]['price'],
            'total_cost': total
        })

        # Update stock
        update_stock(item, +quantity) # Increase stock by the ordered quantity

    # Check if any items were ordered
    if not order_items:
        print("No items were ordered.")
    return
```

Figure: Function for Making Purchase Order by Inventory Staff

try:	Attempts to open and read a file.
while True: item_idx = int(input("\nEnter the item number to order (or 0 to finish): "))	Use while statement to create an infinite loop until a certain requirement is met
if item_idx == 0: break	Use IF statement to check if the value of item_idx is equal to 0, thus break statement is executed.
if item_idx < 1 or item_idx > len(stock): print("Invalid item number.") continue	Use IF statement to check if the value of item_idx is outside the valid range of item numbers. If the value is outside the valid range of item numbers, an error message will be shown and prompting the user to enter a valid number.
item = list(stock.keys())[item_idx - 1] quantity = int(input(f"Enter quantity for {item}: "))	Allows the user to select an item from the stock and specify the amounts of items they would like to buy or modify(cancel, update quantity).
total = stock[item]['price'] * quantity order_items.append({ 'item': item, 'quantity': quantity, 'price': stock[item]['price'], 'total_cost': total })	Calculate the cost of an item based on the selected items and quantity. This information are then added into a list of ordered items by allowing the system to keep track of the details for each item in a purchase order.
update_stock(item, +quantity)	Update the quantity of stock after a purchase order is done.
if not order_items: print("No items were ordered.") return	If order_items is empty, a message will shown hence prevents any processing from occurring. If there are item in the order_items, it will calculate and prints the total price.

inventory_purchase_order Function-Part3

```
# Save the purchase record
total_cost = sum(item['total_cost'] for item in order_items)
status = "Unpaid"
save_purchase_record(username, order_items, total_cost, status)
save_data(stock, "stock.json")

# Print the order summary
print("\nPurchase Order Placed Successfully!")
for idx, item in enumerate(order_items, start=1):
    print(f"{idx}. {item['item']}")
    print(f"    Quantity: {item['quantity']}")
    print(f"    Price: RM{item['price']:.2f}")

print(f"\nTotal Price: RM{total_cost:.2f}")
print(f"Status: {status}")
log_action(username, "Make Purchase Order")

except ValueError:
    print("Invalid input. Please enter numbers only.")
    log_action(username, "Fail to Make Purchase Order")
```

Figure: Function for Making Purchase Order by Inventory Staff

Source Code	Explanation
total_cost = sum(item['total_cost'] for item in order_items)	Summarize total price from a list of orders
status = "Unpaid"	Set status to Unpaid when a purchase order is placed.
save_purchase_record(username, order_items, total_cost, status)	Save the details of a purchase order into save purchase record.
save_data(stock, "stock.json")	Save the data in stock in a JSON file that named stock.json
print("\nPurchase Order Placed Successfully!") for idx, item in enumerate(order_items, start=1): print(f"{idx}. {item['item']}") print(f" Quantity: {item['quantity']}") print(f" Price: RM{item['price']:.2f}")	Inform the user that their purchase order has been placed through a confirmation message and show each item's details clear and well-organized.
print(f"\nTotal Price: RM{total_cost:.2f}") print(f"Status: {status}")	Display the total price for the purchase order that formatted with two decimal places and the current status of the purchase order.
log_action(username, "Make Purchase Order")	Record users' actions with datetime and named the action as Make Purchase Order
except ValueError: print("Invalid input. Please enter numbers only.")	Use to handle a ValueError that may occur during user input processing and notify user with an error message. Hence, record the

log_action(username, "Fail to Make Purchase Order")	users' action with datetime and named the action as Failed to Make Purchase Order.
---	--

generate_invoice Function

```
def generate_invoice(username):
    user_purchase_orders, user_service_orders = check_order_details(username)
    if not user_purchase_orders and not user_service_orders:
        print("No orders found for generating an invoice.")
        return
    print("*****")
    print("                INVOICE")
    print("*****")
    total_amount = 0.0

    if user_purchase_orders:
        print("                Purchase Orders")
        print("-----")
        for order_id, order in user_purchase_orders.items():
            print(f"Order ID: {order_id}")
            for idx, item in enumerate(order['items'], start=1):
                item_details = f"{idx}. {item['name']}"
                if 'quantity' in item:
                    item_details += f" (Quantity: {item['quantity']})"
                item_details += f" - Price: RM{item['price']:.2f}"
                print(f"    {item_details}")
            total_amount += order['total_price']

    elif user_service_orders:
        print("                Service/Repair Orders")
        print("-----\n")
        for order_id, order in user_service_orders.items():
            print(f"Order ID: {order_id}")
            for idx, service in enumerate(order['items'], start=1):
                print(f"{idx}. {service['name']}")
                if 'tentative_time' in service:
                    print(f"    Tentative Time: {service['tentative_time']}")
                print(f"    Price: RM{service['price']:.2f}")
            total_amount += order['total_price']

    print("-----\n")
    print(f"                Total Amount Due: RM{total_amount:.2f}")
    print("-----\n")
    print("                Thank you for your payment!")
    print("*****")
```

Figure: Function for Generate Invoice to the Customer

check_order_details Function

```
def check_order_details(username):
    purchase_orders = load_orders_details('orders.txt')
    service_orders = load_orders_details('service_orders.txt')

    user_purchase_orders = {order_id: order for order_id, order in purchase_orders.items() if order['username'] == username}
    user_service_orders = {order_id: order for order_id, order in service_orders.items() if order['username'] == username}

    return user_purchase_orders, user_service_orders
```

Figure: Function for Check Details of Purchase Order and Service Order of Customer

load_orders_details Function

```
def load_orders_details(filename):
    orders = {}
    try:
        with open(filename, 'r') as file:
            lines = file.readlines()
            for line in lines:
                line = line.strip()
                if not line:
                    continue
                parts = line.split(",")
                if len(parts) < 4:
                    print(f"Skipping malformed line: {line}")
                    continue

                order_id = parts[0]
                try:
                    orders[order_id] = {
                        'username': parts[1],
                        'status': parts[2],
                        'total_price': float(parts[3]),
                        'items': []
                    }
                    for item in parts[4:]:
                        item_parts = item.split(",")
                        if len(item_parts) == 2: # For services
                            item_name = item_parts[0]
                            item_price = float(item_parts[1])
                            orders[order_id]['items'].append({
                                'name': item_name,
                                'price': item_price
                            })
                        elif len(item_parts) == 3: # For items
                            item_name = item_parts[0]
                            quantity = int(item_parts[1])
                            item_price = float(item_parts[2])
                            orders[order_id]['items'].append({
                                'name': item_name,
                                'quantity': quantity,
                                'price': item_price
                            })
                        else:
                            print(f"Skipping malformed item: {item}")
                    except ValueError as e:
                        print(f"Error parsing line {line}: {e}")
            except FileNotFoundError:
                print(f"File {filename} not found.")
            except Exception as e:
                print(f"Error loading orders: {e}")
    return orders
```

Figure: Function for Showing the Details of Orders to the Customer

inquiry_of_order_status Function

```
def inquiry_of_order_status(username):
    log_action(username, "Inquiry of Order Status")
    user_purchase_orders, user_service_orders = check_order_details(username)

    if not user_purchase_orders and not user_service_orders:
        print("You have no orders to inquire about.")
        print("*****")
        print("                        Order Status Inquiry")
        print("*****\n")

    if user_purchase_orders:
        print("Purchase Orders:")
        for order_id, order in user_purchase_orders.items():
            print(f"\nOrder ID: {order_id}")
            print(f"Status: {order['status']}")
            print(f"Total Price: RM{order['total_price']:.2f}")
            print("Items:")
            for item in order['items']:
                print(f" - {item['name']} (RM{item['price']:.2f})")
    else:
        print("No purchase orders found.")

    if user_service_orders:
        print("\nService/Repair Orders:")
        for order_id, order in user_service_orders.items():
            print(f"\nOrder ID: {order_id}")
            print(f"Status: {order['status']}")
            print(f"Total Price: RM{order['total_price']:.2f}")
            print("Services:")
            for item in order['items']:
                print(f" - {item['name']} (RM{item['price']:.2f})")
    else:
        print("No service/repair orders found.")
```

Figure: Function for Inquiry Status of Purchase Order and Service Order by Customer

inventory_check_purchase_order_status Function

```
def inventory_check_purchase_order_status(username):
    log_action(username, "Check Purchase Order Status")
    try:
        #List the purchase orders
        orders = load_data(PURCHASED_ITEMS_FILE)

        user_orders = [order for order in orders if order.get('username') == username]

        if not user_orders:
            print("No orders found.")
            staff_menu(username)
        else:
            print("Purchase orders:")
            #List the purchase order with index number
            for index, order in enumerate(user_orders, start=1):
                # Get the order status, default to 'Unpaid' if not set
                status = "Paid" if order.get('status', 'Unpaid') == 'Paid' else "Unpaid"
                # Safely get the total cost, default to 0.0 if not set
                total_cost = order.get('total_cost', 0.0)

                print(f"\n-----Order ID: {index}-----")
```

Figure: Function for Check Purchase Order Status

Source Code	Explanation
def inventory_check_purchase_order_status(username):	Define a function named inventory_check_purchase_order_status that take 1 parameter which is username.
log_action(username, "Check Purchase Order Status")	Record users' actions with datetime and named the action as Check Purchase Order Status
try:	Attempts to open and read a file.
orders = load_data(PURCHASED_ITEMS_FILE)	Load data from a file and make it as a variable named orders
user_orders = [order for order in orders if order.get('username') == username]	Use to filter a list of orders based on username
if not user_orders: print("No orders found.") staff_menu(username)	Use to handle the case where no orders are found for a specific user
else: print("Purchase orders:")	If the orders are found, a message will be display.
for index, order in enumerate(user_orders, start=1): # Get the order status, default to 'Unpaid' if not set status = "Paid" if order.get('status', 'Unpaid') == 'Paid' else "Unpaid" # Safely get the total cost, default to 0.0 if not set total_cost = order.get('total_cost', 0.0)	Iterate over a list of purchase orders and format each purchase order's status and total cost
print(f"\n-----Order ID: {index}-----")	Use to differentiate the purchase order.
if 'items' in order and isinstance(order['items'], list): for idx, item in enumerate(order['items'], start=1): # Ensure item dictionary contains necessary keys quantity = item.get('quantity', 'Unknown') item_name = item.get('item', 'Unknown') price = item.get('price', 0.0) print(f"{idx}. {item_name}") print(f" Quantity: {quantity}") print(f" Price per unit: RM{price:.2f}")	Use to process and display the details of items in a purchase order.
else: print(" - No items found in this order.")	Display a message about the absence of items when the items key is missing

<pre>print(f"\nTotal Price: RM{total_cost:.2f}") print(f"Status: {status}")</pre>	Ensure the total_cost and status are clearly displayed to the user.
<pre>except Exception as e: print(f"An error occurred while listing orders: {e}")</pre>	Use to handle exceptions that may occur during the execution of code.

service_order Function-Part1

```
def service_order(username):
    services=load_data(SERVICE_ORDERS_FILE)
    print("\nAvailabe Service/ Repairs:")
    for idx,(service, details) in enumerate(services.items(),start=1):
        print(f"{idx}. {service} : {details['tentative_time']}")
        print(f"    Price: RM{details['service_price']:.2f}")

    service_options=input("Enter Y to modify or N to exit.")

    if service_options == 'Y':
        service_name = input("Enter the service name you want to modify: ").strip()
```

Figure: Function for Modify Service Order

Source Code	Explanation
def service_order(username):	Define a function named service_order that take 1 parameter which is username.
services=load_data(SERVICE_ORDERS_FILE)	Load data from a file and make it as a variable named services
print("\nAvailabe Service/ Repairs:")	Print a title which is Available Service/Repairs before showing the list of services and repairs. It will be more neatly and differentiate itself with other section.
for idx,(service, details) in enumerate(services.items(),start=1): print(f"{idx}. {service} : {details['tentative_time']}") print(f" Price: RM{details['service_price']:.2f}")	Iterate over a dictionary of services and display the information about each service including tentative time and price.
service_options=input("Enter Y to modify or N to exit.")	Prompt the user for input and store it in a variable called service_options.
if service_options == 'Y':	Use IF statement to check if the input from the user is equal to Y.
service_name = input("Enter the service name you want to modify: ").strip()	Prompt the user to enter the name of a service to modify.

service_order Function-Part2

```
if service_name in services:
    new_name = input("Enter new service name (or press Enter to keep the current name): ").strip()
    new_time = input("Enter new tentative time (or press Enter to keep the current time): ").strip()
    new_price = input("Enter new service price (or press Enter to keep the current price):RM ").strip()
```

Figure: Function for Modify Service Order

Source Code	Explanation
if service_name in services:	Use IF statement to check if service_name exist in a dictionary called services.
new_name = input("Enter new service name (or press Enter to keep the current name): ").strip()	Prompt user to get input for new_service_name and save it in a variable called new_name.
new_time = input("Enter new tentative time (or press Enter to keep the current time): ").strip()	Prompt user to get input for new_tentative_time and save it in a variable called new_time.
new_price = input("Enter new service price (or press Enter to keep the current price):RM ").strip()	Prompt user to get input for new_service_price and save it in a variable called new_price.

service_order Function-Part3

```
if new_name:
    services[new_name] = services.pop(service_name)
    service_name = new_name

if new_time:
    services[service_name]['tentative_time'] = new_time

if new_price:
    try:
        services[service_name]['service_price'] = float(new_price)
    except ValueError:
        print("Invalid price input. Keeping the old price.")

save_data(services, SERVICE_ORDERS_FILE)
print("Service updated successfully.")
log_action(username, "Modify Service Order")
```

Figure: Function for Modify Service Order

Source Code	Explanation
if new_name:	Use to ensure a variable has valid data.
services[new_name] = services.pop(service_name) service_name = new_name	Update the name of a service in a dictionary which is services
if new_time:	Use to ensure a variable has valid data.
services[service_name]['tentative_time'] = new_time	Update the value associated with tentative_time
if new_price:	Use to ensure a variable has valid data.
try:	Attempts to open and read a file.
services[service_name]['service_price'] = float(new_price)	Update the value associated with service_price
except ValueError: print("Invalid price input. Keeping the old price.")	Handle errors that arise from invalid data
save_data(services, SERVICE_ORDERS_FILE)	Save data in the dictionary which is services into the SERVICE_ORDERS_FILE
print("Service updated successfully.")	Feedback will be show to the user when the service has been updated successfully
log_action(username, "Modify Service Order")	Record the user's action with datetime and named the action as Modify Service Order

service_order Function-Part4

```
else:
    print("Service not found.")
staff_menu(username)
```

Figure: Function for Modify Service Order

Source Code	Explanation
else: print("Service not found.")	Inform user that the specific service is not found.
staff_menu(username)	Call function and led the user to staff_menu

make_payment Function-Part1

```
def make_payment(username):
    user_purchase_orders, user_service_orders = check_order_details(username)

    orders_to_pay = {}

    for order_id, order in user_purchase_orders.items():
        if not order['status'] == 'paid':
            orders_to_pay[order_id] = order

    for order_id, order in user_service_orders.items():
        if not order['status'] == 'paid':
            orders_to_pay[order_id] = order

    if not orders_to_pay:
        print("You have no unpaid orders to pay for.")

    print("Your unpaid orders:")
    for order_id, order in orders_to_pay.items():
        print(f"Order ID: {order_id}, Total Price: RM{order['total_price']:.2f}")

    order_id = input("Enter the Order ID you want to pay for: ")
    if order_id in orders_to_pay:
        orders_to_pay[order_id]['status'] = 'paid'
        save_orders('orders.txt', user_purchase_orders)
        save_orders('service_orders.txt', user_service_orders)
        print(f"Order {order_id} has been paid successfully.")
        generate_invoice(username)
        log_action(username, "Make Payment")
    else:
        print("Invalid Order ID.")
```

Figure: Function for Making Payment by Customer

inventory_payment Function-Part1

```
def inventory_payment(username):  
    log_action(username, "Make Payment")  
  
    # Load purchase orders and filter for the specific user  
    orders = load_data(PURCHASED_ITEMS_FILE)  
    user_orders = [order for order in orders if order.get('username') == username]  
  
    if not user_orders:  
        print(f"No purchase orders found for {username}. No payment needed.")  
        return  
  
    print("\nYour Purchase Orders:")  
    for index, order in enumerate(user_orders, start=1):  
        # Get the order status, default to 'Unpaid' if not set  
        status = "Paid" if order.get('status', 'Unpaid') == 'Paid' else "Unpaid"  
        total_cost = order.get('total_cost', 0.0)
```

Figure: Function for Making Payment

Source Code	Explanation
def inventory_payment(username):	Define a function named inventory_payment that take 1 parameter which is username.
log_action(username, "Make Payment")	Records user's action with datetime and named the action as Make Payment
orders = load_data(PURCHASED_ITEMS_FILE) user_orders = [order for order in orders if order.get('username') == username]	Load data from a file and make it as a variable named orders. Use to filter a list of orders based on username
if not user_orders: print(f"No purchase orders found for {username}. No payment needed.") return	Use to handle a situation when there is no purchase orders found for the specific username.
print("\nYour Purchase Orders:")	Show the output in a clear and well-organized layout starting with a header.
for index, order in enumerate(user_orders, start=1): status = "Paid" if order.get('status', 'Unpaid') == 'Paid' else "Unpaid" total_cost = order.get('total_cost', 0.0)	Loop that iterates over the user_orders list and start the index with 1. Assign the status and total cost of the purchase order

inventory_payment Function-Part2

```
print(f"\n-----Order ID: {index}-----")

# Display Purchase Order
if 'items' in order and isinstance(order['items'], list):
    for idx, item in enumerate(order['items'], start=1):
        item_name = item.get('item', 'Unknown')
        quantity = item.get('quantity', 'Unknown')
        price = item.get('price', 0.0)
        print(f"{idx}. {item_name}")
        print(f"    Quantity: {quantity}")
        print(f"    Price per unit: RM{price:.2f}")
    print(f"\nTotal Price: RM{total_cost:.2f}")
    print(f"Status: {status}")
else:
    print(" - No items found in this order.")
```

Figure: Function for Making Payment

Source Code	Explanation
<pre>print(f"\n-----Order ID: {index}-----")</pre>	Print the Order ID and the order_id will be auto generated by the index. It is used to show the differences between each purchase order. Hence, each purchase order will has their own order_id.
<pre>if 'items' in order and isinstance(order['items'], list):</pre>	Use to check whether the order dictionary contains a key items and whether the value of the key is a list
<pre>for idx, item in enumerate(order['items'], start=1): item_name = item.get('item', 'Unknown') quantity = item.get('quantity', 'Unknown') price = item.get('price', 0.0) print(f"{idx}. {item_name}") print(f" Quantity: {quantity}") print(f" Price per unit: RM{price:.2f}")</pre>	Print details of each item in a clear and well-organized way
<pre>print(f"\nTotal Price: RM{total_cost:.2f}") print(f"Status: {status}")</pre>	Ensure the total_cost and status are clearly displayed to the user.
<pre>else: print(" - No items found in this order.")</pre>	Notify users when the specific item doesn't exist in the purchase order.

inventory_payment Function-Part3

```
# Ask for Order ID to make payment
try:
    payment_id = int(input("\nSelect your Order ID to make payment: "))

    if 1 <= payment_id <= len(user_orders):
        selected_order = user_orders[payment_id - 1]

        # Check current status
        if selected_order.get('status', 'Unpaid') == 'Paid':
            print("This order is already paid.")
            return

        answer = input("Do you want to make payment (Y for Yes, N for No): ").strip().upper()
        if answer == 'Y':
            selected_order['status'] = 'Paid'
            save_data(user_orders, PURCHASED_ITEMS_FILE) # Save the updated orders
            print("Payment successful. You will receive your goods in 3-4 working days.")
        elif answer == 'N':
            print("Payment not completed. Don't forget to pay.")
        else:
            print("Invalid input. Please try again.")
    else:
        print("Invalid Order ID.")
except ValueError:
    print("Invalid input. Please enter a valid number.")

log_action(username, "Attempted to Make Payment")
```

Figure: Function for Making Payment

Source Code	Explanation
try:	Attempts to open and read a file.
payment_id = int(input("\nSelect your Order ID to make payment: "))	Prompt user to get payment_id and limit the input as integer
if 1 <= payment_id <= len(user_orders): selected_order = user_orders[payment_id - 1]	Validate and select an order based on the user's input
if selected_order.get('status', 'Unpaid') == 'Paid': print("This order is already paid.") return	Check the status of a specific order and handle the scenario where the order has already been paid
answer = input("Do you want to make payment (Y for Yes, N for No): ").strip().upper()	Answer is a variable which store the input from the user to check if they want to make payment.
if answer == 'Y': selected_order['status'] = 'Paid' save_data(user_orders, PURCHASED_ITEMS_FILE) print("Payment successful. You will receive your goods in 3-4 working days.")	Use IF statement to check if the answer is equal to Y. Hence, the status will changed to Paid and the data will be saved in the orders dictionary and saved in PURCHASED_ITEMS_FILE. A payment successful message will inform to the user too.
elif answer == 'N': print("Payment not completed. Don't forget to pay.")	Handle a case where the user's response indicates that they do not wish to proceed with a payment
else: print("Invalid input. Please try again.")	Handle cases where none of the specified conditions are met.

else: print("Invalid Order ID.")	Notify user that the entered order_id is invalid
except ValueError: print("Invalid input. Please enter a valid number.")	Handle errors that arise from invalid data
log_action(username, "Attempted to Make Payment")	Record the user's action with datetime and named the actions as Attempted to Make Payment

customer_reports Function

```
def customer_reports(username):
    log_action(username, "Generate Customer Report")
    reports = load_data(REPORTS_FILE)
    user_reports = reports.get(username, [])
    if not user_reports:
        print(f"No reports found for user: {username}.")
    else:
        print(f"\nReports for {username}:")
        print("    Date    | Time |      Actions")
        for action in user_reports:
            print(f" - {action['time']}: {action['action']}")

    input("\nPress Enter to return to the main menu...")
```

Figure: Function for Customer to View Reports

staff_view_reports Function

```
def staff_view_reports(username):
    #View the action logs (reports) for a specific user.
    reports = load_data(REPORTS_FILE)

    # Get reports for the specific username
    user_reports = reports.get(username, [])

    if not user_reports:
        print(f"No reports found for user: {username}.")
    else:
        print(f"\nReports for {username}:")
        print("    Date    | Time |      Actions")
        for action in user_reports:
            print(f" - {action['time']}: {action['action']}")

    input("\nPress Enter to return to the main menu...")
```

Figure: Function for Staff to View Reports

Source Code	Explanation
def staff_view_reports(username):	Define a function named staff_view_reports that take 1 parameter which is username.

reports = load_data(REPORTS_FILE)	Load data from a file and store in a variable named reports
user_reports = reports.get(username, [])	use to retrieve reports for a specific user
if not user_reports: print(f"No reports found for user: {username}.")	Show a message when no reports are available for a specified user
else: print(f"\nReports for {username}:") print(" Date Time Actions")	Handle the case where there are reports available for the specified user
for action in user_reports: print(f" - {action['time']}: {action['action']}")	Iterate over each report entry in the user_reports list and print its details
input("\nPress Enter to return to the main menu...")	Notify user to press enter to return to the main menu

staff_view_service_orders Function-Part1

```
def staff_view_service_orders():
    # Load service orders from the file
    service_orders = load_orders_details('service_orders.txt')

    # Check if there are any service orders to display
    if not service_orders:
        print("No service orders found.")
        return

    # Print the order summary header
    print("*****")
    print("Order Summary")
    print("*****\n")

    # Display each service order
    for order_id, order in service_orders.items():
        print(f"Order ID: {order_id}, Customer: {order['username']}, Status: {order['status']}, Total Price: RM{order['total_price']:.2f}")
        for item in order['items']:
            print(f" - {item['name']} (RM{item['price']:.2f})")
        print() # Print a newline for better readability
```

Figure: Function for Staff to View and Modify Service Orders of the Customers

Source Code	Explanation
def staff_view_service_orders():	Define a function named staff_view_service_orders
service_orders = load_orders_details('service_orders.txt')	Load the details of orders from service_orders.txt and store the data in a variable named service_orders
if not service_orders: print("No service orders found.") return	Handle cases where there are no service orders available
print("*****") print("Order Summary") print("*****\n")	Differentiate Order Summary with other section.
for order_id, order in service_orders.items(): print(f"Order ID: {order_id}, Customer: {order['username']}, Status:	Display a detailed summary of each service order


```
{order['status']}, Total Price:
RM{order['total_price']:.2f}")
    for item in order['items']:
        print(f" - {item['name']}")
    (RM{item['price']:.2f})")
    print()
```

staff_view_service_orders Function-Part2

```
# Prompt for status update
while True:
    order_id = input("Enter the Order ID you want to modify (or type '0' to quit): ")
    if order_id == '0':
        break

    if order_id not in service_orders:
        print("Invalid Order ID.")
        continue

    current_status = service_orders[order_id]['status']
    print(f"Current status for Order ID {order_id}: {current_status}")

    new_status = input("Enter new status (Done/Processing): ").capitalize()
    if new_status not in ['Done', 'Processing']:
        print("Invalid status. Please enter 'Done' or 'Processing'.")
        continue

    # Update the order status
    service_orders[order_id]['status'] = new_status
    print(f"Order ID {order_id} status updated to {new_status}.")

    # Save updated orders back to the file
    save_orders_details('service_orders.txt', service_orders)
```

Figure: Function for Staff to View and Modify Service Orders of the Customers

Source Code	Explanation
while True:	Use while statement to create an infinite loop and the loop will be keep running until a condition is met.
order_id = input("Enter the Order ID you want to modify (or type '0' to quit): ") if order_id == '0': break if order_id not in service_orders: print("Invalid Order ID.") continue	Prompt user for order_id, provide them to exit by entering '0' and notify them if the order_id is invalid
current_status = service_orders[order_id]['status'] print(f"Current status for Order ID {order_id}: {current_status}")	Retrieve and display the current status of a specific service order
new_status = input("Enter new status (Done/Processing): ").capitalize() if new_status not in ['Done', 'Processing']: print("Invalid status. Please enter 'Done' or 'Processing'.")	Prompt the user to input a new status for an order and validate that the input is either "Done" or "Processing"

continue	
service_orders[order_id]['status'] = new_status print(f"Order ID {order_id} status updated to {new_status}.")	Update the status of a specific service order in a dictionary. Inform user with a message that the status has been updated.
save_orders_details('service_orders.txt', service_orders)	Save service_orders dictionary to a file named service_orders.txt

customer_interface Function

```
def customer_interface(username):
    while True:
        print("\n*****")
        print("                                Customer Menu")
        print("*****\n")
        print("1. Purchase Order")
        print("2. Service/Repair Order")
        print("3. Modify Purchase/Service/Repair Order")
        print("4. Make payment")
        print("5. Inquiry of Order Status")
        print("6. Cancel Orders")
        print("7. Reports")
        print("8. Give feedback")
        print("9. Log Out")
        try:
            option = int(input("Please select an option to continue with: "))
            if option == 1:
                order, order_id = place_purchase_order(username)
            elif option == 2:
                create_service_order(username)
            elif option == 3:
                modify_order(username)
            elif option == 4:
                make_payment(username)
            elif option == 5:
                inquiry_of_order_status(username)
            elif option == 6:
                cancel_order(username)
            elif option == 7:
                customer_reports(username)
            elif option == 8:
                collect_feedback(username)
            elif option == 9:
                print("Logging out.")
                log_action(username, "Customer Log Out")
                homepage()
            else:
                print("Invalid option. Please try again.")
        except ValueError:
            print("Invalid input. Please try again.")
```

Figure: Function for Customers to Interact Customer Menu

staff_menu Function

```
def staff_menu(username):
    #Main menu for interacting with the inventory system
    initialize_files()
    log_action(username, "Login")
    while True:
        print("\n*****")
        print("      Inventory Menu      ")
        print("*****")
        print("1. Place Purchase Order")
        print("2. Modify Purchase Order")
        print("3. Cancel Purchase Order")
        print("4. Check Purchase Order Status")
        print("5. Stock Check/Adjustments")
        print("6. Payment")
        print("7. Service/ Repairs")
        print("8. Report")
        print("9. View Service Order")
        print("10. Logout")
        choice = input("Choose an option: ")
```

Figure: Function for Staff to Interact Inventory Menu

Source Code	Explanation
def staff_menu(username):	Define a function named staff_menu that take 1 parameter which is username.
initialize_files()	Used to set up or prepare files necessary for the application to function properly.
log_action(username, "Login")	Record the user's action with datetime and named the action as Login
while True:	Use while statement to create an infinite loop until a certain requirement is met
print("\n*****") print(" Inventory Menu ") print("*****") print("1. Place Purchase Order") print("2. Modify Purchase Order") print("3. Cancel Purchase Order") print("4. Check Purchase Order Status") print("5. Stock Check/Adjustments") print("6. Payment") print("7. Service/ Repairs") print("8. Report") print("9. Logout") print("10. View Feedback") print("11. Give Feedback")	Print the title which is Inventory Menu to differentiate Inventory Menu with other section. All option will be listed according to the index number.

choice = input("Choose an option: ")	Choice is a variable that will accept input from the user. Hence, it will lead the user to the function that they chose.
--------------------------------------	--

staff_menu Function-Part2

```

if choice == '1':#Place purchase order
    inventory_purchase_order(username)
elif choice == '2':#Modify
    inventory_check_purchase_order_status(username) # Show orders to find the ID
    order_id = int(input("Enter Order ID to modify: "))
    item=str(input("Enter Item Name to modify:"))
    new_quantity = int(input("Enter new quantity: "))
    inventory_modify_order(username, order_id, item, new_quantity)
elif choice == '3':#Cancel purchase order
    inventory_check_purchase_order_status(username) # Show orders to find the ID
    order_id = int(input("Enter order ID to cancel: "))
    inventory_cancel_order(username, order_id)
elif choice == '4':#Check status
    inventory_check_purchase_order_status(username)
elif choice == '5':#Able to make adjustments with stock
    inventory_stock_check(username, for_staff=False)
elif choice == '6':#Make payment
    inventory_payment(username)
elif choice == '7':#Show service/repairs
    service_order(username)
elif choice == '8':#View reports
    log_action(username,"View Reports")
    staff_view_reports(username)
elif choice == '9':
    log_action(username,"View Service Orders")
    staff_view_service_orders()
elif choice == '10':
    log_action(username,"Logout")
    homepage()
else:
    print("Invalid choice.Please try again.")
    continue

```

Figure: Function for Customers to Interact Inventory Menu

Source Code	Explanation
if choice == '1': inventory_purchase_order(username)	Redirect the user to the inventory_purchase_order when the choice is equal to 1
elif choice == '2':#Modify inventory_check_purchase_order_status(username) order_id = int(input("Enter Order ID to modify: ")) item=str(input("Enter Item Name to modify:")) new_quantity = int(input("Enter new quantity: ")) inventory_modify_order(username, order_id, item, new_quantity)	Allow the user to modify purchase order after checking the status. Gather order_id item and new_quantity from the user and modify the specific purchase order.
elif choice == '3':	Allow user to cancel purchase order by prompting user for order_id

<pre> inventory_check_purchase_order_status(use rname) order_id = int(input("Enter order ID to cancel: ")) inventory_cancel_order(username, order_id) </pre>	
<pre> elif choice == '4': inventory_check_purchase_order_status(use rname) </pre>	Allow user to check the status of the purchase order
<pre> elif choice == '5': inventory_stock_check(username, for staff=False) </pre>	Allow user to have a stock check and making adjustment with the initial stock
<pre> elif choice == '6': inventory_payment(username) </pre>	Allow user to make payment for the unpaid purchase order
<pre> elif choice == '7': service_order(username) </pre>	Allow user to modify service order
<pre> elif choice == '8': log_action(username,"View Reports") staff_view_reports(username) </pre>	Allow user to view reports which include username, datetime and actions
<pre> elif choice == '9': log_action(username,"View Service Orders") staff_view_service_orders() </pre>	Allow user to view service orders placed by customers and modify the status
<pre> elif choice == '10': log_action(username,"Logout") homepage() </pre>	Allow the user to logout from the system by redirecting the user to homepage
<pre> else: print("Invalid choice.Please try again.") continue </pre>	An error message is shown when invalid choice is given by the user

```

#SuperUser_Login
def SuperUser_login():
    max_attempts=3
    attempts=0

    while attempts<max_attempts:
        print("\n*****")
        print("      Super User Login      ")
        print("*****")
        print("Enter Super User ID: ")
        superuser_id = input()
        print("Enter Password: ")
        password = input()

        valid_login=False

        try:
            with open('SuperUser.txt', 'r') as file:
                for line in file:
                    stored_superuser_id, stored_password = line.strip().split(',')
                    if stored_superuser_id == superuser_id and stored_password == password:
                        valid_login=True
                        break
        except FileNotFoundError:
            print("User data file not found.")
            return

        if valid_login:
            print("Login successful.")
            SuperUser_menu(superuser_id)
            return
        else:
            attempts +=1
            print(f"Invalid credentials. You have {max_attempts-attempts} attempts left.")

    print("Too many failed attempts. Access denied.")
    reset_superuser_password(superuser_id)

#SuperUserLoginPage
def create_SuperUser_data():
    SuperUser_data=[
        'superuser,superpass'
    ]
    with open('SuperUser.txt','w') as file:
        for line in SuperUser_data:
            file.write(line + '\n')
    print("Super User data file created with initial data.")

def read_file():
    try:
        with open('SuperUser.txt', 'r') as file:
            content=file.read()
            print("File content:")
            print(content)
    except FileNotFoundError:
        print("File not found.")

```

```

def reset_superuser_password(superuser_id):
    print("*****")
    print("      Reset Password      ")
    print("*****")

    try:
        with open('SuperUser.txt', 'r') as file:
            superusers = file.readlines()
    except FileNotFoundError:
        print("User data file not found.")
        return

    for i, line in enumerate(superusers):
        stored_superuser_id, stored_password = line.strip().split(',')
        if stored_superuser_id == superuser_id:
            while True:
                new_password = input("Enter your new password (at least 8 characters, including uppercase letter, number, and special character): ")

                if len(new_password) < 8:
                    print("Password must be at least 8 characters.")
                    continue

                has_upper = any(char.isupper() for char in new_password)
                has_digit = any(char.isdigit() for char in new_password)
                special_characters = "!@#$%^&* -+=()./?:;'\<>[]{}|"
                has_special = any(char in special_characters for char in new_password)

                if not has_upper:
                    print("Password must contain at least 1 uppercase letter.")
                    continue
                if not has_digit:
                    print("Password must contain at least 1 number.")
                    continue
                if not has_special:
                    print("Password must contain at least 1 special character.")
                    continue

                superusers[i] = f"{stored_superuser_id},{new_password}\n"
                with open('SuperUser.txt', 'w') as file:
                    file.writelines(superusers)
                print("Password reset successfully.")
                return
            print("SuperUser ID not found.")

```

```

def SuperUser_menu(superuser_id):
    print("Welcome to the Super User Menu.")

```

```

SuperUser_menu(SuperUser_login)

```

```

def save_user_to_file(username, password, role):
    updated_users = []
    print("*****")
    print("      Add User      ")
    print("*****")
    if role == 'staff':
        file_path = 'staff.txt'
    elif role == 'admin':
        file_path = 'admin.txt'
    elif role == 'customer':
        file_path = 'customers.txt'
    else:
        return

    try:
        with open(file_path, 'a') as file:
            file.write(f"{username},{password},active\n")
    except Exception as e:
        print(f"An error occurred while writing to {file_path}: {e}")

```

```

#Superuser_View_Reports
def Superuser_View_Reports(superuser_id):
    """View the action logs (reports)."""
    log_action(superuser_id, "View Reports")
    reports = load_data(REPORTS_FILE)

    if not reports:
        print("No reports found.")
    else:
        for superuser_id, actions in reports.items():
            print(f"\nReports:")
            print(f"Super User ID: {superuser_id}")
            print("    Date    |    Time    |    Actions")
            for action in actions:
                print(f"    - {action['time']}: {action['action']}")

        input("\nPress Enter to return to the main menu...")

#view feedback
def view_all_feedback(superuser_id):
    log_action(superuser_id, "View Feedback")
    feedback_data = load_data('feedback.json')
    print("Loaded feedback data:", feedback_data) # Inspect the data
    for feedback in feedback_data:
        if isinstance(feedback, dict): # Ensure feedback is a dictionary
            print(f"Username: {feedback['username']}")
            print(f"Rating: {feedback['rating']}")
            print(f"Comments: {feedback['comments']}")
            print(f>Date: {feedback['date']}")
            print("-" * 20)
        else:
            print("Unexpected data format:", feedback)

#AdminSignUp
def Admin_Signup():
    role="admin"
    status="pending"
    print("\n*****")
    print("    Admin Registration Page    ")
    print("*****")

    while True:
        username = input("\nCreate username: ")
        if not check_username(username, "admin.txt"):
            break
        else:
            print("Username already exists. Please try another one.")

    while True:
        password = input("Create password (At least 8 characters, including uppercase letter, number, and special character): ")
        valid, message = check_password(password)

        if valid:
            verify_pw = input("Re-enter password: ")
            if verify_pw == password:
                print("-Password match-")
                break
            else:
                print("Password do not match. Please try again.")
        else:
            print(message)

    save_pendingAdmin(username, password, role, status, "users.txt")
    print("Your registration is pending to be approved.")
    print("You will be returned to Homepage.")

    admin_interface()

```



```

def save_pendingAdmin(username, password, role, status, filename):
    try:
        with open('users.txt', "a") as file:
            file.writelines(f"{username},{password},{role},{status}\n")

    except FileNotFoundError:
        return False
    return False

#reset_admin_password
def reset_admin_password(username, filename):
    print("*****")
    print("Reset Password")
    print("*****")

    attempt_count = 0
    max_attempts = 5

    while attempt_count < max_attempts:
        new_password = input("Please enter new password (At least 8 characters, including uppercase letter, number, and special character): ")

        if len(new_password) < 8:
            print("Password must be at least 8 characters.")
            attempt_count += 1
            continue

        has_upper = any(char.isupper() for char in new_password)
        has_digit = any(char.isdigit() for char in new_password)
        special_characters = "!@#$%^&*_-+=()./?;:\"><>[]{}"
        has_special = any(char in special_characters for char in new_password)

        if not has_upper:
            print("Password must contain at least 1 uppercase letter.")
            attempt_count += 1
            continue

        if not has_digit:
            print("Password must contain at least 1 number.")
            attempt_count += 1
            continue

        if not has_special:
            print("Password must contain at least 1 special character.")
            attempt_count += 1
            continue

        # Password Confirmation
        confirm_password = input("Please confirm your new password: ")
        if new_password != confirm_password:
            print("Passwords do not match. Please try again.")
            attempt_count += 1
            continue

        # If all checks passed, modify the password
        valid, message = modify_admin_password(username, new_password, filename)
        if valid:
            return True, "Password has been reset successfully."
        else:
            return False, "Failed to reset password. Please try again."

    return False, "Too many failed attempts. Please contact the system administrator."

#verify_Admincredentials
def verify_Admincredentials(username, password, active, filename):
    print("Verifying Admin Credentials...")
    try:
        with open(filename, "r") as file:
            for line in file.readlines():
                stored_username, stored_password, stored_active = line.strip().split(",")

                if username == stored_username and password == stored_password and active==stored_active:
                    print("Credentials verified!")
                    return True
    except FileNotFoundError:
        return False

```

Additional features source code

Reset Password

reset_customer_password Function

```
def reset_customer_password(username, filename):
    print("*****")
    print("Reset Password")
    print("*****\n")
    while True:
        print("Password Requirements: At least 8 characters, including uppercase letter, number, and special character")
        new_password = input("Please enter new password: ")

        if len(new_password) < 8:
            print("Password must be at least 8 characters.")
            continue

        has_upper = any(char.isupper() for char in new_password)
        has_digit = any(char.isdigit() for char in new_password)
        special_characters = "!@#$%^&* _+=()./?;:\<>[]{}|"
        has_special = any(char in special_characters for char in new_password)

        if not has_upper:
            print("Password must contain at least 1 uppercase letter.")
            continue
        if not has_digit:
            print("Password must contain at least 1 number.")
            continue
        if not has_special:
            print("Password must contain at least 1 special character.")
            continue

        success = modify_customer_password(username, new_password, filename)
        if success:
            return True, "Password has been reset successfully."
        else:
            return False, "Failed to reset password. Please try again."
```

Figure : Function for the Staff to Reset Password

reset_staff_password Function-Part1

```
def reset_staff_password(username, filename):
    print("*****")
    print("Reset Password")
    print("*****")

    attempt_count = 0
    max_attempts = 5
```

Figure : Function for the Staff to Reset Password

def reset_password(username, filename):	Define a function named reset_password that take two parameters which are username and filename.
def reset_password(username, filename):	Define a function named reset_password that take two parameters which are username and filename.
attempt_count = 0 max_attempts = 5	Set attempt_count = 0 and max_attempts = 5 to define and initialize variables that control how many attempts can be down by the user within a loop

reset_staff_password Function-Part2

```
while attempt_count < max_attempts:  
    new_password = input("Please enter new password (At least 8 characters, including uppercase letter, number, and special character): ")
```

Figure : Function for the Staff to Reset Password

Source Code	Explanation
while attempt_count < max_attempts:	Use While loop to control how many times a user can attempt to perform a certain action. When the attempt_count reached max_attempts, the loop will be terminate.
new_password = input("Please enter new password (At least 8 characters, including uppercase letter, number, and special character): ")	Prompt a new password from the user with a message stating that password should meet specific security criteria of at least 8 characters, including 1 uppercase letter, 1 number and 1 special character.

reset_staff_password Function-Part3

```
if len(new_password) < 8:  
    print("Password must be at least 8 characters.")  
    attempt_count += 1  
    continue  
  
has_upper = any(char.isupper() for char in new_password)  
has_digit = any(char.isdigit() for char in new_password)  
special_characters = "!@#$%^&*_-+=()./?;:\<>[]{}|}"  
has_special = any(char in special_characters for char in new_password)
```

Figure : Function for the Staff to Reset Password

Source Code	Explanation
if len(new_password) < 8:	Use IF statement to check if the length of the new_password entered is less than 8 characters. It is used to ensure that the password has meet a minimum length requirement.
print("Password must be at least 8 characters.") attempt_count += 1 continue	Inform the user that the entered password doesn't meet the minimum length requirement which is 8 characters. The attempt_count is increased by 1 to track how many attempts has the user made by entering an invalid password. The continue statement skips the rest of the loop's code and goes back to the beginning of the loop to prompt the user for the input which is new_password.
has_upper = any(char.isupper() for char in new_password)	has_upper is a variable that used to check whether the new_password contains at least 1 uppercase letter.

has_digit = any(char.isdigit() for char in new_password)	has_digit is a variable that used to check whether the new_password contains at least 1 digit.
special_characters = "!@#\$%^&*_-+=()./?;'\"><>[]{}"	special_characters is a variable that holds a set of special characters. It has includes various special characters that are commonly used in passwords to ensure the complexity and security of the password.
has_special = any(char in special_characters for char in new_password)	has_special is a variable that used to check whether the new_password contains at least 1 special character.

reset_staff_password Function-Part4

```

if not has_upper:
    print("Password must contain at least 1 uppercase letter.")
    attempt_count += 1
    continue
if not has_digit:
    print("Password must contain at least 1 number.")
    attempt_count += 1
    continue
if not has_special:
    print("Password must contain at least 1 special character.")
    attempt_count += 1
    continue

```

Figure : Function for the Staff to Reset Password

Source Code	Explanation
<pre> if not has_upper: print("Password must contain at least 1 uppercase letter.") attempt_count += 1 continue </pre>	if not has_upper is used to check if the password does not contain any uppercase letter. Hence, user will receive a message that the password must include at least 1 uppercase letter. attempt_count will be increased by 1 to use for limiting the number of attempts. continue is used to skip its current iteration and move on to the next iteration
<pre> if not has_digit: print("Password must contain at least 1 number.") attempt_count += 1 continue </pre>	if not has_upper is used to check if the password does not contain any number. Hence, user will receive a message that the password must include at least 1 number. attempt_count will be increased by 1 to use for limiting the number of attempts. continue is used to skip its current iteration and move on to the next iteration
<pre> if not has_special: print("Password must contain at least 1 special character.") attempt_count += 1 </pre>	if not has_upper is used to check if the password does not contain any special character. Hence, user will receive a

continue	message that the password must include at least 1 special character. attemp_count will be increased by 1 to use for limiting the number of attempts. continue is used to skip its current iteration and move on to the next iteration
----------	---

reset_staff_password Function-Part5

```
# Password Confirmation
confirm_password = input("Please confirm your new password: ")
if new_password != confirm_password:
    print("Passwords do not match. Please try again.")
    attempt_count += 1
    continue
```

Figure : Function for the Staff to Reset Password

Source Code	Explanation
confirm_password = input("Please confirm your new password: ")	confirm_password is a variable that will store the input of the user which is confirm_password. It is used to ensure the new_password is correctly entered
if new_password != confirm_password: print("Passwords do not match. Please try again.") attempt_count += 1 continue	If new_password is not match with confirm_password, user will informed that the password do not match and they will be asked to enter the new_password and confirm_password again. attempt_count will be increased by 1 to use for limiting the number of attempts. continue is used to skip its current iteration and move on to the next iteration

reset_staff_password Function-Part6

```
# If all checks passed, modify the password
valid, message = modify_staff_password(username, new_password, filename)
if valid:
    return True, "Password has been reset successfully."
else:
    return False, "Failed to reset password. Please try again."
return False, "Too many failed attempts. Please contact the system administrator."
```

Figure : Function for the Staff to Reset Password

Source Code	Explanation
valid, message = modify_staff_password(username, new_password, filename)	valid will hold a value indicating if the password has been modified successfully. It is a boolean which contains only two values which are True or False. message is a variable that will hold a message returned by the function. It may be used to provide the information about the modification of password. modify_staff_password(username, new_password, filename) is a function that is used to update the password for a staff member. It uses three parameters which are username, new_password and filename.
if valid: return True, "Password has been reset successfully."	Use IF statement to check if the value of valid is True or False. If the value of valid is True, the password will be reset and a message will be shown to the user about password has been reset successfully
else: return False, "Failed to reset password. Please try again."	when the if condition is not met, the value of valid is False. Hence, the user will be notified that they have failed to reset the password.
return False, "Too many failed attempts. Please contact the system administrator."	return is used to exit a function and return the values to the caller. return False indicates that the operation was unsuccessful due to too many failed attempts. Hence, a message will be shown to the user and advise them to seek help from the system administrator.

Collect Feedback

```
def collect_feedback(username):
    print("Please provide your feedback.")

    rating = None
    while rating is None:
        try:
            rating = int(input("Rate our service (1-5): "))
            if rating < 1 or rating > 5:
                print("Please enter a number between 1 and 5.")
                rating = None
        except ValueError:
            print("Invalid input. Please enter a number between 1 and 5.")

    comments = input("Please leave your comments: ")

    feedback = {
        'username': username,
        'rating': rating,
        'comments': comments,
        'date': datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    }

    feedback_data = load_data('feedback.json')
    feedback_data.append(feedback)
    save_data(feedback_data, 'feedback.json')

    print("Thank you for your feedback!")
    log_action(username, "Give Feedback")
```

Figure: Function for Collecting Feedback from the Customer

Source Code	Explanation
def collect_feedback(username):	Define a function and named it collect_feedback that take one parameter which is username
print("Please provide your feedback.")	Prompts the user to start the feedback process
rating = None while rating is None:	Initializes the rating variable to None, ensuring it enters the loop for user input and continues to prompt the user for a valid rating until a correct value is provided.
try:	Attempts to open and read a file.
rating = int(input("Rate our service (1-5): ")) if rating < 1 or rating > 5: print("Please enter a number between 1 and 5.") rating = None	Prompts the user to input a rating and limit the input as integer. The program will check if the entered rating is within the valid range (1to5).
except ValueError: print("Invalid input. Please enter a number between 1 and 5.")	An error message will be displayed if the user has entered an invalid input.

<pre> comments = input("Please leave your comments: ") feedback = { 'username': username, 'rating': rating, 'comments': comments, 'date': datetime.now().strftime("%Y-%m-%d %H: %M:%S") } </pre>	Collects any comments from the user and the feedback dictionary will captures all the details.
<pre> feedback_data = load_data('feedback.json') feedback_data.append(feedback) save_data(feedback_data, 'feedback.json') </pre>	Load the existing data from feedback.json and add the new collected feedback to the existing list. Hence, save the existing list back to feedback.json to ensure the data is stored permanently.
<pre> print("Thank you for your feedback!") </pre>	Notify users when they gave feedback.

View Feedback

```

#view feedback
def view_all_feedback(superuser_id):
    log_action(superuser_id, "View Feedback")
    feedback_data = load_data('feedback.json')
    print("Loaded feedback data:", feedback_data) # Inspect the data
    for feedback in feedback_data:
        if isinstance(feedback, dict): # Ensure feedback is a dictionary
            print(f"Username: {feedback['username']}")
            print(f"Rating: {feedback['rating']}")
            print(f"Comments: {feedback['comments']}")
            print(f>Date: {feedback['date']}")
            print("-" * 20)
        else:
            print("Unexpected data format:", feedback)

```

Figure: Function for View Feedback Given by the Customer

Source Code	Explanation
<pre>def view_all_feedback(superuser_id):</pre>	Define a function named view_all_feedback that take one parameter with is superuser_id.
<pre>log_action(superuser_id, "View Feedback")</pre>	Record the user's action with datetime and named the action as View Feedback.
<pre>feedback_data = load_data('feedback.json')</pre>	Load data from feedback.json and store the values into a variable named feedback_data.
<pre>print("Loaded feedback data:", feedback_data)</pre>	Prints the loaded feedback data for inspection to ensure the data is loaded correctly.
<pre>for feedback in feedback_data: if isinstance(feedback, dict): # Ensure feedback is a dictionary print(f"Username: {feedback['username']}") print(f"Rating: {feedback['rating']}")</pre>	Loop iterates over each feedback entry in the loaded data. Hence, ensure that each entry is a dictionary before attempting to access its fields. It may helps to avoid errors by skipping of handling invalid data. If the

<pre> print(f'Comments: {feedback['comments']}') print(f'Date: {feedback['date']}') print("-" * 20) </pre>	feedback entry is valid, the details of each feedback will be displayed.
<pre> else: print("Unexpected data format:", feedback) </pre>	Inform user when an unexpected data format is received.

Example of Input and Output

System Main Page

```
*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role: 1
*****
                        SIGN IN/SIGN UP
*****

Dear customer, do you have an account?
1. Yes, I would like to SIGN IN.
2. No, I would like to REGISTER for an account.
3. Exit
Please enter your choice: |
```

Figure : Sign up or Sign in for Customer

```
*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role: 2
*****
                        Admin Interface
*****
1. Login
2. Signup
3. Exit
Choose an option:
```

Figure : Sign up or Sign in for Admin

```
*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role: 3
*****
                        Super User Login
*****
Enter Super User ID:
|
```

Figure : Sign in for Super User

```
*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role: 4

*****
Inventory Staff Homepage
*****
1. Login
2. Signup
Please enter your option:|
```

Figure : Sign up or Sign in for Inventory

When the user selects the selected role by a specified number, will go to the specific page to choose sign up or sign in.

Customer Menu

```
*****
SIGN IN/SIGN UP
*****

Dear customer, do you have an account?
1. Yes, I would like to SIGN IN.
2. No, I would like to REGISTER for an account.
3. Exit
Please enter your choice: 1
*****
Customer Log In Page
*****

Enter Username: |
```

Figure : Login Page

```
*****
SIGN IN/SIGN UP
*****

Dear customer, do you have an account?
1. Yes, I would like to SIGN IN.
2. No, I would like to REGISTER for an account.
3. Exit
Please enter your choice: 2
*****
Create an account
*****

Create username (At least 4 characters and must contain only alphabetic character): |
```

Figure : Signup Page

```
*****
SIGN IN/SIGN UP
*****

Dear customer, do you have an account?
1. Yes, I would like to SIGN IN.
2. No, I would like to REGISTER for an account.
3. Exit
Please enter your choice: 3
*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role:
```

Figure : Exit

When customers go into the customer menu, there have three selection let them to choose, which are Sign In, Register and Exit. When they choose the selective number, will turn into the page they choose.

Super User Menu

```
*****
      Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 1
Add User

1. Username: staff2, Role: staff, Status: pending
Approve (A) or Reject (R) this user? |
```

Figure : Add User

```
1. Username: staff2, Role: staff, Status: pending
Approve (A) or Reject (R) this user? R
User staff2 rejected.
```

Figure : Add User

```
*****
      Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 2
Approve/Reject User Registration
1. User data format error: ['staff2', 'Staff@321', 'staff', 'pending']
2. Username: staff@3,
   Password: Staff@666,
   Identity Card: 9876543210,
   Role: staff,
   Status: pending
```

Figure : Approve/Reject User Registration

```
*****
      Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 6
Check Customer Purchase Order Status
You have no orders of this type.
```

Figure : Check Customer Purchase Order Status

```
*****
      Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 7
Check Customer Service Order Status
File service_orders.txt not found.
No service orders found.
```

Figure : Check Customer Service Order Status

```

*****
Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 8
Check Staff Purchase Order Status
No purchase orders found.

```

Figure : Check Staff Purchase Order Status

```

*****
Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 9
View Reports

Reports:
Super User ID: superuser
  Date      | Time |      Actions
- 2024-08-15 22:23:30: Login
- 2024-08-15 22:23:46: View Reports
- 2024-08-15 22:24:44: Logout
- 2024-08-15 22:28:48: Login

```

Figure : View Reports

```
*****
Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 10
View Feedback
Loaded feedback data: [{'username': 'abcd', 'rating': 3, 'comments': 'good', 'date': '2024-08-14 21:54:27'}, {'username': 'abcd', 'rating': 5, 'comments': 'good', 'date': '2024-08-14 21:57:00'}, {'username': 'abcd', 'rating': 5, 'comments': 'bagus ', 'date': '2024-08-14 21:57:11'}, {'username': 'Lily', 'rating': 4, 'comments': 'Good', 'date': '2024-08-16 05:13:26'}]
Username: abcd
Rating: 3
Comments: good
Date: 2024-08-14 21:54:27
-----
Username: abcd
Rating: 5
Comments: good
Date: 2024-08-14 21:57:00
-----
Username: abcd
Rating: 5
Comments: bagus
Date: 2024-08-14 21:57:11
-----
Username: Lily
Rating: 4
Comments: Good
Date: 2024-08-16 05:13:26
```

Figure : View Feedback

```
*****
Super User Menu
*****
1. Add User
2. Approve/Reject User Registrations
3. Modify User Personal Details
4. Disable User Access
5. Inquiry of User's System Usage
6. Check Customer Purchase Order Status
7. Check Customer Service Order Status
8. Check Staff Purchase Order Status
9. View Reports
10. View Feedback
11. Logout
Choose an option: 11
Logout

*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role:
```

Figure : Logout

When the super user login to the Super User Menu, there are 11 selections for super user to choose. Super user types serial number of the selections they want and it will turn into the specific page from the Super User menu.

Admin Menu

```
*****
      Admin Interface
*****
1. Login
2. Signup
3. Exit
Choose an option: 1
Login

*****
      Admin Registration Page
*****
Enter Username:
```

Figure : Approve/Reject User Registrations

```
*****
      Admin Interface
*****
1. Login
2. Signup
3. Exit
Choose an option: 2
Signup

*****
      Admin Registration Page
*****
Create username:
```

Figure :

Inventory Staff Menu

```
*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 1

-----Available Items for Inventory Staff-----
1. Desktop
   Quantity: 17
   Price: RM2500.00
2. Laptop
   Quantity: 20
   Price: RM4000.00
3. Mouse
   Quantity: 10
   Price: RM80.00
4. Keyboard
   Quantity: 20
   Price: RM150.00
5. Monitor
   Quantity: 10
   Price: RM500.00
6. Charger
   Quantity: 10
   Price: RM100.00
7. CPU
   Quantity: 10
   Price: RM500.00
Enter the item number to order (or 0 to finish): |
```

Figure : Place Purchase Order

```
-----Available Items for Inventory Staff-----
1. Desktop
   Quantity: 17
   Price: RM2500.00
2. Laptop
   Quantity: 20
   Price: RM4000.00
3. Mouse
   Quantity: 10
   Price: RM80.00
4. Keyboard
   Quantity: 20
   Price: RM150.00
5. Monitor
   Quantity: 10
   Price: RM500.00
6. Charger
   Quantity: 10
   Price: RM100.00
7. CPU
   Quantity: 10
   Price: RM500.00

Enter the item number to order (or 0 to finish): 3
Enter quantity for Mouse: 4

Stock updated: Mouse now has 14 units.

Enter the item number to order (or 0 to finish): 4
Enter quantity for Keyboard: 2

Stock updated: Keyboard now has 22 units.

Enter the item number to order (or 0 to finish): 0

Purchase Order Placed Successfully!
1. Mouse
   Quantity: 4
   Price: RM80.00
2. Keyboard
   Quantity: 2
   Price: RM150.00

Total Price: RM620.00
Status: Unpaid
```

Figure : Place Purchase Order

```
*****
Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 2
Purchase orders:

-----Order ID: 1-----
1. Mouse
   Quantity: 4
   Price per unit: RM80.00
2. Keyboard
   Quantity: 2
   Price per unit: RM150.00

Total Price: RM620.00
Status: Unpaid
Enter Order ID to modify: |
```

Figure : Modify Purchase Order

```

*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 3
Purchase orders:

-----Order ID: 1-----
1. Mouse
   Quantity: 4
   Price per unit: RM80.00
2. Keyboard
   Quantity: 2
   Price per unit: RM150.00

Total Price: RM620.00
Status: Unpaid
Enter order ID to cancel: |

```

Figure : Cancel Purchase Order

```

-----Order ID: 1-----
1. Mouse
   Quantity: 4
   Price per unit: RM80.00
2. Keyboard
   Quantity: 2
   Price per unit: RM150.00

Total Price: RM620.00
Status: Unpaid
Enter order ID to cancel: 1

Stock updated: Mouse now has 14 units.

Stock updated: Keyboard now has 22 units.
Order ID: 1 canceled.

```

Figure : Cancel Purchase Order

```
*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 4
No orders found.
```

Figure : Check Purchase Order Status

```

*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 5

```

Figure : Stock Check/Adjustments

```

Current stock:
1. Desktop
   Quantity: 17
   Price: RM2500.00
   Available for: Staff and Customer
2. Laptop
   Quantity: 20
   Price: RM4000.00
   Available for: Staff and Customer
3. Mouse
   Quantity: 14
   Price: RM80.00
   Available for: Staff and Customer
4. Keyboard
   Quantity: 22
   Price: RM150.00
   Available for: Staff and Customer
5. Monitor
   Quantity: 10
   Price: RM500.00
   Available for: Staff and Customer
6. Charger
   Quantity: 10
   Price: RM100.00
   Available for: Staff and Customer
7. CPU
   Quantity: 10
   Price: RM500.00
   Available for: Staff Only

Options:
1. Update Quantity
2. Update Price
3. Add New Item
4. Remove Item
5. Exit
Select an option (1-5):

```

Figure : Stock Check/Adjustments

```
Options:
1. Update Quantity
2. Update Price
3. Add New Item
4. Remove Item
5. Exit
Select an option (1-5): 1
Enter the item number to update quantity: |
```

Figure : Stock Check/Adjustments

```
Options:
1. Update Quantity
2. Update Price
3. Add New Item
4. Remove Item
5. Exit
Select an option (1-5): 2
Enter the item number to update price: |
```

Figure : Stock Check/Adjustments

```
*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 6
No purchase orders found for staff1. No payment needed.
```

Figure : Payment

```
Availabe Service/ Repairs:
1. Computer Repair : 3-5 days
   Price: RM250.00
2. Device Repair : 2-4 days
   Price: RM100.00
3. Software Installation : 1 days
   Price: RM200.00
4. Hardware Upgrade : 2-3 days
   Price: RM250.00
Enter Y to modify or N to exit. |
```

Figure : Payment


```
Availabe Service/ Repairs:
1. Computer Repair : 3-5 days
   Price: RM250.00
2. Device Repair : 2-4 days
   Price: RM100.00
3. Software Installation : 1 days
   Price: RM200.00
4. Hardware Upgrade : 2-3 days
   Price: RM250.00
Enter Y to modify or N to exit.N

*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option:
```

Figure : Payment

```

*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 8

Reports for staff1:
      Date      |   Time   |      Actions
- 2024-08-16 11:16:34: Login
- 2024-08-16 11:20:09: Make Purchase Order
- 2024-08-16 11:21:19: Check Purchase Order Status
- 2024-08-16 11:22:58: Login
- 2024-08-16 11:23:41: Check Purchase Order Status
- 2024-08-16 11:24:24: Cancel Purchase Order
- 2024-08-16 11:24:39: Check Purchase Order Status
- 2024-08-16 11:24:39: Login
- 2024-08-16 11:29:21: Stock Check - Update Quantity
- 2024-08-16 11:31:34: Make Payment

Press Enter to return to the main menu...

```

Figure : Report

```
*****
      Inventory Menu
*****
1. Place Purchase Order
2. Modify Purchase Order
3. Cancel Purchase Order
4. Check Purchase Order Status
5. Stock Check/Adjustments
6. Payment
7. Service/ Repairs
8. Report
9. View Service Order
10. Logout
Choose an option: 10

*****
Welcome to KL Central Computer Company (KLCCC) website!
*****
Please select a role to access to the system
1. Customer
2. Admin
3. Super User
4. Inventory
Please enter your role: |
```

Figure : Logout

When Inventory Staff login successfully from the main page, they will go to inventory staff menu. They are able to place purchase order, modify purchase order, cancel purchase order, check purchase order status, stock check or adjustments, payment, service or repairs, report, view service order and logout through typing the serial number.

Conclusion

In conclusion, we developed a robust system tailored to various requirements, focusing on managing business transactions among customers, admin, super user, and inventory staff. This system guarantees optimal performance for customers. This report discusses the assumptions made during the design phase of the system to strengthen system usability. Designing the Pseudocode and Flowchart of the system outlines the logical steps of problem-solving. Additionally, the program source code and additional features source code of the system are clearly explained. The sample input/output of the system is also demonstrated. We found that Programming with Python module is useful in assisting us to understand the importance of using pseudocode and flowchart to design a system.

References

McKinney, W. (2022). *Python for data analysis*. “O’Reilly Media, Inc.”. [Python for Data Analysis - Wes McKinney - Google Books](#)

Abdul Kadhar, K. M. & Anand, G. (2021). Basics of Python Programming. In: Data Science with Raspberry Pi, 13-47. https://doi.org/10.1007/978-1-4842-6825-4_2